

Modern C++ Design

*Generic Programming
and Design Patterns Applied*

Andrei Alexandrescu

Foreword by Scott Meyers

Foreword by John Vlissides



C++ In-Depth Series ♦ Bjarne Stroustrup

FREE SAMPLE CHAPTER



SHARE WITH OTHERS

Modern C++ Design

*Generic Programming
and Design Patterns Applied*

Andrei Alexandrescu

Foreword by Scott Meyers

Foreword by John Vlissides



C++ In-Depth Series ♦ Bjarne Stroustrup

FREE SAMPLE CHAPTER



SHARE WITH OTHERS

Modern C++ Design

现代 C++ 设计

The C++ In-Depth Series

Bjarne Stroustrup, Editor

"I have made this letter longer than usual, because I lack the time to make it short."

—BLAISE PASCAL

The advent of the ISO/ANSI C++ standard marked the beginning of a new era for C++ programmers. The standard offers many new facilities and opportunities, but how can a real-world programmer find the time to discover the key nuggets of wisdom within this mass of information? **The C++ In-Depth Series** minimizes learning time and confusion by giving programmers concise, focused guides to specific topics.

Each book in this series presents a single topic, at a technical level appropriate to that topic. The Series' practical approach is designed to lift professionals to their next level of programming skills. Written by experts in the field, these short, in-depth monographs can be read and referenced without the distraction of unrelated material. The books are cross-referenced within the Series, and also reference *The C++ Programming Language* by Bjarne Stroustrup.

As you develop your skills in C++, it becomes increasingly important to separate essential information from hype and glitz, and to find the in-depth content you need in order to grow. The C++ In-Depth Series provides the tools, concepts, techniques, and new approaches to C++ that will give you a critical edge.

Titles in the Series

Accelerated C++: Practical Programming by Example, Andrew Koenig and Barbara E. Moo

Applied C++: Practical Techniques for Building Better Software, Philip Romanik and Amy Muntz

The Boost Graph Library: User Guide and Reference Manual, Jeremy G. Siek, Lie-Quan Lee, and Andrew Lumsdaine

C++ Coding Standards: 101 Rules, Guidelines, and Best Practices, Herb Sutter and Andrei Alexandrescu

C++ In-Depth Box Set, Bjarne Stroustrup, Andrei Alexandrescu, Andrew Koenig, Barbara E. Moo, Stanley B. Lippman, and Herb Sutter

C++ Network Programming, Volume 1: Mastering Complexity with ACE and Patterns, Douglas C. Schmidt and Stephen D. Huston

C++ Network Programming, Volume 2: Systematic Reuse with ACE and Frameworks, Douglas C. Schmidt and Stephen D. Huston

C++ Template Metaprogramming: Concepts, Tools, and Techniques from Boost and Beyond, David Abrahams and Aleksey Gurtovoy

Essential C++, Stanley B. Lippman

Exceptional C++: 47 Engineering Puzzles, Programming Problems, and Solutions, Herb Sutter

Exceptional C++ Style: 40 New Engineering Puzzles, Programming Problems, and Solutions, Herb Sutter

Modern C++ Design: Generic Programming and Design Patterns Applied, Andrei Alexandrescu

More Exceptional C++: 40 New Engineering Puzzles, Programming Problems, and Solutions, Herb Sutter

For more information, check out the series web site at www.awprofessional.com/series/indepth/

C++ 深入系列

比亚恩·斯特鲁斯特鲁普，编辑

"I have made this letter longer than usual, because I lack the time to make it short."

—BLAISE PASCAL

ISO/ANSI C++ 标准的出现标志着 C++ 程序员新时代的开始。该标准提供了许多新的设施 and 机会，但现实世界的程序员如何才能找到时间在大量信息中发现关键的智慧呢？C++ 深入系列通过为程序员提供针对特定主题的简洁、集中的指南，最大限度地减少学习时间和困惑。

本系列中的每一本书都以适合该主题的技术水平呈现一个主题。该系列的实用方法旨在将专业人士的编程技能提升到新的水平。这些简短而深入的专著由该领域的专家撰写，可以在不干扰不相关材料的情况下阅读和参考。这些书在该系列中相互引用，还参考了 Bjarne Stroustrup 的 *The C++ Programming Language*。

随着您发展 C++ 技能，将基本信息与炒作和浮华区分开来并找到成长所需的深入内容变得越来越重要。C++ 深入系列提供了 C++ 的工具、概念、技术和新方法，将为您带来关键优势

。

系列中的标题

Accelerated C++: Practical Programming by Example, 安德鲁·科尼格和芭芭拉·穆

Applied C++: Practical Techniques for Building Better Software, 菲利普·罗曼尼克和艾米·蒙兹

The Boost Graph Library: User Guide and Reference Manual, Jeremy G. Siek, Lie-Quan Lee 和 Andrew Lumsdaine
C++ Coding Standards: 101 Rules, Guidelines, and Best Practices, Herb Sutter 和 Andrei Alexandrescu

C++ In-Depth Box Set, Bjarne Stroustrup, Andrei Alexandrescu, Andrew Koenig, Barbara E. Moo, Stanley B. Lippman 和 Herb Sutter

C++ Network Programming, Volume 1: Mastering Complexity with ACE and Patterns, 道格拉斯·C施密特和斯蒂芬·D休斯顿

C++ Network Programming, Volume 2: Systematic Reuse with ACE and Frameworks, 道格拉斯·C施密特和斯蒂芬·D休斯顿

C++ Template Metaprogramming: Concepts, Tools, and Techniques from Boost and Beyond, 大卫·亚伯拉罕和阿列克西·古尔托沃伊
Essential C++, 斯坦利·B利普曼

Exceptional C++: 47 Engineering Puzzles, Programming Problems, and Solutions, 赫伯·萨特

Exceptional C++ Style: 40 New Engineering Puzzles, Programming Problems, and Solutions, 赫伯·萨特

Modern C++ Design: Generic Programming and Design Patterns Applied, 安德烈·亚历山大雷斯库

More Exceptional C++: 40 New Engineering Puzzles, Programming Problems, and Solutions, 赫伯·萨特

Modern C++ Design

*Generic Programming
and Design Patterns Applied*

Andrei Alexandrescu

◆Addison-Wesley

Boston • San Francisco • New York • Toronto • Montreal

London • Munich • Paris • Madrid

Capetown • Sydney • Tokyo • Singapore • Mexico City

现代 C++ 设计

Generic Programming and Design Patterns Applied

安德烈·亚历山德雷斯库

◆ Addison-Wesley

Boston • San Francisco • New York • Toronto • Montreal

London • Munich • Paris • Madrid

Capetown • Sydney • Tokyo • Singapore • Mexico City

Many of the designations used by manufacturers and sellers to distinguish their products are claimed as trademarks. Where those designations appear in this book, and Addison-Wesley was aware of a trademark claim, the designations have been printed with initial capital letters or in all capitals.

The author and publisher have taken care in the preparation of this book, but make no expressed or implied warranty of any kind and assume no responsibility for errors or omissions. No liability is assumed for incidental or consequential damages in connection with or arising out of the use of the information or programs contained herein.

The publisher offers discounts on this book when ordered in quantity for special sales. For more information, please contact:

Pearson Education Corporate Sales Division
201 W. 103rd Street
Indianapolis, IN 46290
(800) 428-5331
corpsales@pearsoned.com

Visit AW on the Web: www.awprofessional.com

Library of Congress Cataloging-in-Publication Data
Alexandrescu, Andrei.

Modern C++ design : generic programming and design patterns applied /
Andrei Alexandrescu.

p. cm. — (C++ in depth series)

Includes bibliographical references and index.

ISBN 0-201-70431-5

1. C++ (Computer program language) 2. Generic programming (Computer science)

I. Title. II. Series.

QA76.73.C153 A42 2001

005.13'3—dc21

00-049596

Copyright © 2001 by Addison-Wesley

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form, or by any means, electronic, mechanical, photocopying, recording, or otherwise, without the prior consent of the publisher. Printed in the United States of America. Published simultaneously in Canada.

ISBN 0-201-70431-5

Text printed in the United States on recycled paper at RR Donnelley Crawfordsville in Crawfordsville, Indiana.

19th Printing January 2011

制造商和销售商用来区分其产品的许多名称都被称为商标。如果这些名称出现在本书中，并且 Addison-Wesley 知道商标声明，则这些名称均以首字母大写或全部大写印刷。

作者和出版商在本书的准备过程中非常谨慎，但没有做出任何形式的明示或暗示的保证，并且对错误或遗漏不承担任何责任。对于因使用此处包含的信息或程序而产生的偶然或间接损害，我们不承担任何责任。

出版商在特价销售中批量订购本书时会提供折扣。欲了解更多信息，请联系：

Pearson Education Corporate Sales Division 201
W. 103rd Street Indianapolis, IN 46290 (800) 42
8-5331 corpsales@pearsoned.com

访问 AW 网站：www.awprofessional.com

Library of Congress Cataloging-in-Publication Data

亚历山德雷斯库，安德烈。现代 C++ 设计：应用通用编程和设计模式 / Andrei Alexandrescu
。p. 厘米。—（深度系列中的 C++）包括参考文献和索引。ISBN 0-201-70431-5 1. C++ (计
算机程序语言) 2. 通用编程 (计算机科学) I. 标题。二. 系列。QA76.73.C153 A42 2001 005
.13'3—dc21

00-049596

版权所有 © 2001 Addison-Wesley

版权所有。未经出版商事先同意，不得以任何形式或任何方式（电子、机械、复印、录制或其他
方式）复制、存储或传播本出版物的任何部分。美国印刷。加拿大同步出版。

国际标准书号 0-201-70431-5

文本在美国印地安纳州克劳福兹维尔的 RR Donnelley Crawfordsville 的再生纸上印刷。

2011 年 1 月 19 日印刷

Contents

Foreword by Scott Meyers	xi
Foreword by John Vlissides	xv
Preface	xvii
Acknowledgments	xxi
Part I Techniques	1
Chapter 1 Policy-Based Class Design	3
1.1 The Multiplicity of Software Design	3
1.2 The Failure of the Do-It-All Interface	4
1.3 Multiple Inheritance to the Rescue?	5
1.4 The Benefit of Templates	6
1.5 Policies and Policy Classes	7
1.6 Enriched Policies	12
1.7 Destructors of Policy Classes	12
1.8 Optional Functionality Through Incomplete Instantiation	13
1.9 Combining Policy Classes	14
1.10 Customizing Structure with Policy Classes	16
1.11 Compatible and Incompatible Policies	17
1.12 Decomposing a Class into Policies	19
1.13 Summary	20

内容

斯科特·迈耶斯 xi 的前言

约翰·弗利赛德十五世的序言

前言十七

致谢二十一

Part I	Techniques	1
Chapter 1	Policy-Based Class Design	3
1.1	The Multiplicity of Software Design	3
1.2	The Failure of the Do-It-All Interface	4
1.3	Multiple Inheritance to the Rescue?	5
1.4	The Benefit of Templates	6
1.5	Policies and Policy Classes	7
1.6	Enriched Policies	12
1.7	Destructors of Policy Classes	12
1.8	Optional Functionality Through Incomplete Instantiation	13
1.9	Combining Policy Classes	14
1.10	Customizing Structure with Policy Classes	16
1.11	Compatible and Incompatible Policies	17
1.12	Decomposing a Class into Policies	19
1.13	Summary	20

Chapter 2	Techniques	23
2.1	Compile-Time Assertions	23
2.2	Partial Template Specialization	26
2.3	Local Classes	28
2.4	Mapping Integral Constants to Types	29
2.5	Type-to-Type Mapping	31
2.6	Type Selection	33
2.7	Detecting Convertibility and Inheritance at Compile Time	34
2.8	A Wrapper Around <code>type_info</code>	37
2.9	<code>NullType</code> and <code>EmptyType</code>	39
2.10	Type Traits	40
2.11	Summary	46
Chapter 3	Typelists	49
3.1	The Need for Typelists	49
3.2	Defining Typelists	51
3.3	Linearizing Typelist Creation	52
3.4	Calculating Length	53
3.5	Intermezzo	54
3.6	Indexed Access	55
3.7	Searching Typelists	56
3.8	Appending to Typelists	57
3.9	Erasing a Type from a Typelist	58
3.10	Erasing Duplicates	59
3.11	Replacing an Element in a Typelist	60
3.12	Partially Ordering Typelists	61
3.13	Class Generation with Typelists	64
3.14	Summary	74
3.15	<code>Typelist</code> Quick Facts	75
Chapter 4	Small-Object Allocation	77
4.1	The Default Free Store Allocator	78
4.2	The Workings of a Memory Allocator	78
4.3	A Small-Object Allocator	80
4.4	Chunks	81
4.5	The Fixed-Size Allocator	84
4.6	The <code>SmallObjAllocator</code> Class	87
4.7	A Hat Trick	89
4.8	Simple, Complicated, Yet Simple in the End	92
4.9	Administrivia	93
4.10	Summary	94
4.11	Small-Object Allocator Quick Facts	94

第 2 章 技术 23 2.1 编译时断言 23 2.2 部分模板特化 26 2.3 局部类 28 2.4 将整型常量映射到类型 29 2.5 类型到类型的映射 31 2.6 类型选择 33 2.7 在编译时检测可转换性和继承 34 2.8 包装器 type_info 37 2.9 nullptr 和 EmptyType 39 2.10 类型特征 40 2.11 摘要 46

第三章 类型列表 49

3.1 对类型列表的需求 49

3.2 定义类型列表 51

3.3 线性化类型列表创建 52

3.4 计算长度 53

3.5 间奏曲 54

3.6 索引访问 55

3.7 搜索类型列表 56

3.8 附加到类型列表 57

3.9 从类型列表中删除类型 58

3.10 删除重复项 59

3.11 替换类型列表中的元素 60

3.12 部分排序类型列表 61

3.13 使用类型列表生成类 64

3.14 概括 74

3.15 类型列表 要闻速览 75

第 4 章 小对象分配 77 4.1 默认自由存储分配器 78 4.2 内存分配器的工作原理 78 4.3 小对象分配器 80 4.4 块 81 4.5 固定大小分配器 84 4.6 SmallObjAllocator 类 87 4.7 帽子戏法 89 4.8 简单，复杂，但最终很简单 92 4.9 管理 93 4.10 总结 94 4.11 小对象分配器速览 94

Part II	Components	97
Chapter 5	Generalized Functors	99
5.1	The Command Design Pattern	100
5.2	Command in the Real World	102
5.3	C++ Callable Entities	103
5.4	The Functor Class Template Skeleton	104
5.5	Implementing the Forwarding Functor::operator()	108
5.6	Handling Functors	110
5.7	Build One, Get One Free	112
5.8	Argument and Return Type Conversions	114
5.9	Handling Pointers to Member Functions	115
5.10	Binding	119
5.11	Chaining Requests	122
5.12	Real-World Issues I: The Cost of Forwarding Functions	122
5.13	Real-World Issues II: Heap Allocation	124
5.14	Implementing Undo and Redo with Functor	125
5.15	Summary	126
5.16	Functor Quick Facts	126
Chapter 6	Implementing Singletons	129
6.1	Static Data + Static Functions != Singleton	130
6.2	The Basic C++ Idioms Supporting Singleton	131
6.3	Enforcing the Singleton's Uniqueness	132
6.4	Destroying the Singleton	133
6.5	The Dead Reference Problem	135
6.6	Addressing the Dead Reference Problem (I): The Phoenix Singleton	137
6.7	Addressing the Dead Reference Problem (II): Singletons with Longevity	139
6.8	Implementing Singletons with Longevity	142
6.9	Living in a Multithreaded World	145
6.10	Putting It All Together	148
6.11	Working with SingletonHolder	153
6.12	Summary	155
6.13	SingletonHolder Class Template Quick Facts	155
Chapter 7	Smart Pointers	157
7.1	Smart Pointers 101	157
7.2	The Deal	158
7.3	Storage of Smart Pointers	160
7.4	Smart Pointer Member Functions	161

第二部分组件 97 第 5 章

- 广义函子 99
- 5.1 命令设计模式 100
- 5.2 现实世界中的命令 102
- 5.3 C++ 可调用实体 103
- 5.4 Functor 类模板骨架 104
- 5.5 实现转发函子::operator() 108
- 5.6 处理函子 110
- 5.7 建一送一 112
- 5.8 参数和返回类型转换 114
- 5.9 处理成员函数指针 115
- 5.10 绑定 119
- 5.11 链接请求 122
- 5.12 现实世界问题 I: 转发功能的成本 122
- 5.13 现实世界问题 II: 堆分配 124
- 5.14 使用 Functor 实现撤消和重做 125
- 5.15 总结 126
- 5.16 函子速览 126

- 第6章 实现单例 129
 - 6.1 静态数据 + 静态函数 != 单例 130
 - 6.2 支持 Singleton 的基本 C++ 习惯用法 131
 - 6.3 强化单例的唯一性 132
 - 6.4 摧毁单例 133
 - 6.5 死亡参考问题 135
 - 6.6 解决死引用问题（一）：Phoenix Singleton
 - 6.7 解决死亡引用问题（二）：长寿的单例 139
 - 6.8 实现长寿的单例 142
 - 6.9 生活在多线程世界 145
 - 6.10 第148章
 - 6.11 使用 SingletonHolder 153
 - 6.12 总结 155
 - 6.13 SingletonHolder 类模板速览 155

第7章智能指针157 7.1

- 智能指针 101 157
- 7.2 交易 158
- 7.3 智能指针的存储 160
- 7.4 智能指针成员函数 161

7.5	Ownership-Handling Strategies	163
7.6	The Address-of Operator	170
7.7	Implicit Conversion to Raw Pointer Types	171
7.8	Equality and Inequality	173
7.9	Ordering Comparisons	178
7.10	Checking and Error Reporting	181
7.11	Smart Pointers to <code>const</code> and <code>const</code> Smart Pointers	182
7.12	Arrays	183
7.13	Smart Pointers and Multithreading	184
7.14	Putting It All Together	187
7.15	Summary	194
7.16	<code>SmartPtr</code> Quick Facts	194
Chapter 8	Object Factories	197
8.1	The Need for Object Factories	198
8.2	Object Factories in C++: Classes and Objects	200
8.3	Implementing an Object Factory	201
8.4	Type Identifiers	206
8.5	Generalization	207
8.6	Minutiae	210
8.7	Clone Factories	211
8.8	Using Object Factories with Other Generic Components	215
8.9	Summary	216
8.10	Factory Class Template Quick Facts	216
8.11	<code>CloneFactory</code> Class Template Quick Facts	217
Chapter 9	Abstract Factory	219
9.1	The Architectural Role of Abstract Factory	219
9.2	A Generic Abstract Factory Interface	223
9.3	Implementing <code>AbstractFactory</code>	226
9.4	A Prototype-Based Abstract Factory Implementation	228
9.5	Summary	233
9.6	<code>AbstractFactory</code> and <code>ConcreteFactory</code> Quick Facts	233
Chapter 10	Visitor	235
10.1	Visitor Basics	235
10.2	Overloading and the Catch-All Function	242
10.3	An Implementation Refinement: The Acyclic Visitor	243
10.4	A Generic Implementation of Visitor	248
10.5	Back to the “Cyclic” Visitor	255
10.6	Hooking Variations	258
10.7	Summary	260
10.8	<code>Visitor</code> Generic Component Quick Facts	261

7.5 所有权处理策略 163 7.6 运算符地址 170 7.7 到原始指针类型的隐式转换 171 7.8 相等和不相等 173 7.9 排序比较 178 7.10 检查和错误报告 181 7.11 指向 const 的智能指针和 const 智能指针 182 7.12 数组 183 7.13 智能指针和多线程 184 7.14 综合 187 7.15 总结 194 7.16 SmartPtr 概览 194

第 8 章 对象工厂 197 8.1 对象工厂的需求 198 8.2 C++ 中的对象工厂：类和对象 200 8.3 实现对象工厂 201 8.4 类型标识符 206 8.5 泛化 207 8.6 细节 210 8.7 克隆工厂 211 8.8 使用对象工厂其他通用组件 215 8.9 摘要 216 8.10 Factory 类模板速览 216 8.11 CloneFactory 类模板速览 217

第 9 章 抽象工厂 219 9.1 抽象工厂的架构角色 219 9.2 通用抽象工厂接口 223 9.3 实现 AbstractFactory 226 9.4 基于原型的抽象工厂实现 228 9.5 总结 233 9.6 AbstractFactory 和 ConcreteFactory 概览 233

第 10 章 访问者 235 10.1 访问者基础知识 235 10.2 重载和 Catch-All 函数 242 10.3 实现改进：非循环访问者 243 10.4 访问者的通用实现 248 10.5 回到“循环”访问者 255 10.6 挂钩变体 258 10.7 总结 260 10.8 访客通用组件速览 261

Chapter 11	Multimethods	263
11.1	What Are Multimethods?	264
11.2	When Are Multimethods Needed?	264
11.3	Double Switch-on-Type: Brute Force	265
11.4	The Brute-Force Approach Automated	268
11.5	Symmetry with the Brute-Force Dispatcher	273
11.6	The Logarithmic Double Dispatcher	276
11.7	FnDispatcher and Symmetry	282
11.8	Double Dispatch to Functors	282
11.9	Converting Arguments: <code>static_cast</code> or <code>dynamic_cast</code> ?	285
11.10	Constant-Time Multimethods: Raw Speed	290
11.11	BasicDispatcher and BasicFastDispatcher as Policies	293
11.12	Looking Forward	294
11.13	Summary	296
11.14	Double Dispatcher Quick Facts	297
Appendix	A Minimalist Multithreading Library	301
A.1	A Critique of Multithreading	302
A.2	Loki's Approach	303
A.3	Atomic Operations on Integral Types	303
A.4	Mutexes	305
A.5	Locking Semantics in Object-Oriented Programming	306
A.6	Optional <code>volatile</code> Modifier	308
A.7	Semaphores, Events, and Other Good Things	309
A.8	Summary	309
Bibliography		311
Index		313

第 11 章多重方法	263
11.1 什么是多重方法?	264
11.2 什么时候需要多种方法?	264
11.3 双开启类型: 蛮力	265
11.4 蛮力方法自动化	268
11.5 蛮力调度器的对称性	273
11.6 对数双调度器	276
11.7 FnDispatcher 和对称性	282
11.8 到函子的双重调度	282
11.9 转换参数: static_cast 还是dynamic_cast?	285
11.10 恒定时间多方法: 原始速度	290
11.11 BasicDispatcher 和 BasicFastDispatcher 作为策略	293
11.12 展望	294
11.13 总结	296
11.14 双调度程序速览	297

附录 A 极简多线程库	301
A.1 对多线程的批判	3
A.2 洛基的方法	3
A.3 整数类型的原子操作	3
A.4 互斥体	3
A.5 面向对象编程中的锁定语义	3
A.6 可选的 volatile 修饰符	3
A.7 信号量、事件和其他好东西	3
A.8 总结	3
参考书目	311
指数	313

This page intentionally left blank

This page intentionally left blank

Foreword

by Scott Meyers

In 1991, I wrote the first edition of *Effective C++*. The book contained almost no discussions of templates, because templates were such a recent addition to the language, I knew almost nothing about them. What little template code I included, I had verified by e-mailing it to other people, because none of the compilers to which I had access offered support for templates.

In 1995, I wrote *More Effective C++*. Again I wrote almost nothing about templates. What stopped me this time was neither a lack of knowledge of templates (my initial outline for the book included an entire chapter on the topic) nor shortcomings on the part of my compilers. Instead, it was a suspicion that the C++ community's understanding of templates was about to undergo such dramatic change, anything I had to say about them would soon be considered trite, superficial, or just plain wrong.

There were two reasons for that suspicion. The first was a column by John Barton and Lee Nackman in the January 1995 C++ *Report* that described how templates could be used to perform typesafe dimensional analysis with zero runtime cost. This was a problem I'd spent some time on myself, and I knew that many had searched for a solution, but none had succeeded. Barton and Nackman's revolutionary approach made me realize that templates were good for a lot more than just creating containers of T.

As an example of their design, consider this code for multiplying two physical quantities of arbitrary dimensional type:

```
template<int m1, int l1, int t1, int m2, int l2, int t2>
Physical<m1+m2, l1+l2, t1+t2> operator*(Physical<m1, l1, t1> lhs,
                                         Physical<m2, l2, t2> rhs)
{
    return Physical<m1+m2, l1+l2, t1+t2>::unit*lhs.value()*rhs.value();
}
```

Even without the context of the column to clarify this code, it's clear that this function template takes six parameters, none of which represents a type! This use of templates was such a revelation to me, I was positively giddy.

Shortly thereafter, I started reading about the STL. Alexander Stepanov's elegant library design, where containers know nothing about algorithms; algorithms know nothing about

前言

作者：斯科特·迈耶斯

1991 年，我写了第一版 *Effective C++*。这本书几乎没有讨论模板，因为模板是该语言最近才添加的内容，我对它们几乎一无所知。我通过电子邮件发送给其他人来验证了我所包含的少量模板代码，因为我有权访问的编译器都不提供对模板的支持。

1995 年，我写了 *More Effective C++*。我再次几乎没有写任何关于模板的内容。这次阻止我的既不是缺乏模板知识（我的书的最初大纲包括了关于该主题整个章节），也不是我的编译器的缺点。相反，人们怀疑 C++ 社区对模板的理解即将发生如此巨大的变化，我对它们所说的任何话很快就会被认为是陈词滥调、肤浅的，或者完全是错误的。

这种怀疑有两个原因。第一篇是 John Barton 和 Lee Nackman 在 1995 年 1 月的 C++ *Report* 中发表的专栏，描述了如何使用模板以零运行时间成本执行类型安全的维度分析。这是我自己花了一些时间解决的一个问题，我知道很多人都在寻找解决方案，但没有一个成功。Barton 和 Nackman 的革命性方法让我意识到模板的用途不仅仅是创建 T 容器。

作为其设计的一个示例，请考虑以下用于将任意维度类型的两个物理量相乘的代码：

```
模板<int m1, int l1, int t1, int m2, int l2, int t2> 物理<m1+m2, l1+l2, t1+t2> 运算符*(物理<m1, l1, t1  
> lhs, 物理<m2, l2, t2> rhs) { return 物理<m1+m2, l1+l2, t1+t2>::unit*lhs.value()*rhs.value(); }
```

即使没有专栏上下文来阐明此代码，很明显此函数模板需要六个参数，其中没有一个代表类型！模板的使用对我来说是一种启示，我简直头晕目眩。

此后不久，我开始阅读有关 STL 的内容。Alexander Stepanov 优雅的库设计，容器对算法一无所知；算法一无所知

containers; iterators act like pointers (but may be objects instead); containers and algorithms accept function pointers and function objects with equal aplomb; and library clients may extend the library without having to inherit from any base classes or redefine any virtual functions, made me feel—as I had when I read Barton and Nackman’s work—like I knew almost *nothing* about templates.

So I wrote almost nothing about them in *More Effective C++*. How could I? My understanding of templates was still at the containers-of-T stage, while Barton, Nackman, Stepanov, and others were demonstrating that such uses barely scratched the surface of what templates could do.

In 1998, Andrei Alexandrescu and I began an e-mail correspondence, and it was not long before I recognized that I was again about to modify my thinking about templates. Where Barton, Nackman, and Stepanov had stunned me with what templates could *do*, however, Andrei’s work initially made more of an impression on me for *how* it did what it did.

One of the simplest things he helped popularize continues to be the example I use when introducing people to his work. It’s the CTAssert template, analogous in use to the assert macro, but applied to conditions that can be evaluated during compilation. Here it is:

```
template<bool> struct CTAssert;
template<> struct CTAssert<true> {};
```

That’s it. Notice how the general template, CTAssert, is never defined. Notice how there is a specialization for true, but not for false. In this design, what’s *missing* is at least as important as what’s present. It makes you look at template code in a new way, because large portions of the “source code” are deliberately omitted. That’s a very different way of thinking from the one most of us are used to. (In this book, Andrei discusses the more sophisticated CompileTimeChecker template instead of CTAssert.)

Eventually, Andrei turned his attention to the development of template-based implementations of popular language idioms and design patterns, especially the GoF* patterns. This led to a brief skirmish with the Patterns community, because one of their fundamental tenets is that patterns cannot be represented in code. Once it became clear that Andrei was automating the generation of pattern *implementations* rather than trying to encode patterns themselves, that objection was removed, and I was pleased to see Andrei and one of the GoF (John Vlissides) collaborate on two columns in the C++ *Report* focusing on Andrei’s work.

In the course of developing the templates to generate idiom and pattern implementations, Andrei was forced to confront the variety of design decisions that all implementers face. Should the code be thread safe? Should auxiliary memory come from the heap, from the stack, or from a static pool? Should smart pointers be checked for nullness prior to dereferencing? What should happen during program shutdown if one Singleton’s destructor tries to use another Singleton that’s already been destroyed? Andrei’s goal was to offer his clients all possible design choices while mandating none.

*“GoF” stands for “Gang of Four” and refers to Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides, authors of the definitive book on patterns, *Design Patterns: Elements of Reusable Object-Oriented Software* (Addison-Wesley, 1995).

容器；迭代器的作用类似于指针（但也可以是对象）；容器和算法同样泰然自若地接受函数指针和函数对象；并且库客户端可以扩展库，而无需从任何基类继承或重新定义任何虚拟函数，这让我感觉——就像我在阅读 Barton 和 Nackman 的著作时所感受到的那样——就像我几乎了解模板一样。

所以我在 *More Effective C++* 中几乎没有写任何关于它们的内容。我怎么能呢？我对模板的理解还停留在 T 容器阶段，而 Barton、Nackman、Stepanov 和其他人正在证明这种用法仅仅触及了模板功能的表面。

1998 年，Andrei Alexandrescu 和我开始通过电子邮件通信，不久之后我就意识到我将再次修改我对模板的想法。Barton、Nackman 和 Stepanov 的模板功能令我震惊 *do*，然而，Andrei 的作品最初给我留下了更多的印象，因为 *how* 它做了它所做的事情。

他帮助普及的最简单的事情之一仍然是我在向人们介绍他的作品时使用的例子。它是 C TAssert 模板，与断言宏的使用类似，但应用于可以在编译期间评估的条件。这里是：

```
模板<布尔> 结构 CTAssert;模板<>结构CTAssert<true>{};
```

就是这样。请注意通用模板 CTAssert 从未被定义。请注意 *true* 的专业化，但 *false* 没有专业化。在这个设计中，*missing* 的内容至少与当前的内容一样重要。它让你以一种新的方式看待模板代码，因为大部分“源代码”被故意省略。这是一种与我们大多数人习惯的思维方式截然不同的思维方式。（在本书中，Andrei 讨论了更复杂的 *CompileTimeChecker* 模板，而不是 CTAssert。）

最终，Andrei 将注意力转向流行语言习惯和设计模式的基于模板实现的开发，尤其是 GoF* 模式。这导致了与模式社区的短暂冲突，因为他们的基本原则之一是模式不能用代码表示。一旦发现 Andrei 正在自动生成模式 *implementations*，而不是尝试对模式本身进行编码，这种反对意见就被消除了，我很高兴看到 Andrei 和一位 GoF (John Vlissides) 在 *C++ Report* 中合作撰写了两篇专栏文章，重点关注 Andrei 的工作。

在开发模板以生成习惯用法和模式实现的过程中，Andrei 被迫面对所有实现者面临的各种设计决策。代码应该是线程安全的吗？辅助内存应该来自堆、堆栈还是静态池？在取消引用之前是否应该检查智能指针是否为空？如果一个 Singleton 的析构函数试图使用另一个已经被销毁的 Singleton，那么在程序关闭期间会发生什么？安德烈的目标是为客户提供所有可能的设计选择，但不强制任何设计选择。

*“GoF” stands for “Gang of Four” and refers to Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides, authors of the definitive book on patterns, *Design Patterns: Elements of Reusable Object-Oriented Software* (Addison-Wesley, 1995).

His solution was to encapsulate such decisions in the form of *policy classes*, to allow clients to pass policy classes as template parameters, and to provide reasonable default values for such classes so that most clients could ignore them. The results can be astonishing. For example, the Smart Pointer template in this book takes only 4 policy parameters, but it can generate over 300 different smart pointer types, each with unique behavioral characteristics! Programmers who are content with the default smart pointer behavior, however, can ignore the policy parameters, specify only the type of object pointed to by the smart pointer, and reap the benefits of a finely crafted smart pointer class with virtually no effort.

In the end, this book tells three different technical stories, each compelling in its own way. First, it offers new insights into the power and flexibility of C++ templates. (If the material on typelists doesn't knock your socks off, it's got to be because you're already barefoot.) Second, it identifies orthogonal dimensions along which idiom and pattern implementations may differ. This is critical information for template designers and pattern implementers, but you're unlikely to find this kind of analysis in most idiom or pattern descriptions. Finally, the source code to Loki (the template library described in this book) is available for free download, so you can study Andrei's implementation of the templates corresponding to the idioms and patterns he discusses. Aside from providing a nice stress test for your compilers' support for templates, this source code serves as an invaluable starting point for templates of your own design. Of course, it's also perfectly respectable (and completely legal) to use Andrei's code right out of the box. I know he'd want you to take advantage of his efforts.

From what I can tell, the template landscape is changing almost as quickly now as it was in 1995 when I decided to avoid writing about it. At the rate things continue to develop, I may *never* write about templates. Fortunately for all of us, some people are braver than I am. Andrei is one such pioneer. I think you'll get a lot out of his book. I did.

Scott Meyers
September 2000

他的解决方案是将此类决策封装为 *policy classes* 的形式，以允许客户端将策略类作为模板参数传递，并为此类类提供合理的默认值，以便大多数客户端可以忽略它们。结果可能会令人惊讶。例如，本书中的智能指针模板仅需要 4 个策略参数，但它可以生成 300 多种不同的智能指针类型，每种类型都有独特的行为特征！然而，满足于默认智能指针行为的程序员可以忽略策略参数，仅指定智能指针所指向的对象类型，并几乎不费吹灰之力地获得精心设计的智能指针类的好处。

最后，本书讲述了三个不同的技术故事，每个故事都以自己的方式引人入胜。首先，它提供了对 C++ 模板的强大功能和灵活性的新见解。（如果类型列表上的材料没有让你大吃一惊，那一定是因为你已经赤脚了。）其次，它确定了习惯用法和模式实现可能不同的正交维度。这对于模板设计者和模式实现者来说是至关重要的信息，但你不太可能在大多数习语或模式描述中找到这种分析。最后，Loki（本书中描述的模板库）的源代码可以免费下载，因此您可以研究 Andrei 对与他讨论的习语和模式相对应的模板的实现。除了为编译器对模板的支持提供良好的压力测试之外，此源代码还可以作为您自己设计的模板的宝贵起点。当然，开箱即用地使用 Andrei 的代码也是完全值得尊重的（并且完全合法的）。我知道他希望你能利用他的努力。

据我所知，模板格局现在的变化速度几乎与 1995 年我决定避免撰写相关文章时一样快。按照事情继续发展的速度，我可能会写一些关于模板的文章。对我们所有人来说幸运的是，有些人比我更勇敢。安德烈就是这样的先驱之一。我想你会从他的书中得到很多东西。我做到了。

Scott Meyers
September 2000

This page intentionally left blank

This page intentionally left blank

Foreword

by John Vlissides

What's left to say about C++ that hasn't already been said? Plenty, it turns out. This book documents a convergence of programming techniques—generic programming, template metaprogramming, object-oriented programming, and design patterns—that are well understood in isolation but whose synergies are only beginning to be appreciated. These synergies have opened up whole new vistas for C++, not just for programming but for software design itself, with profound implications for software analysis and architecture as well.

Andrei's generic components raise the level of abstraction high enough to make C++ begin to look and feel like a design specification language. Unlike dedicated design languages, however, you retain the full expressiveness and familiarity of C++. Andrei shows you how to program in terms of design concepts: singletons, visitors, proxies, abstract factories, and more. You can even vary implementation trade-offs through template parameters, with positively no runtime overhead. And you don't have to blow big bucks on new development tools or learn reams of methodological mumbo jumbo. All you need is a trusty, late-model C++ compiler—and this book.

Code generators have held comparable promise for years, but my own research and practical experience have convinced me that, in the end, code generation doesn't compare. You have the round-trip problem, the not-enough-code-worth-generating problem, the inflexible-generator problem, the inscrutable-generated-code problem, and of course the I-can't-integrate-the-bloody-generated-code-with-my-own-code problem. Any one of these problems may be a showstopper; together, they make code generation an unlikely solution for most programming challenges.

Wouldn't it be great if we could realize the theoretical benefits of code generation—quicker, easier development, reduced redundancy, fewer bugs—without the drawbacks? That's what Andrei's approach promises. Generic components implement good designs in easy-to-use, mixable-and-matchable templates. They do pretty much what code generators do: produce boilerplate code for compiler consumption. The difference is that they do it within C++, not apart from it. The result is seamless integration with application code.

前言

作者：约翰·弗利塞兹

关于 C++ 还有什么没有说的呢？事实证明，很多。本书记录了编程技术的融合——泛型编程、模板元编程、面向对象编程和设计模式——这些技术单独来看可以很好地理解，但它们的协同作用才刚刚开始被认识到。这些协同作用为 C++ 开辟了全新的前景，不仅对于编程，而且对于软件设计本身，也对软件分析和架构产生了深远的影响。

Andrei 的通用组件将抽象级别提高到足够高，使 C++ 开始看起来像一种设计规范语言。然而，与专用设计语言不同的是，您保留了 C++ 的完整表现力和熟悉性。Andrei 向您展示如何根据设计概念进行编程：单例、访问者、代理、抽象工厂等等。您甚至可以通过模板参数来改变实现权衡，并且绝对不会产生运行时开销。而且您不必花大钱购买新的开发工具或学习大量方法论的胡言乱语。您所需要的只是一个值得信赖的新型 C++ 编译器以及这本书。

多年来，代码生成器一直具有可比性，但我自己的研究和实践经验让我相信，最终，代码生成器无法与之相比。你会遇到往返问题、生成代码不足的问题、生成器不灵活的问题、生成代码难以理解的问题，当然还有我无法将生成的代码与我自己的代码集成的问题。这些问题中的任何一个都可能成为一个大问题。它们共同使得代码生成成为解决大多数编程挑战的不可能的解决方案。

如果我们能够实现代码生成的理论上的好处——更快、更容易的开发、减少冗余、更少的错误——而没有缺点，那不是很好吗？这就是安德烈的方法所承诺的。通用组件在易于使用、可混合和匹配的模板中实现良好的设计。它们的作用与代码生成器的作用非常相似：生成供编译器使用的样板代码。不同之处在于它们是在 C++ 内完成的，而不是脱离它。结果是与应用程序代码无缝集成。

You can also use the full power of the language to extend, override, and otherwise tweak the designs to suit your needs.

Some of the techniques herein are admittedly tricky to grasp, especially the template metaprogramming in Chapter 3. Once you've mastered that, however, you'll have a solid foundation for the edifice of generic componentry, which almost builds itself in the ensuing chapters. In fact, I would argue that the metaprogramming material of Chapter 3 alone is worth the book's price—and there are ten other chapters full of insights to profit from. "Ten" represents an order of magnitude. Even so, the return on your investment will be far greater.

John Vlissides
IBM T.J. Watson Research
September 2000

您还可以使用该语言的全部功能来扩展、覆盖或以其他方式调整设计以满足您的需求。

诚然，本文中的一些技术很难掌握，尤其是第 3 章中的模板元编程。但是，一旦掌握了这些技术，您将为通用组件的大厦奠定坚实的基础，而通用组件的大厦几乎将在接下来的章节中自行构建。事实上，我认为仅第三章的元编程材料就值这本书的价格——而且还有其
他十章充满了可以从中获利的见解。“十”代表一个数量级。即便如此，您的投资回报率也会高得多。

约翰·弗利赛德
IBM T.J.沃森研究中心 20
00 年 9 月

Preface

You might be holding this book in a bookstore, asking yourself whether you should buy it. Or maybe you are in your employer's library, wondering whether you should invest time in reading it. I know you don't have time, so I'll cut to the chase. If you have ever asked yourself how to write higher-level programs in C++, how to cope with the avalanche of irrelevant details that plague even the cleanest design, or how to build reusable components that you don't have to hack into each time you take them to your next application, then this book is for you.

Imagine the following scenario. You come from a design meeting with a couple of printed diagrams, scribbled with your annotations. Okay, the event type passed between these objects is not `char` anymore; it's `int`. You change one line of code. The smart pointers to `Widget` are too slow; they should go unchecked. You change one line of code. The object factory needs to support the new `Gadget` class just added by another department. You change one line of code.

You have changed the design. Compile. Link. Done.

Well, there is something wrong with this scenario, isn't there? A much more likely scenario is this: You come from the meeting in a hurry because you have a pile of work to do. You fire a global search. You perform surgery on code. You add code. You introduce bugs. You remove the bugs . . . that's the way a programmer's job is, right? Although this book cannot possibly promise you the first scenario, it is nonetheless a resolute step in that direction. It tries to present C++ as a newly discovered language for software architects.

Traditionally, code is the most detailed and intricate aspect of a software system. Historically, in spite of various levels of language support for design methodologies (such as object orientation), a significant gap has persisted between the blueprints of a program and its code because the code must take care of the ultimate details of the implementation and of many ancillary tasks. The intent of the design is, more often than not, dissolved in a sea of quirks.

This book presents a collection of reusable design artifacts, called *generic components*, together with the techniques that make them possible. These generic components bring their users the well-known benefits of libraries, but in the broader space of system architecture. The coding techniques and the implementations provided focus on tasks and issues

前言

您可能在书店里拿着这本书，问自己是否应该购买它。或者，也许您在雇主的图书馆里，想知道是否应该花时间阅读它。我知道你没有时间，所以我就切入正题。如果您曾经问过自己如何用 C++ 编写更高级别的程序，如何应对大量不相关的细节（这些细节甚至困扰着最简洁的设计），或者如何构建可重用的组件，而不必在每次将它们带到下一个应用程序时都进行攻击，那么这本书就适合您。

想象一下以下场景。您参加完设计会议后，手里拿着几张打印的图表，上面潦草地写着您的注释。好吧，这些对象之间传递的事件类型不再是 `char` 了；它是整数。您更改一行代码。`Widget` 的智能指针太慢；他们应该不受控制。您更改一行代码。对象工厂需要支持另一个部门刚刚添加的新 `Gadget` 类。您更改一行代码。

你改变了设计。编译。关联。完毕。

嗯，这个场景有问题，不是吗？更可能的情况是这样的：你匆忙结束会议，因为你有一大堆工作要做。您进行全局搜索。您对代码进行手术。你添加代码。你引入了错误。您删除错误。。。这就是程序员的工作，对吧？尽管本书不可能向您承诺第一种情况，但它仍然是朝着这个方向迈出的坚定一步。它试图将 C++ 呈现为软件架构师新发现的语言。

传统上，代码是软件系统最详细、最复杂的方面。从历史上看，尽管设计方法（例如面向对象）有不同级别的语言支持，但程序的蓝图与其代码之间仍然存在显着的差距，因为代码必须处理实现和许多辅助任务的最终细节。设计的目的往往是溶解在大量的怪癖中。

本书介绍了一系列可重用的设计工件（称为 *generic components*，）以及使它们成为可能的技术。这些通用组件为用户带来了众所周知的库的好处，而且是在更广泛的系统架构空间中。提供的编码技术和实现侧重于任务和问题

that traditionally fall in the area of design, activities usually done *before* coding. Because of their high level, generic components make it possible to map intricate architectures to code in unusually expressive, terse, and easy-to-maintain ways.

Three elements are reunited here: design patterns, generic programming, and C++. These elements are combined to achieve a very high rate of reuse, both in the horizontal and vertical “markets” of software design. In the horizontal dimension, a small amount of library code implements a combinatorial—and essentially open-ended—number of structures and behaviors. On the vertical dimension, the generality of these components makes them applicable to a vast range of programs.

This book owes much to design patterns, powerful solutions to ever-recurring problems in object-oriented development. Design patterns are distilled pieces of good design—recipes for sound, reusable solutions to problems that can be encountered in many contexts. Design patterns concentrate on providing a suggestive lexicon for designs to be conveyed. They describe the problem, a time-proven solution with its variants, and the consequences of choosing each variant of that solution. Design patterns go above and beyond anything a programming language, no matter how advanced, could possibly express. By following and combining certain design patterns, the components presented in this book tend to address a large category of concrete problems.

Generic programming is a paradigm that focuses on abstracting types to a narrow collection of functional requirements and on implementing algorithms in terms of these requirements. Because algorithms define a strict and narrow interface to the types they operate on, the same algorithm can be used against a wide collection of types. The implementations in this book use generic programming techniques to achieve a minimal commitment to specificity, extraordinary terseness, and efficiency that rivals carefully handcrafted code.

C++ is the only implementation tool used in this book. You will not find in this book code that implements nifty windowing systems, complex networking libraries, or clever logging mechanisms. Instead, you will find the fundamental components that make it easy to implement all of the above, and much more. C++ has the breadth necessary to make this possible. Its underlying C memory model ensures raw performance, its support for polymorphism enables object-oriented techniques, and its templates unleash an incredible code generation machine. Templates pervade all the code in the book because they allow close cooperation between the user and the library. The user of the library literally controls the way code is generated, in ways constrained by the library. The role of a generic component library is to allow user-specified types and behaviors to be combined with generic components in a sound design. Because of the static nature of the techniques used, errors in mixing and matching the appropriate pieces are usually caught during compilation.

This book’s manifest intent is to create generic components—preimplemented pieces of design whose main characteristics are flexibility, versatility, and ease of use. Generic components do not form a framework. In fact, their approach is complementary—whereas a framework defines interdependent classes to foster a specific object model, generic components are lightweight design artifacts that are independent of each other, yet can be mixed and matched freely. They can be of great help in *implementing* frameworks.

传统上属于设计领域的活动通常完成*before*编码。由于其高水平，通用组件使得能够以异常表达、简洁且易于维护的方式将复杂的体系结构映射到代码。

这里重新组合了三个元素：设计模式、泛型编程和 C++。这些元素结合在一起，可以在软件设计的水平和垂直“市场”中实现非常高的重用率。在水平维度上，少量的库代码实现了一些组合的（本质上是开放式的）结构和行为。在垂直维度上，这些组件的通用性使其适用于广泛的程序。

这本书很大程度上归功于设计模式，以及针对面向对象开发中不断出现的问题的强大解决方案。设计模式是优秀设计的精华——针对在许多情况下可能遇到的问题提供合理的、可重用的解决方案。设计模式专注于为要传达的设计提供暗示性的词汇。他们描述了问题、经过时间验证的解决方案及其变体，以及选择该解决方案的每个变体的后果。设计模式超越了编程语言所能表达的任何东西，无论它有多先进。通过遵循和组合某些设计模式，本书中介绍的组件往往可以解决一大类具体问题。

通用编程是一种范式，专注于将类型抽象为功能需求的狭窄集合，并根据这些需求实现算法。因为算法为其操作的类型定义了严格而狭窄的接口，所以相同的算法可以用于广泛的类型集合。本书中的实现使用通用编程技术来实现对特定性的最小承诺、非凡的简洁性和与精心手工编写的代码相媲美的效率。

C++是本书中使用的唯一实现工具。在本书中你不会找到实现漂亮的窗口系统、复杂的网络库或巧妙的日志机制的代码。相反，您将找到可以轻松实现上述所有内容以及更多功能的基本组件。C++ 具有使这成为可能所需的广度。它的底层 C 内存模型确保了原始性能，它对多态性的支持实现了面向对象的技术，它的模板释放了令人难以置信的代码生成机。模板遍布本书中的所有代码，因为它们允许用户和库之间的密切合作。库的用户实际上以受库约束的方式控制代码的生成方式。通用组件库的作用是允许用户指定的类型和行为与健全设计中的通用组件相结合。由于所使用技术的静态性质，在编译过程中通常会发现混合和匹配适当部分的错误。

本书的明显意图是创建通用组件——预先实现的设计片段，其主要特点是灵活性、多功能性和易用性。通用组件不形成框架。事实上，他们的方法是互补的——框架定义了相互依赖的类来培育特定的对象模型，而通用组件是彼此独立的轻量级设计工件，但可以自由地混合和匹配。它们在 *implementing* 框架中可以提供很大的帮助。

Audience

The intended audience of this book falls into two main categories. The first category is that of experienced C++ programmers who want to master the most modern library writing techniques. The book presents new, powerful C++ idioms that have surprising capabilities, some of which weren't even thought possible. These idioms are of great help in writing high-level libraries. Intermediate C++ programmers who want to go a step further will certainly find the book useful, too, especially if they invest a bit of perseverance. Although pretty hard-core C++ code is sometimes presented, it is thoroughly explained.

The second category consists of busy programmers who need to get the job done without undergoing a steep learning investment. They can skim the most intricate details of implementation and concentrate on *using* the provided library. Each chapter has an introductory explanation and ends with a Quick Facts section. Programmers will find these features a useful reference in understanding and using the components. The components can be understood in isolation, are very powerful yet safe, and are a joy to use.

You need to have a solid working experience with C++ and, above all, the desire to learn more. A degree of familiarity with templates and the Standard Template Library (STL) is desirable.

Having an acquaintance with design patterns (Gamma et al. 1995) is recommended but not mandatory. The patterns and idioms applied in the book are described in detail. However, this book is not a pattern book—it does not attempt to treat patterns in full generality. Because patterns are presented from the pragmatic standpoint of a library writer, even readers interested mostly in patterns may find the perspective refreshing, if constrained.

Loki

The book describes an actual C++ library called Loki. Loki is the god of wit and mischief in Norse mythology, and the author's hope is that the library's originality and flexibility will remind readers of the playful Norse god. All the elements of the library live in the namespace `Loki`. The namespace is not mentioned in the coding examples because it would have unnecessarily increased indentation and the size of the examples. Loki is freely available; you can download it from <http://www.modernpcdesign.com>.

Except for its threading part, Loki is written exclusively in standard C++. This, alas, means that many current compilers cannot cope with parts of it. I implemented and tested Loki using Metrowerks' CodeWarrior Pro 6.0 and Comeau C++ 4.2.38, both on Windows. It is likely that KAI C++ wouldn't have any problem with the code, either. As vendors release new, better compiler versions, you will be able to exploit everything Loki has to offer.

Loki's code and the code samples presented throughout the book use a popular coding standard originated by Herb Sutter. I'm sure you will pick it up easily. In a nutshell,

- Classes, functions, and enumerated types look `LikeThis`.
- Variables and enumerated values look `likeThis`.
- Member variables look `likeThis_`.
- Template parameters are declared with `class` if they can be only a user-defined type, and with `typename` if they can also be a primitive type.

观众

本书的目标读者分为两大类。第一类是经验丰富的 C++ 程序员，他们希望掌握最现代的库编写技术。本书介绍了新的、强大的 C++ 习语，它们具有令人惊讶的功能，其中一些功能甚至被认为是不可能的。这些习惯用法对于编写高级库有很大帮助。想要更进一步的中级 C++ 程序员肯定也会发现这本书很有用，特别是如果他们付出一点毅力的话。尽管有时会出现相当硬核的 C++ 代码，但它的解释很详尽。

第二类是忙碌的程序员，他们需要在不进行大量学习投资的情况下完成工作。他们可以略过最复杂的实现细节，并专注于提供的库 *using*。每一章都有介绍性解释，并以概览部分结尾。程序员会发现这些特性对于理解和使用组件来说是有用的参考。这些组件可以单独理解，非常强大但安全，并且使用起来很有趣。

您需要拥有丰富的 C++ 工作经验，最重要的是，需要了解更多信息的愿望。对模板和标准模板库 (STL) 有一定程度的熟悉是可取的。

建议熟悉设计模式 (Gamma et al. 1995)，但不是强制要求。书中应用的模式和习语都有详细的描述。然而，本书不是一本模式书——它并不试图以完全的普遍性来对待模式。因为模式是从库作者的务实立场呈现的，所以即使是对模式最感兴趣的读者也可能会发现这种观点令人耳目一新，如果受到限制的话。

洛基

这本书描述了一个名为 Loki 的实际 C++ 库。洛基是北欧神话中的机智和恶作剧之神，作者希望图书馆的独创性和灵活性能让读者想起这位顽皮的北欧神。该库的所有元素都位于命名空间 Loki 中。编码示例中未提及命名空间，因为它会不必要地增加缩进和示例的大小。Loki 是免费的；您可以从 <http://www.modernpcdesign.com> 下载它。

除了线程部分外，Loki 完全采用标准 C++ 编写。唉，这意味着许多当前的编译器无法处理其中的部分内容。我使用 Metrowerks 的 CodeWarrior Pro 6.0 和 Comeau C++ 4.2.38（均在 Windows 上）实现并测试了 Loki。KAI C++ 的代码也可能不会有任何问题。随着供应商发布新的、更好的编译器版本，您将能够利用 Loki 提供的一切。

Loki 的代码和整本书中提供的代码示例使用由 Herb Sutter 发起的流行编码标准。我相信你会很容易学会的。简而言之，

- 类、函数和枚举类型看起来像这样。
- 变量和枚举值看起来像这样。
- 成员变量看起来像 `This_`。
- 如果模板参数只能是用户定义的类型，则使用 `class` 进行声明；如果模板参数也可以是基本类型，则使用 `typename` 进行声明。

Organization

The book consists of two major parts: techniques and components. Part I (Chapters 1 to 4) describes the C++ techniques that are used in generic programming and in particular in building generic components. A host of C++-specific features and techniques are presented: policy-based design, partial template specialization, typelists, local classes, and more. You may want to read this part sequentially and return to specific sections for reference.

Part II builds on the foundation established in Part I by implementing a number of generic components. These are not toy examples; they are industrial-strength components used in real-world applications. Recurring issues that C++ developers face in their day-to-day activity, such as smart pointers, object factories, and functor objects, are discussed in depth and implemented in a generic way. The text presents implementations that address basic needs and solve fundamental problems. Instead of explaining what a body of code does, the approach of the book is to discuss problems, take design decisions, and implement those decisions gradually.

Chapter 1 presents policies—a C++ idiom that helps in creating flexible designs.

Chapter 2 discusses general C++ techniques related to generic programming.

Chapter 3 implements typelists, which are powerful type manipulation structures.

Chapter 4 introduces an important ancillary tool: a small object allocator.

Chapter 5 introduces the concept of generalized functors, useful in designs that use the Command design pattern.

Chapter 6 describes Singleton objects.

Chapter 7 discusses and implements smart pointers.

Chapter 8 describes generic object factories.

Chapter 9 treats the Abstract Factory design pattern and provides implementations of it.

Chapter 10 implements several variations of the Visitor design pattern in a generic manner.

Chapter 11 implements several multimethod engines, solutions that foster various trade-offs.

The design themes cover many important situations that C++ programmers have to cope with on a regular basis. I personally consider object factories (Chapter 8) a cornerstone of virtually any quality polymorphic design. Also, smart pointers (Chapter 7) are an important component of many C++ applications, small and large. Generalized functors (Chapter 5) have an incredibly broad range of applications. Once you have generalized functors, many complicated design problems become very simple. The other, more specialized, generic components, such as Visitor (Chapter 10) or multimethods (Chapter 11), have important niche applications and stretch the boundaries of language support.

组织

本书由两大部分组成：技术和组件。第一部分（第 1 章到第 4 章）描述了通用编程中使用的 C++ 技术，特别是构建通用组件的技术。提出了许多 C++ 特定的功能和技术：基于策略的设计、部分模板专门化、类型列表、本地类等等。您可能需要按顺序阅读本部分并返回特定部分以供参考。

第二部分建立在第一部分建立的基础上，实现了一些通用组件。这些不是玩具示例；而是示例。它们是实际应用中使用的工业级组件。我们深入讨论了 C++ 开发人员在日常活动中经常遇到的问题，例如智能指针、对象工厂和函子对象，并以通用方式实现。本文介绍了满足基本需求和解决基本问题的实现。本书的方法不是解释代码体的作用，而是讨论问题、做出设计决策并逐步实施这些决策。

第一章介绍了策略——一种有助于创建灵活设计的 C++ 习惯用法。第 2 章讨论与泛型编程相关的通用 C++ 技术。第 3 章实现了类型列表，它是强大的类型操作结构。第 4 章介绍了一个重要的辅助工具：小对象分配器。第 5 章介绍了广义函子的概念，它在使用命令设计模式的设计中非常有用。第 6 章描述 Singleton 对象。第 7 章讨论和实现智能指针。第 8 章描述了通用对象工厂。第 9 章讨论抽象工厂设计模式并提供它的实现。第 10 章以通用方式实现了访问者设计模式的几种变体。第 11 章实现了几种多方法引擎，以及促进各种权衡的解决方案。

设计主题涵盖了 C++ 程序员必须定期处理的许多重要情况。我个人认为对象工厂（第 8 章）几乎是任何高质量多态设计的基石。此外，智能指针（第 7 章）是许多小型和大型 C++ 应用程序的重要组成部分。广义函子（第 5 章）有着极其广泛的应用。一旦有了广义函子，许多复杂的设计问题就会变得非常简单。其他更专业的通用组件，例如 Visitor（第 10 章）或多方法（第 11 章），具有重要的利基应用程序并扩展了语言支持的边界。

Acknowledgments

I would like to thank my parents for diligently taking care of the longest, toughest part of them all.

It should be stressed that this book, and much of my professional development, wouldn't have existed without Scott Meyers. Since we met at the C++ World Conference in 1998, Scott has constantly helped me do more and do better. Scott was the first person who enthusiastically encouraged me to develop my early ideas. He introduced me to John Vlissides, catalyzing another fruitful cooperation; lobbied Herb Sutter to accept me as a columnist for *C++ Report*; and introduced me to Addison-Wesley, practically forcing me into starting this book, at a time when I still had trouble understanding New York sales clerks. Ultimately, Scott helped me all the way through the book with reviews and advice, sharing with me all the pains of writing, and none of the benefits.

Many thanks to John Vlissides, who, with his sharp insights, convinced me of the problems with my solutions and suggested much better ones. Chapter 9 exists because John insisted that "things could be done better."

Thanks to P. J. Plauger and Marc Briand for encouraging me to write for the *C/C++ Users Journal*, at a time when I still believed that magazine columnists were inhabitants of another planet.

I am indebted to my editor, Debbie Lafferty, for her continuous support and sagacious advice.

My colleagues at RealNetworks, especially Boris Jerkunica and Jim Knaack, helped me very much by fostering an atmosphere of freedom, emulation, and excellence. I am grateful to them all for that.

I also owe much to all participants in the `comp.lang.c++.moderated` and `comp.std.c++` Usenet newsgroups. These people greatly and generously contributed to my understanding of C++.

I would like to address thanks to the reviewers of early drafts of the manuscript: Mihail Antonescu, Bob Archer (my most thorough reviewer of all), Allen Broadman, Ionut Burete, Mirel Chirita, Steve Clamage, James O. Coplien, Doug Hazen, Kevlin Henney, John Hickin, Howard Hinnant, Sorin Jianu, Zoltan Kormos, James Kuyper, Lisa Lippincott, Jonathan H. Lundquist, Petru Marginean, Patrick McKillen, Florin Mihaila, Sorin Oprea,

致谢

我要感谢我的父母，他们辛勤地照顾了他们中最长、最艰难的部分。

应该强调的是，如果没有斯科特·迈耶斯，这本书以及我的大部分职业发展就不会存在。自从我们 1998 年在 C++ 世界大会上见面以来，Scott 一直帮助我做得更多、做得更好。斯科特是第一个热情鼓励我发展早期想法的人。他把我介绍给了约翰·弗利赛兹（John Vlissides），促成了另一次富有成效的合作。游说赫伯·萨特（Herb Sutter）接受我作为《C++ Report》的专栏作家，并将我介绍给艾迪生·韦斯利（Addison-Wesley），实际上迫使我开始写这本书，当时我仍然无法理解纽约的销售人员。最终，斯科特通过评论和建议帮助我完成了这本书，与我分享了写作的所有痛苦，却没有得到任何好处。

非常感谢 John Vlissides，他以其敏锐的洞察力使我确信我的解决方案存在问题，并提出了更好的建议。第九章的存在是因为约翰坚持认为“事情可以做得更好”。

感谢 P. J. Plauger 和 Marc Briand 鼓励我为 *C/C++ Users Journal* 撰写文章，当时我仍然相信杂志专栏作家是另一个星球的居民。

我感谢我的编辑黛比·拉弗蒂 (Debbie Lafferty) 的持续支持和明智的建议。

我在 RealNetworks 的同事，尤其是 Boris Jerkunica 和 Jim Knaack，为我提供了很大帮助，营造了一种自由、竞争和卓越的氛围。我为此感谢他们所有人。

我还非常感谢 `comp.lang.c++.moderated` 和 `comp.std.c++` Usenet 新闻组的所有参与者。这些人为我对 C++ 的理解做出了巨大而慷慨的贡献。

我要感谢手稿早期草稿的审稿人：Mihail Antonescu、Bob Archer（我最彻底的审稿人）、Allen Broadman、Ionut Burete、Mirel Chirita、Steve Clamage、James O. Coplien、Doug Hazen、Kevlin Henney、John Hickin、Howard Hinnant、Sorin Jianu、Zoltan Kormos、James Kuyper、Lisa Lippincott、Jonathan H. Lundquist、Petru Marginean、Patrick McKillen、Florin Mihaila、Sorin Oprea、

John Potter, Adrian Rapiteanu, Monica Rapiteanu, Brian Stanton, Adrian Steflea, Herb Sutter, John Torjo, Florin Trofin, and Cristi Vlasceanu. They all have invested significant efforts in reading and providing input, without which this book wouldn't have been half of what it is.

Thanks to Greg Comeau for providing me with his top-notch C++ compiler for free.

Last but not least, I would like to thank all my family and friends for their continuous encouragement and support.

约翰·波特、阿德里安·拉皮泰亚努、莫妮卡·拉皮泰亚努、布莱恩·斯坦顿、阿德里安·斯特夫利亚、赫伯·萨特、约翰·托乔、弗洛林·特罗芬和克里斯蒂·弗拉斯恰努。他们都在阅读和提供意见方面投入了巨大的努力，没有他们，这本书就不可能有一半的成就。

感谢 Greg Comeau 免费为我提供了他一流的 C++ 编译器。最后但并非最不重要的一点是，我要感谢我所有的家人和朋友一直以来的鼓励和支持。

Smart Pointers

Smart pointers have been the subject of hecatombs of code written and rivers of ink consumed by programmers and writers around the world. Perhaps the most popular, intricate, and powerful C++ idiom, smart pointers are interesting in that they combine many syntactic and semantic issues. This chapter discusses smart pointers, from their simplest aspects to their most complex ones and from the most obvious errors in implementing them to the subtlest ones—some of which also happen to be the most gruesome.

In brief, smart pointers are C++ objects that simulate simple pointers by implementing operator-> and the unary operator*. In addition to sporting pointer syntax and semantics, smart pointers often perform useful tasks—such as memory management or locking—under the covers, thus freeing the application from carefully managing the lifetime of pointed-to objects.

This chapter not only discusses smart pointers but also implements a `SmartPtr` class template. `SmartPtr` is designed around policies (see Chapter 1), and the result is a smart pointer that has the exact levels of safety, efficiency, and ease of use that you want.

After reading this chapter, you will be an expert in smart pointer issues such as the following:

- The advantages and disadvantages of smart pointers
- Ownership management strategies
- Implicit conversions
- Tests and comparisons
- Multithreading issues

This chapter implements a generic `SmartPtr` class template. Each section presents one implementation issue in isolation. At the end, the implementation puts all the pieces together. In addition to understanding the design rationale of `SmartPtr`, you will know how to use, tweak, and extend it.

7.1 Smart Pointers 101

So what's a smart pointer? A smart pointer is a C++ class that mimics a regular pointer in syntax and some semantics, but it does more. Because smart pointers to different types

智能指针

智能指针一直是世界各地程序员和作家编写的大量代码和消耗大量墨水的主题。智能指针也许是最流行、最复杂、最强大的 C++ 惯用法，它的有趣之处在于它们结合了许多语法和语义问题。本章讨论智能指针，从最简单的方面到最复杂的方面，从实现它们时最明显的错误到最微妙的错误——其中一些恰好也是最可怕的。

简而言之，智能指针是通过实现 `operator->` 和一元运算符 `*` 来模拟简单指针的 C++ 对象。除了使用指针语法和语义之外，智能指针还经常在幕后执行有用的任务，例如内存管理或锁定，从而使应用程序无需仔细管理所指向对象的生命周期。

本章不仅讨论智能指针，还实现了 `SmartPtr` 类模板。`SmartPtr` 是围绕策略设计的（参见第 1 章），其结果是一个智能指针，它具有您想要的确切安全性、效率和易用性级别。

阅读完本章后，您将成为智能指针问题的专家，例如以下内容：

- 智能指针的优点和缺点
- 所有权管理策略
- 隐式转换
- 测试与比较
- 多线程问题

本章实现了一个通用的 `SmartPtr` 类模板。每个部分都单独提出一个实施问题。最后，实现将所有部分组合在一起。除了了解 `SmartPtr` 的设计原理之外，您还将知道如何使用、调整和扩展它。

7.1 智能指针 101

那么什么是智能指针呢？智能指针是一个 C++ 类，它在语法和一些语义上模仿常规指针，但它的作用更多。因为智能指针指向不同的类型

of objects tend to have a lot of code in common, almost all good-quality smart pointers in existence are templated by the pointee type, as you can see in the following code:

```
template <class T>
class SmartPtr
{
public:
    explicit SmartPtr(T* pointee) : pointee_(pointee) {...}
    SmartPtr& operator=(const SmartPtr& other);
    ~SmartPtr();
    T& operator*() const
    {
        ...
        return *pointee_;
    }
    T* operator->() const
    {
        ...
        return pointee_;
    }
private:
    T* pointee_;
    ...
};
```

`SmartPtr<T>` aggregates a pointer to `T` in its member variable `pointee_`. That's what most smart pointers do. In some cases, a smart pointer might aggregate some handles to data and compute the pointer on the fly.

The two operators give `SmartPtr` pointer-like syntax and semantics. That is, you can write

```
class Widget
{
public:
    void Fun();
};

SmartPtr<Widget> sp(new Widget);
sp->Fun();
(*sp).Fun();
```

Aside from the definition of `sp`, nothing reveals it as not being a pointer. This is the mantra of smart pointers: You can replace pointer definitions with smart pointer definitions without incurring major changes to your application's code. You thus get extra goodies with ease. Minimizing code changes is very appealing and vital for getting large applications to use smart pointers. As you will soon see, however, smart pointers are not a free lunch.

7.2 The Deal

But what's the deal with smart pointers? you might ask. What do you gain by replacing simple pointers with smart pointers? The explanation is simple. Smart pointers have value semantics, whereas some simple pointers do not.

对象往往有很多共同的代码，几乎所有现有的高质量智能指针都是由被指针类型模板化的，如下面的代码所示：

```
模板 <class T> class SmartPtr {public: 显式 SmartPtr(T* pointee) : pointee_(pointee)
{...} SmartPtr& 运算符=(const SmartPtr& other); ~SmartPtr(); T& 运算符*() const
{ ... return *pointee_; } T* 运算符->() const { ... return pointee_; } 私有: T* pointee
_; ... };
```

`SmartPtr<T>`在其成员变量`pointee_`中聚合了一个指向`T`的指针。这就是大多数智能指针所做的事情。在某些情况下，智能指针可能会聚合一些数据句柄并动态计算指针。

这两个运算符提供了 `SmartPtr` 类似指针的语法和语义。也就是说，你可以写

```
类 Widget {public: void Fun(); }; SmartPtr<Wi
dget> sp(新小部件); sp->Fun(); (*sp).Fun();
```

除了 `sp` 的定义之外，没有任何东西表明它不是一个指针。这是智能指针的口头禅：您可以用智能指针定义替换指针定义，而无需对应用程序代码进行重大更改。因此，您可以轻松获得额外的好东西。最大限度地减少代码更改对于让大型应用程序使用智能指针非常有吸引力且至关重要。然而，您很快就会看到，智能指针并不是免费的午餐。

7.2 交易

但是智能指针有什么用呢？你可能会问。用智能指针替换简单指针可以获得什么好处？解释很简单。智能指针具有值语义，而某些简单指针则没有。

An object with value semantics is an object that you can *copy* and *assign to*. A plain `int` is the perfect example of a first-class object. You can create, copy, and change integer values freely. A pointer that you use to iterate in a buffer also has value semantics—you initialize it to point to the beginning of the buffer, and you bump it until you reach the end. Along the way, you can copy its value to other variables to hold temporary results.

With pointers that hold values allocated with `new`, however, the story is very different. Once you have written

```
Widget* p = new Widget;
```

the variable `p` not only points to, but also *owns*, the memory allocated for the `Widget` object. This is because later you must issue `delete p` to ensure that the `Widget` object is destroyed and its memory is released. If in the line after the line just shown you write

```
p = 0; // assign something else to p
```

you lose ownership of the object previously pointed to by `p`, and you have no chance at all to get a grip on it again. You have a resource leak, and resource leaks never help.

Furthermore, when you copy `p` into another variable, the compiler does not automatically manage the ownership of the memory to which the pointer points. All you get is two raw pointers pointing to the same object, and you have to track them even more carefully because double deletions are even more catastrophic than no deletion. Consequently, pointers to allocated objects do *not* have value semantics—you cannot copy and assign to them at will.

Smart pointers can be of great help in this area. Most smart pointers offer *ownership management* in addition to pointer-like behavior. Smart pointers can figure out how ownership evolves, and their destructors can release the memory according to a well-defined strategy. Many smart pointers hold enough information to take full initiative in releasing the pointed-to object.

Smart pointers may manage ownership in various ways, each appropriate to a category of problems. Some smart pointers transfer ownership automatically: After you copy a smart pointer to an object, the source smart pointer becomes null, and the destination points to (and holds ownership of) the object. This is the behavior implemented by the standard-provided `std::auto_ptr`. Other smart pointers implement reference counting: They track the total count of smart pointers that point to the same object, and when this count goes down to zero, they `delete` the pointed-to object. Finally, some others duplicate their pointed-to object whenever you copy them.

In short, in the smart pointers' world, ownership is an important topic. By providing ownership management, smart pointers are able to support integrity guarantees and full value semantics. Because ownership has much to do with constructing, copying, and destroying smart pointers, it's easy to figure out that these are the most vital functions of a smart pointer.

The following few sections discuss various aspects of smart pointer design and implementation. The goal is to render smart pointers as close to raw pointers as possible, but not closer. It's a contradictory goal: After all, if your smart pointers behave *exactly* like dumb pointers, they *are* dumb pointers.

具有值语义的对象是可以 *copy* 和 *assign to* 的对象。普通 `int` 是第一类对象的完美示例。您可以自由创建、复制和更改整数值。用于在缓冲区中迭代的指针也具有值语义 - 您将其初始化为指向缓冲区的开头，然后对其进行碰撞直至到达结尾。在此过程中，您可以将其值复制到其他变量以保存临时结果。

然而，对于保存用 `new` 分配的值的指针，情况就大不相同了。一旦你写完

```
Widget* p = new Widget;
```

变量 `p` 不仅指向 *owns*，为 `Widget` 对象分配的内存。这是因为稍后您必须发出 `delete p` 来确保 `Widget` 对象被销毁并释放其内存。如果在刚刚显示的行之后的行中写

```
p = 0; // 给 p 分配别的东西
```

你失去了 `p` 先前指向的对象的所有权，并且你根本没有机会再次掌握它。你遇到了资源泄漏，而资源泄漏根本没有帮助。

此外，当您将 `p` 复制到另一个变量时，编译器不会自动管理指针指向的内存的所有权。您得到的只是指向同一个对象的两个原始指针，并且您必须更加仔细地跟踪它们，因为双重删除比不删除更具灾难性。因此，指向已分配对象的指针 *not* 具有值语义 - 您不能随意复制并分配给它们。

智能指针在这方面可以提供很大的帮助。除了类似指针的行为之外，大多数智能指针还提供 *ownership management*。智能指针可以弄清楚所有权如何演变，并且它们的析构函数可以根据明确定义的策略释放内存。许多智能指针拥有足够的信息来完全主动地释放所指向的对象。

智能指针可以通过多种方式管理所有权，每种方式都适合一类问题。某些智能指针会自动转移所有权：将智能指针复制到对象后，源智能指针变为空，目标指向该对象（并持有该对象的所有权）。这是由标准提供的 `std::auto_ptr` 实现的行为。其他智能指针实现引用计数：它们跟踪指向同一对象的智能指针的总数，当该计数减至零时，它们删除指向的对象。最后，当您复制其他一些对象时，它们也会复制它们所指向的对象。

简而言之，在智能指针的世界中，所有权是一个重要的话题。通过提供所有权管理，智能指针能够支持完整性保证和完整的值语义。因为所有权与智能指针的构造、复制和销毁有很大关系，所以很容易看出这些是智能指针最重要的功能。

以下几节讨论智能指针设计和实现的各个方面。目标是使智能指针尽可能接近原始指针，但不是更接近。这是一个矛盾的目标：毕竟，如果你的智能指针的行为 *exactly* 像哑指针，那么它们 *are* 就是哑指针。

In implementing compatibility between smart pointers and raw pointers, there is a thin line between nicely filling compatibility checklists and paving the way to chaos. You will find that adding seemingly worthwhile features might expose the clients to costly risks. Much of the craft of implementing good smart pointers consists of carefully balancing their set of features.

7.3 Storage of Smart Pointers

To start, let's ask a fundamental question about smart pointers. Is `pointee_`'s type necessarily `T*`? If not, what else could it be? In generic programming, you should always ask yourself questions like these. Each type that's hardcoded in a piece of generic code decreases the genericity of the code. Hardcoded types are to generic code what magic constants are to regular code.

In several situations, it is worthwhile to allow customizing the pointee type. One situation is when you deal with nonstandard pointer modifiers. In the 16-bit Intel 80x86 days, you could qualify pointers with modifiers like `__near`, `__far`, and `__huge`. Other segmented memory architectures use similar modifiers.

Another situation is when you want to layer smart pointers. What if you have a `LegacySmartPointer<T>` smart pointer implemented by someone else, and you want to enhance it? Would you derive from it? That's a risky decision. It's better to wrap the legacy smart pointer into a smart pointer of your own. This is possible because the inner smart pointer supports pointer syntax. From the outer smart pointer's viewpoint, the pointee type is not `T*` but `LegacySmartPointer<T>`.

There are interesting applications of smart pointer layering, mainly because of the mechanics of `operator->`. When you apply `operator->` to a type that's not a built-in pointer, the compiler does an interesting thing. After looking up and applying the user-defined `operator->` to that type, it applies `operator->` again to the result. The compiler keeps doing this recursively until it reaches a native pointer, and only then proceeds with member access. It follows that a smart pointer's `operator->` does not have to return a pointer. It can return an object that in turn implements `operator->`, without changing the use syntax.

This leads to a very interesting idiom: pre- and postfunction calls (Stroustrup 2000). If you return an object of type `PointerType` by value from `operator->`, the sequence of execution is as follows:

1. Constructor of `PointerType`
2. `PointerType::operator->` called; likely returns a pointer to an object of type `PointeeType`
3. Member access for `PointeeType`—likely a function call
4. Destructor of `PointerType`

In a nutshell, you have a nifty way of implementing locked function calls. This idiom has broad uses with multithreading and locked resource access. You can have `PointerType`'s constructor lock the resource, and then you can access the resource; finally, `PointerType`'s destructor unlocks the resource.

The generalization doesn't stop here. The syntax-oriented "pointer" part might some-

在实现智能指针和原始指针之间的兼容性时，很好地填充兼容性检查表和为混乱铺平道路之间只有一线之隔。您会发现添加看似有价值的功能可能会让客户面临代价高昂的风险。实现良好智能指针的大部分技巧都包括仔细平衡其功能集。

7.3 智能指针的存储

首先，让我们问一个有关智能指针的基本问题。pointee_的类型一定是T*吗？如果不是，那还能是什么？在通用编程中，您应该始终问自己这样的问题。硬编码在通用代码中的每种类型都会降低代码的通用性。硬编码类型对于泛型代码来说就像魔术常数对于常规代码一样。

在某些情况下，允许自定义指针对象类型是值得的。一种情况是当您处理非标准指针修饰符时。在 16 位 Intel 80x86 时代，您可以使用 __near、__far 和 __huge 等修饰符来限定指针。其他分段内存架构使用类似的修饰符。

另一种情况是当你想要分层智能指针时。如果您有一个由其他人实现的旧版 SmartPtr<T> 智能指针，并且您想要增强它，该怎么办？你会从中得到启发吗？这是一个冒险的决定。最好将遗留智能指针包装成您自己的智能指针。这是可能的，因为内部智能指针支持指针语法。从外部智能指针的角度来看，指针对象类型不是 T* 而是 LegacySmartPtr<T>。

智能指针分层有一些有趣的应用，主要是因为运算符-> 的机制。当您将operator->应用于不是内置指针的类型时，编译器会做一件有趣的事情。在查找并将用户定义的运算符->应用于该类型之后，它再次将运算符->应用于结果。编译器不断递归地执行此操作，直到到达本机指针，然后才继续进行成员访问。由此可见，智能指针的运算符-> 不必返回指针。它可以返回一个对象，该对象又实现了operator->，而无需更改use语法。

这导致了一个非常有趣的习惯用法：前函数调用和后函数调用（Stroustrup 2000）。如果从operator->按值返回PointerType类型的对象，则执行顺序如下：

1. PointerType 的构造函数 2. 调用 PointerType::operator-> ； 可能返回指向 PointeeType 类型的对象的指针 3. PointeeType 的成员访问 — 可能是函数调用 4. PointerType 的析构函数

简而言之，您有一种实现锁定函数调用的巧妙方法。这种习惯用法在多线程和锁定资源访问方面有着广泛的用途。你可以让PointerType的构造函数锁定资源，然后你就可以访问该资源；最后，PointerType 的析构函数解锁资源。

概括并不止于此。面向语法的“指针”部分可能会-

times pale in comparison with the powerful resource management techniques that are included in smart pointers. It follows that, in rare cases, smart pointers could drop the pointer syntax. An object that does not define `operator->` and `operator*` violates the definition of a smart pointer, but there are objects that do deserve smart pointer-like treatment, although they are not, strictly speaking, smart pointers.

Look at real-world APIs and applications. Many operating systems foster *handles* as accessors to certain internal resources, such as windows, mutexes, or devices. Handles are intentionally obfuscated pointers; one of their purposes is to prevent their users from manipulating critical operating system resources directly. Most of the time, handles are integral values that are indices in a hidden table of pointers. The table provides the additional level of indirection that protects the inner system from the application programmers. Although they don't provide an `operator->`, handles resemble pointers in semantics and in the way they are managed.

For such a smart resource, it does not make sense to provide `operator->` or `operator*`. However, you do take advantage of all the resource management techniques that are specific to smart pointers.

To generalize the type universe of smart pointers, we distinguish three potentially distinct types in a smart pointer:

- *The storage type.* This is the type of `pointee_`. By “default”—in regular smart pointers—it is a raw pointer.
- *The pointer type.* This is the type returned by `operator->`. It can be different from the storage type if you want to return a proxy object instead of just a pointer. (You will find an example of using proxy objects later in this chapter.)
- *The reference type.* This is the type returned by `operator*`.

It would be useful if `SmartPtr` supported this generalization in a flexible way. Thus, the three types mentioned here ought to be abstracted in a policy called **Storage**.

In conclusion, smart pointers can, and should, generalize their pointee type. To do this, `SmartPtr` abstracts three types in a **Storage** policy: the stored type, the pointer type, and the reference type. Not all types necessarily make sense for a given `SmartPtr` instantiation. Therefore, in rare cases (handles), a policy might disable access to `operator->` or `operator*` or both.

7.4 Smart Pointer Member Functions

Many existing smart pointer implementations allow operations through member functions, such as `Get` for accessing the pointee object, `Set` for changing it, and `Release` for taking over ownership. This is the obvious and natural way of encapsulating `SmartPtr`'s functionality.

However, experience has proven that member functions are not very suitable for smart pointers. The reason is that the interaction between member function calls for the smart pointer and for the *pointed-to* object can be extremely confusing.

Suppose, for instance, that you have a `Printer` class with member functions such as `Acquire` and `Release`. With `Acquire` you take ownership of the printer so that no other application prints to it, and with `Release` you relinquish ownership. As you use a smart

与智能指针中包含的强大资源管理技术相比，时代显得苍白无力。因此，在极少数情况下，智能指针可能会删除指针语法。没有定义`operator->`和`operator*`的对象违反了智能指针的定义，但是有些对象确实值得像智能指针一样处理，尽管严格来说它们不是智能指针。

查看现实世界的 API 和应用程序。许多操作系统将 *handles* 作为某些内部资源（例如窗口、互斥锁或设备）的访问器。句柄是故意混淆的指针；它们的目的之一是防止用户直接操纵关键操作系统资源。大多数时候，句柄是整数值，是隐藏指针表中的索引。该表提供了额外的间接级别，可以保护内部系统免受应用程序程序员的侵害。尽管它们不提供运算符`->`，但句柄在语义和管理方式上类似于指针。

对于这样的智能资源，提供`operator->`或`operator*`是没有意义的。但是，您确实可以利用智能指针特有的所有资源管理技术。

为了概括智能指针的类型域，我们区分了智能指针中三种可能不同的类型：

- *The storage type.* 这是`pointee_`的类型。默认情况下（在常规智能指针中），它是一个原始指针。
- *The pointer type.* 这是`operator->`返回的类型。如果您想返回代理对象而不仅仅是指针，它可以与存储类型不同。（您将在本章后面找到使用代理对象的示例。）
- *The reference type.* 这是`operator*`返回的类型。

如果 `SmartPtr` 以灵活的方式支持这种概括，那将会很有用。因此，这里提到的三种类型应该被抽象为一个名为“存储”的策略。

总之，智能指针可以而且应该概括它们的指针类型。为此，`SmartPtr` 在存储策略中抽象了三种类型：存储类型、指针类型和引用类型。并非所有类型对于给定的 `SmartPtr` 实例都一定有意义。因此，在极少数情况下（句柄），策略可能会禁用对`operator->`或`operator*`或两者的访问。

7.4 智能指针成员函数

许多现有的智能指针实现允许通过成员函数进行操作，例如用于访问指针对象的 `Get`、用于更改它的 `Set` 以及用于接管所有权的 `Release`。这是封装 `SmartPtr` 功能的明显且自然的方式。

然而，经验证明，成员函数不太适合智能指针。原因是智能指针和 *pointed-to* 对象的成员函数调用之间的交互可能非常混乱。

例如，假设您有一个带有 `Acquire` 和 `Release` 等成员函数的 `Printer` 类。使用“获取”，您可以获取打印机的所有权，这样其他应用程序就不会打印该打印机；使用“释放”，您可以放弃所有权。当您使用智能

pointer to `Printer`, you may notice a strange syntactical closeness to things that are very far apart semantically.

```
SmartPtr<Printer> spRes = ...;
spRes->Acquire(); // acquire the printer
... print a document ...
spRes->Release(); // release the printer
spRes.Release(); // release the pointer to the printer
```

The user of `SmartPtr` now has access to two totally different worlds: the world of the pointed-to object members and the world of the smart pointer members. A matter of a dot or an arrow thinly separates the two worlds.

On the face of it, C++ does force you routinely to observe certain slight differences in syntax. A Pascal programmer learning C++ might even feel that the slight syntactic difference between `&` and `&&` is an abomination. Yet C++ programmers don't even blink at it. They are trained by habit to distinguish such syntax matters easily.

However, smart pointer member functions defeat training by habit. Raw pointers don't have member functions, so C++ programmers' eyes are not habituated to detect and distinguish dot calls from arrow calls. The compiler does a good job at that: If you use a dot after a raw pointer, the compiler will yield an error. Therefore, it is easy to imagine, and experience proves, that even seasoned C++ programmers find it extremely disturbing that both `sp.Release()` and `sp->Release()` compile flag-free but do very different things. The cure is simple: A smart pointer should not use member functions. `SmartPtr` uses only nonmember functions. These functions become friends of the smart pointer class.

Overloaded functions can be just as confusing as member functions of smart pointers, but there is an important difference. C++ programmers already use overloaded functions. Overloading is an important part of the C++ language and is used routinely in library and application development. This means that C++ programmers *do* pay attention to differences in function call syntax—such as `Release(*sp)` versus `Release(sp)`—in writing and reviewing code.

The only functions that necessarily remain members of `SmartPtr` are the constructors, the destructor, `operator=`, `operator->`, and unary `operator*`. All other operations of `SmartPtr` are provided through named nonmember functions.

For reasons of clarity, `SmartPtr` does not have any named member functions. The only functions that access the pointee object are `GetImpl`, `GetImplRef`, `Reset`, and `Release`, which are defined at the namespace level.

```
template <class T> T* GetImpl(SmartPtr<T>& sp);
template <class T> T*& GetImplRef(SmartPtr<T>& sp);
template <class T> void Reset(SmartPtr<T>& sp, T* source);
template <class T> void Release(SmartPtr<T>& sp, T*& destination);
```

- `GetImpl` returns the pointer object stored by `SmartPtr`.
- `GetImplRef` returns *a reference* to the pointer object stored by `SmartPtr`. `GetImplRef` allows you to change the underlying pointer, so it requires extreme care in use.
- `Reset` resets the underlying pointer to another value, releasing the previous one.
- `Release` releases ownership of the smart pointer, giving its user the responsibility of managing the pointee object's lifetime.

当指针指向打印机时，您可能会注意到语义上相距甚远的事物在语法上有一种奇怪的相似性。

```
SmartPtr<打印机> spRes = ...; spRes->获取(); // 获取打印机...打印文档... spRe
s->Release(); // 释放打印机 spRes.Release(); // 释放打印机指针
```

SmartPtr 的用户现在可以访问两个完全不同的世界：指向对象成员的世界和智能指针成员的世界。一个点或一个箭头就将两个世界分开了。

从表面上看，C++ 确实迫使您经常观察语法中的某些细微差异。学习 C++ 的 Pascal 程序员甚至可能会觉得 & 和 && 之间细微的语法差异是令人厌恶的。然而 C++ 程序员甚至对此不屑一顾。他们经过习惯训练，能够轻松区分此类语法问题。

然而，智能指针成员函数却无法通过习惯进行训练。原始指针没有成员函数，因此 C++ 程序员的眼睛不习惯检测和区分点调用和箭头调用。编译器在这方面做得很好：如果在原始指针后面使用点，编译器将产生错误。因此，很容易想象，并且经验证明，即使是经验丰富的 C++ 程序员也会发现 sp.Release() 和 sp->Release() 编译无标志但执行的操作截然不同，这非常令人不安。解决方法很简单：智能指针不应该使用成员函数。SmartPtr 仅使用非成员函数。这些函数成为智能指针类的朋友。

重载函数可能与智能指针的成员函数一样令人困惑，但有一个重要的区别。C++ 程序员已经使用重载函数。重载是 C++ 语言的重要组成部分，在库和应用程序开发中经常使用。这意味着 C++ 程序员 *do* 在编写和审查代码时会注意函数调用语法的差异，例如 Release(*sp) 与 Release(sp)。

唯一必须保留为 SmartPtr 成员的函数是构造函数、析构函数、operator=、operator-> 和一元运算符*。SmartPtr 的所有其他操作都是通过命名非成员函数提供的。

为了清楚起见，SmartPtr 没有任何命名成员函数。访问被指对象的唯一函数是 GetImpl、GetImplRef、Reset 和 Release，它们是在命名空间级别定义的。

```
模板 <类 T> T* GetImpl(SmartPtr<T>& sp);模板 <类 T> T*& GetImplRef(SmartPtr<T>& sp);
模板 <class T> void Reset(SmartPtr<T>& sp, T* source);模板 <class T> void Release(SmartPtr<
T>& sp, T*& 目的地);
```

- GetImpl 返回 SmartPtr 存储的指针对象。
- GetImplRef 返回 *a reference* 到 SmartPtr 存储的指针对象。GetImplRef 允许您更改底层指针，因此使用时需要格外小心。
- Reset 将底层指针重置为另一个值，释放前一个值。
- Release 释放智能指针的所有权，让其用户负责管理被指针对象的生命周期。

The actual declarations of these four functions in Loki are slightly more elaborate. They don't assume that the type of the pointer object stored by `SmartPtr` is `T*`. As discussed in Section 7.3, the `Storage` policy defines the storage type. Most of the time, it's a straight pointer, except in exotic implementations of `Storage`, when it might be a handle or an elaborate type.

7.5 Ownership-Handling Strategies

Ownership handling is often the most important *raison d'être* of smart pointers. Usually, from their clients' viewpoint, smart pointers own the objects to which they point. A smart pointer is a first-class value that takes care of deleting the pointed-to object under the covers. The client can intervene in the pointee object's lifetime by issuing calls to helper management functions.

For implementing self-ownership, smart pointers must carefully track the pointee object, especially during copying, assignment, and destruction. Such tracking brings some overhead in space, time, or both. An application should settle on the strategy that best fits the problem at hand and does not cost too much.

The following subsections discuss the most popular ownership management strategies and how `SmartPtr` implements them.

7.5.1 Deep Copy

The simplest strategy applicable is to copy the pointee object whenever you copy the smart pointer. If you ensure this, there is only one smart pointer for each pointee object. Therefore, the smart pointer's destructor can safely delete the pointee object. Figure 7.1 depicts the state of affairs if you use smart pointers with deep copy.

At first glance, the deep copy strategy sounds rather dull. It seems as if the smart pointer does not add any value over regular C++ value semantics. Why would you make the effort of using a smart pointer, when simple pass by value of the pointee object works just as well?

The answer is *support for polymorphism*. Smart pointers are vehicles for transporting polymorphic objects safely. You hold a smart pointer to a base class, which might actually point to a derived class. When you copy the smart pointer, you want to copy its polymorphic behavior, too. It's interesting that you don't exactly know what behavior and state you are dealing with, but you certainly need to duplicate that behavior and state.

Because deep copy most often deals with polymorphic objects, the following naive implementation of the copy constructor is wrong:

```
template <class T>
class SmartPtr
{
public:
    SmartPtr(const SmartPtr& other)
        : pointee_(new T(*other.pointee_))
    {
    }
    ...
};
```

Loki 中这四个函数的实际声明稍微复杂一些。他们不假设SmartPtr存储的指针对象的类型是T*。正如第 7.3 节中所讨论的，存储策略定义了存储类型。大多数时候，它是一个直接指针，除了在存储的奇异实现中，它可能是一个句柄或一个精心设计的类型。

7.5 所有权处理策略

所有权处理通常是智能指针最重要的*raison d'être*。通常，从客户的角度来看，智能指针拥有它们所指向的对象。智能指针是第一类值，负责删除所指向的对象。客户端可以通过调用辅助管理函数来干预被指对象的生命周期。

为了实现自我所有权，智能指针必须仔细跟踪被指对象，特别是在复制、赋值和销毁期间。这种跟踪会带来一些空间、时间或两者的开销。应用程序应该选择最适合当前问题且成本不高的策略。

以下小节讨论最流行的所有权管理策略以及 SmartPtr 如何实施它们。

7.5.1 深拷贝

最简单的适用策略是在复制智能指针时复制被指对象。如果确保这一点，则每个被指对象只有一个智能指针。因此，智能指针的析构函数可以安全地删除指针对象。图 7.1 描述了使用带有深层复制的智能指针时的情况。

乍一看，深度复制策略听起来相当乏味。智能指针似乎没有在常规 C++ 值语义上添加任何值。当被指对象的简单值传递也同样有效时，为什么要努力使用智能指针呢？

答案是*support for polymorphism*。智能指针是安全传输多态对象的工具。您持有一个指向基类的智能指针，该指针实际上可能指向派生类。当你复制智能指针时，你也想复制它的多态行为。有趣的是，您并不完全知道您正在处理什么行为和状态，但您肯定需要复制该行为和状态。

由于深复制最常处理多态对象，因此复制构造函数的以下简单实现是错误的：

```
模板 <class T> class SmartPtr {public: SmartPtr(const  
SmartPtr& other) : pointee_(new T(*other.pointee_)) {  
}... };
```

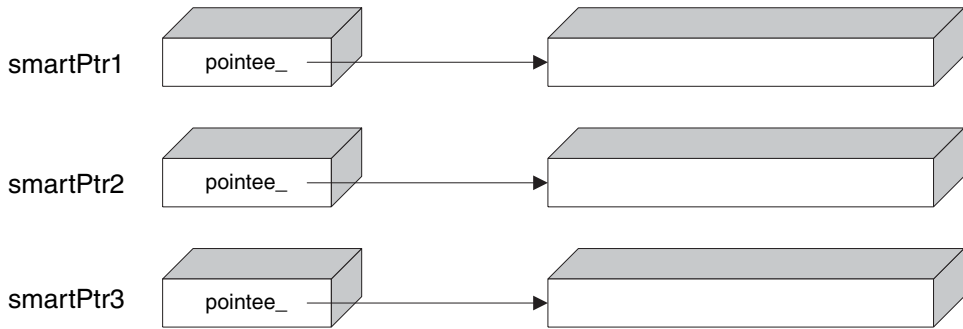


Figure 7.1: Memory layout for smart pointers with deep copy

Say you copy an object of type `SmartPtr<Widget>`. If other points to an instance of a class `ExtendedWidget` that derives from `Widget`, the copy constructor above copies only the `Widget` part of the `ExtendedWidget` object. This phenomenon is known as *slicing*—only the `Widget` “slice” of the object of the presumably larger type `ExtendedWidget` gets copied. Slicing is most often undesirable. It is a pity that C++ allows slicing so easily—a simple call by value slices objects without any warning.

Chapter 8 discusses cloning in depth. As shown there, the classic way of obtaining a polymorphic clone for a hierarchy is to define a virtual `Clone` function and implement it as follows:

```
class AbstractBase
{
    ...
    virtual AbstractBase* Clone() = 0;
};

class Concrete : public AbstractBase
{
    ...
    virtual AbstractBase* Clone()
    {
        return new Concrete(*this);
    }
};
```

The `Clone` implementation must follow the same pattern in all derived classes; in spite of its repetitive structure, there is no reasonable way to automate defining the `Clone` member function (beyond macros, that is).

A generic smart pointer cannot count on knowing the exact name of the cloning member function—maybe it’s `clone`, or maybe `MakeCopy`. Therefore, the most flexible approach is to parameterize `SmartPtr` with a policy that addresses cloning.

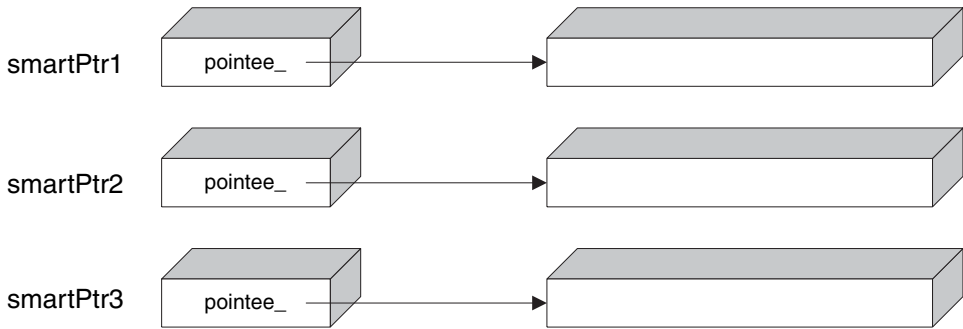


图 7.1: 具有深度复制的智能指针的内存布局

假设您复制 `SmartPtr<Widget>` 类型的对象。如果 `other` 指向派生自 `Widget` 的类 `ExtendedWidget` 的实例，则上面的复制构造函数仅复制 `ExtendedWidget` 对象的 `Widget` 部分。这种现象称为 *slicing* — 仅复制可能较大类型 `ExtendedWidget` 的对象的 `Widget` “切片”。切片通常是不可取的。遗憾的是，C++ 允许如此轻松地进行切片——简单的按值调用就可以在任何警告的情况下对对象进行切片。

第 8 章深入讨论克隆。如图所示，获取层次结构的多态克隆的经典方法是定义一个虚拟 `Clone` 函数并按如下方式实现它：

```
类 AbstractBase { ... 虚拟 AbstractBase* Clone() = 0;
}; 类 Concrete : public AbstractBase { ... virtual AbstractBase* Clone() { return new Concrete(*this); } };
```

`Clone` 实现必须在所有派生类中遵循相同的模式；尽管其结构重复，但没有合理的方法来自定义 `Clone` 成员函数（即，除了宏之外）。

通用智能指针不能指望知道克隆成员函数的确切名称——可能是 `clone`，也可能是 `MakeCopy`。因此，最灵活的方法是使用解决克隆问题的策略对 `SmartPtr` 进行参数化。

7.5.2 Copy on Write

Copy on write (COW, as it is fondly called by its fans) is an optimization technique that avoids unnecessary object copying. The idea that underlies COW is to clone the pointee object at the first attempt of modification; until then, several pointers can share the same object.

Smart pointers, however, are not the best place to implement COW, because smart pointers cannot differentiate between calls to `const` and non-`const` member functions of the pointee object. Here is an example:

```
template <class T>
class SmartPtr
{
public:
    T* operator->() { return pointee_; }
    ...
};

class Foo
{
public:
    void ConstFun() const;
    void NonConstFun();
};

...
SmartPtr<Foo> sp;
sp->ConstFun(); // invokes operator->, then ConstFun
sp->NonConstFun(); // invokes operator->, then NonConstFun
```

The same `operator->` gets invoked for both functions called; therefore, the smart pointer does not have any clue whether to make the COW or not. Function invocations for the pointee object happen somewhere beyond the reach of the smart pointer. (Section 7.11 explains how `const` interacts with smart pointers and the objects they point to.)

In conclusion, COW is effective mostly as an implementation optimization for full-featured classes. Smart pointers are at too low a level to implement COW semantics effectively. Of course, smart pointers can be good building blocks in implementing COW for a class.

The `SmartPtr` implementation in this chapter does not provide support for COW.

7.5.3 Reference Counting

Reference counting is the most popular ownership strategy used with smart pointers. Reference counting tracks the number of smart pointers that point to the same object. When that number goes to zero, the pointee object is deleted. This strategy works very well if you don't break certain rules—for instance, you should not keep dumb pointers and smart pointers to the same object.

The actual counter must be shared among smart pointer objects, leading to the structure

7.5.2 写入时复制

写时复制（COW，粉丝们亲切地称呼它）是一种优化技术，可以避免不必要的对象复制。COW 的基本思想是在第一次尝试修改时克隆指针对象。在此之前，多个指针可以共享同一个对象。

然而，智能指针并不是实现 COW 的最佳场所，因为智能指针无法区分对被指针对象的 `const` 和非 `const` 成员函数的调用。这是一个例子：

```
template <class T>
class SmartPtr
{
public:
    T* operator->() { return pointee_; }
    ...
};

class Foo
{
public:
    void ConstFun() const;
    void NonConstFun();
};

...
SmartPtr<Foo> sp;
sp->ConstFun(); // invokes operator->, then ConstFun
sp->NonConstFun(); // invokes operator->, then NonConstFun
```

两个被调用的函数都会调用相同的运算符 `->`；因此，智能指针不知道是否创建 COW。对被指针对象的函数调用发生在智能指针无法到达的地方。（第 7.11 节解释了 `const` 如何与智能指针及其指向的对象交互。）

总之，COW 主要作为全功能类的实现优化而有效。智能指针的级别太低，无法有效实现 COW 语义。当然，智能指针可以成为实现类的 COW 的良好构建块。

本章中的 `SmartPtr` 实现不提供对 COW 的支持。

7.5.3 引用计数

引用计数是与智能指针一起使用的最流行的所有权策略。引用计数跟踪指向同一对象的智能指针的数量。当该数字变为零时，被指对象将被删除。如果您不违反某些规则，则此策略非常有效 - 例如，您不应将哑指针和智能指针保留到同一对象。

实际计数器

必须在智能指针对象之间共享，导致

电子结构

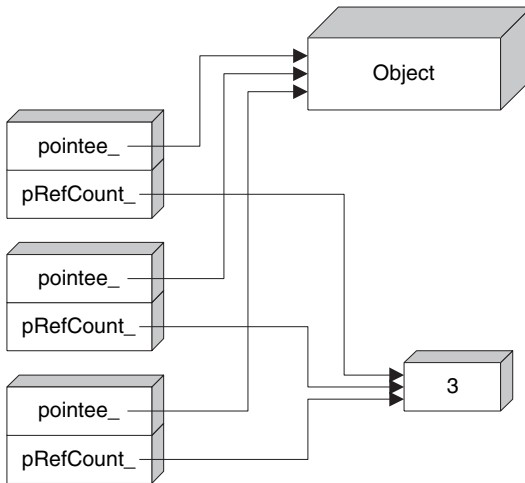


Figure 7.2: Three reference-counted smart pointers pointing to the same object

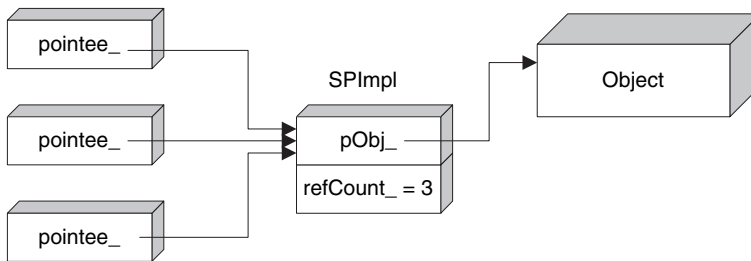


Figure 7.3: An alternate structure of reference-counted pointers

depicted in Figure 7.2. Each smart pointer holds a pointer to the reference counter (`pRefCount_` in Figure 7.2) in addition to the pointer to the object itself. This usually doubles the size of the smart pointer, which may or may not be an acceptable amount of overhead, depending on your needs and constraints.

There is another, subtler overhead issue. Reference-counted smart pointers must store the reference counter on the free store. The problem is that in many implementations, the default C++ free store allocator is remarkably slow and wasteful of space when it comes to allocating small objects, as discussed in Chapter 4. (Obviously, the reference count, typically occupying 4 bytes, does qualify as a small object.) The overhead in speed stems from slow algorithms in finding available chunks of memory, and the overhead in size is incurred by the bookkeeping information that the allocator holds for each chunk.

The relative size overhead can be partially mitigated by holding the pointer and the reference count together, as in Figure 7.3. The structure in Figure 7.3 reduces the size of the smart pointer to that of a pointer, but at the expense of access speed: The pointee object is

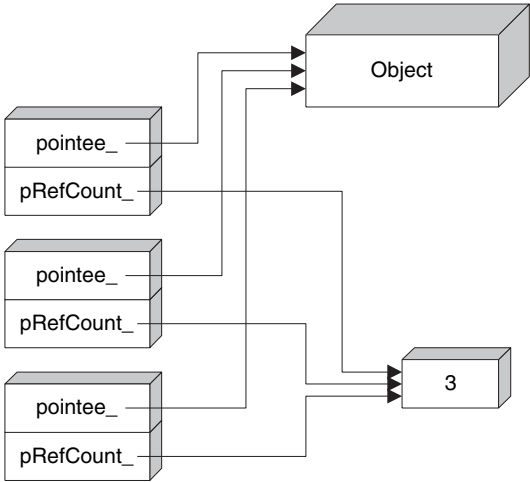


图 7.2: 指向同一个对象的三个引用计数智能指针

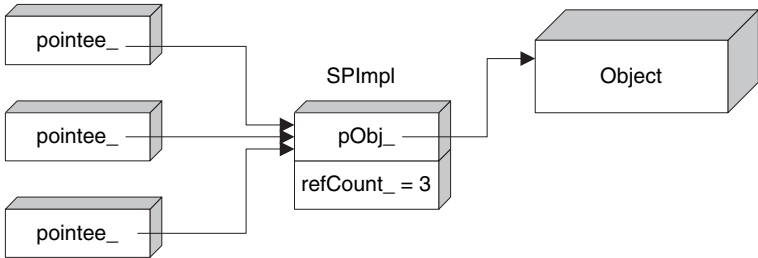


图 7.3: 引用计数指针的另一种结构

如图 7.2 所示。每个智能指针除了指向对象本身的指针外，还保存一个指向引用计数器（图 7.2 中的 pRefCount_）的指针。这通常会使得智能指针的大小加倍，这可能是也可能不是可接受的开销量，具体取决于您的需求和限制。

还有另一个更微妙的开销问题。引用计数智能指针必须将引用计数器存储在空闲存储中。问题是，在许多实现中，默认的 C++ 自由存储分配器在分配小对象时非常慢并且浪费空间，如第 4 章所述。（显然，引用计数通常占用 4 个字节，确实符合小对象的条件。）速度的开销源于查找可用内存块的缓慢算法，而大小的开销是由分配器为每个块保存的簿记信息引起的。

通过将指针和引用计数放在一起，可以部分减轻相对大小开销，如图 7.3 所示。图 7.3 中的结构将智能指针的大小减小到指针的大小，但以牺牲访问速度为代价：被指针对象是

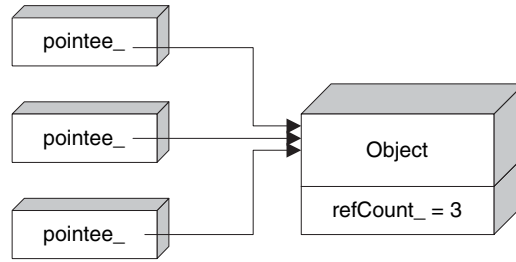


Figure 7.4: Intrusive reference counting

an extra level of indirection away. This is a considerable drawback because you typically use a smart pointer several times, whereas you obviously construct and destroy it only once.

The most efficient solution is to hold the reference counter in the pointee object itself, as shown in Figure 7.4. This way `SmartPtr` is just the size of a pointer, and there is no extra overhead at all. This technique is known as *intrusive reference counting*, because the reference count is an “intruder” in the pointee—it semantically belongs to the smart pointer. The name also gives a hint about the Achilles’ heel of the technique: You must design up front or modify the pointee class to support reference counting.

A generic smart pointer should use intrusive reference counting where available and implement a nonintrusive reference counting scheme as an acceptable alternative. For implementing nonintrusive reference counting, the small-object allocator presented in Chapter 4 can help a great deal. The `SmartPtr` implementation using nonintrusive reference counting leverages the small-object allocator, thus slashing the performance overhead caused by the reference counter.

7.5.4 Reference Linking

Reference linking relies on the observation that you don’t really need the actual count of smart pointer objects pointing to one pointee object; you only need to detect when that count goes down to zero. This leads to the idea of keeping an “ownership list,” as shown in Figure 7.5.¹

All `SmartPtr` objects that point to a given pointee form a doubly linked list. When you create a new `SmartPtr` from an existing `SmartPtr`, the new object is appended to the list; `SmartPtr`’s destructor takes care of removing the destroyed object from the list. When the list becomes empty, the pointee object is deleted.

The doubly linked list structure fits reference linking like a glove. You cannot use a singly linked list because removals from such a list take linear time. You cannot use a vector because the `SmartPtr` objects are not contiguous (and removals from vectors take linear time anyway). You need a structure sporting constant-time append, constant-time remove, and constant-time empty detection. This bill is fit precisely and exclusively by doubly linked lists.

¹Risto Lankinen described the reference-linking mechanism on the Usenet in November 1995.

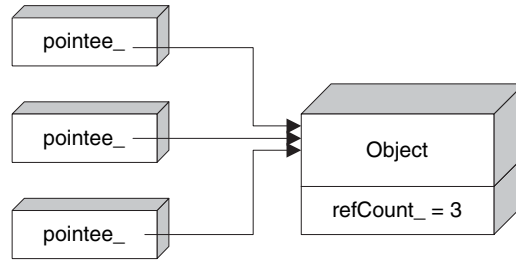


图 7.4: 侵入式引用计数

额外的间接级别。这是一个相当大的缺点，因为您通常会多次使用智能指针，而显然您只构造和销毁它一次。

最有效的解决方案是将引用计数器保存在指针对象本身中，如图 7.4 所示。这样 SmartPtr 就只是一个指针的大小，完全没有额外的开销。这种技术被称为 *intrusive reference counting*，因为引用计数是指针对象中的“入侵者”——它在语义上属于智能指针。该名称还暗示了该技术的致命弱点：您必须预先设计或修改指针类以支持引用计数。

通用智能指针应在可用的情况下使用侵入式引用计数，并实现非侵入式引用计数方案作为可接受的替代方案。为了实现非侵入式引用计数，第 4 章中介绍的小对象分配器可以提供很大帮助。使用非侵入式引用计数的 SmartPtr 实现利用了小对象分配器，从而减少了引用计数器造成的性能开销。

7.5.4 参考链接

引用链接依赖于这样的观察：您实际上并不需要指向一个被指针对象的智能指针对象的实际数量；您只需要检测该计数何时降至零。这就产生了保留“所有权列表”的想法，如图 7.5 所示。¹

所有指向给定指针对象的 SmartPtr 对象形成一个双向链表。当您从现有的 SmartPtr 创建新的 SmartPtr 时，新对象将附加到列表中；SmartPtr 的析构函数负责从列表中删除被破坏的对象。当列表变空时，指针对象将被删除。

双向链表结构像手套一样适合引用链接。您不能使用单链表，因为从此类列表中删除需要线性时间。您不能使用向量，因为 SmartPtr 对象不是连续的（并且从向量中删除无论如何都需要线性时间）。您需要一个具有恒定时间追加、恒定时间删除和恒定时间空检测的结构。该法案完全由双向链表精确拟合。

¹Risto Lankinen 于 1995 年 11 月描述了 Usenet 上的参考链接机制。

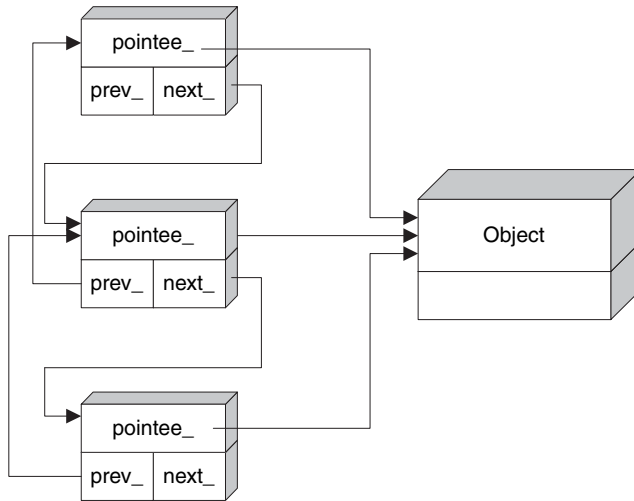


Figure 7.5: Reference linking in action

In a reference-linking implementation, each `SmartPtr` object holds two extra pointers—one to the next element and one to the previous element.

The advantage of reference linking over reference counting is that the former does not use extra free store, which makes it more reliable: Creating a reference-linked smart pointer cannot fail. The disadvantage is that reference linking needs more memory for its book-keeping (three pointers versus only one pointer plus one integer). Also, reference counting should be a bit speedier—when you copy smart pointers, only an indirection and an increment are needed. The list management is slightly more elaborate. In conclusion, you should use reference linking only when the free store is scarce. Otherwise, prefer reference counting.

To wrap up the discussion on reference count management strategies, let's note a significant disadvantage that they have. Reference management—be it counting or linking—is a victim of the resource leak known as *cyclic reference*. Imagine an object A holds a smart pointer to an object B. Also, object B holds a smart pointer to A. These two objects form a cyclic reference; even though you don't use any of them anymore, they use each other. The reference management strategy cannot detect such cyclic references, and the two objects remain allocated forever. The cycles can span multiple objects, closing circles that often reveal unexpected dependencies that are very hard to debug.

In spite of this, reference management is a robust, speedy ownership-handling strategy. If used with precaution, reference management makes application development significantly easier.

7.5.5 Destructive Copy

Destructive copy does exactly what you think it does: During copying, it destroys the object being copied. In the case of smart pointers, destructive copy destroys the source smart

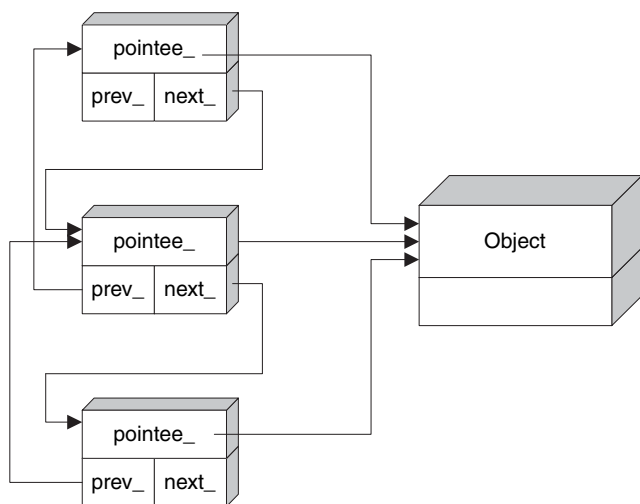


图 7.5: 实际的参考链接

在引用链接实现中，每个 `SmartPtr` 对象都包含两个额外的指针 - 一个指向下一个元素，一个指向前一个元素。

引用链接相对于引用计数的优点是前者不使用额外的空闲存储，这使得它更可靠：创建引用链接的智能指针不会失败。缺点是引用链接需要更多内存来进行簿记（三个指针与仅一个指针加一个整数）。此外，引用计数应该更快一点——当您复制智能指针时，只需要间接寻址和增量。列表管理稍微复杂一些。总之，只有当免费商店稀缺时，您才应该使用参考链接。否则，更喜欢引用计数。

为了结束对引用计数管理策略的讨论，让我们注意它们的一个显着缺点。引用管理（无论是计数还是链接）都是称为 *cyclic reference* 的资源泄漏的受害者。想象一下，对象 A 持有指向对象 B 的智能指针。此外，对象 B 持有指向 A 的智能指针。这两个对象形成循环引用；即使你不再使用它们中的任何一个，它们也会互相使用。引用管理策略无法检测到这种循环引用，并且这两个对象永远保持分配状态。这些循环可以跨越多个对象，闭合的循环通常会揭示难以调试的意外依赖关系。

尽管如此，参考管理是一种强大、快速的所有权处理策略。如果谨慎使用，参考管理会使应用程序开发变得更加容易。

7.5.5 破坏性复制

破坏性复制的作用正如您所想象的那样：在复制过程中，它会破坏被复制的对象。对于智能指针，破坏性复制会破坏源智能指针

pointer by taking its pointee object and passing it to the destination smart pointer. The `std::auto_ptr` class template features destructive copy.

In addition to being suggestive about the action taken, “destructive” also vividly describes the dangers associated with this strategy. Misusing destructive copy may have destructive effects on your program data, your program correctness, and your brain cells.

Smart pointers may use destructive copy to ensure that at any time there is only one smart pointer pointing to a given object. During the copying or assignment of one smart pointer to another, the “living” pointer is passed to the destination of the copy, and the source’s `pointee_` becomes zero. The following code illustrates a copy constructor and an assignment operator of a simple `SmartPtr` featuring destructive copy.

```
template <class T>
class SmartPtr
{
public:
    SmartPtr(SmartPtr& src)
    {
        pointee_ = src.pointee_;
        src.pointee_ = 0;
    }
    SmartPtr& operator=(SmartPtr& src)
    {
        if (this != &src)
        {
            delete pointee_;
            pointee_ = src.pointee_;
            src.pointee_ = 0;
        }
        return *this;
    }
    ...
};
```

C++ etiquette calls for the right-hand side of the copy constructor and the assignment operator to be a reference to a `const` object. Classes that foster destructive copy break this convention for obvious reasons. Because etiquette exists for a reason, you should expect negative consequences if you break it. Indeed, here they are:

```
void Display(SmartPtr<Something> sp);
...
SmartPtr<Something> sp(new Something);
Display(sp); // sinks sp
```

Although `Display` means no harm to its argument (accepts it by value), it acts like a maelstrom of smart pointers: It sinks any smart pointer passed to it. After `Display(sp)` is called, `sp` holds the null pointer.

Because they do not support value semantics, smart pointers with destructive copy cannot be stored in standard containers and in general must be handled with almost as much care as raw pointers.

The ability to store smart pointers in a container is very important. Containers of raw

指针，获取其指针对象并将其传递给目标智能指针。 `std::auto_ptr` 类模板具有破坏性复制功能。

除了暗示所采取的行动外，“破坏性”还生动地描述了与这一策略相关的危险。滥用破坏性副本可能会对您的程序数据、程序正确性和脑细胞产生破坏性影响。

智能指针可以使用破坏性复制来确保在任何时候只有一个智能指针指向给定的对象。在将一个智能指针复制或分配给另一个智能指针的过程中，“活动”指针被传递到复制的目的地，并且源的 `pointee_` 变为零。以下代码说明了具有破坏性复制的简单 `SmartPtr` 的复制构造函数和赋值运算符。

```
模板 <class T> class SmartPtr {public: SmartPtr(Smart
Ptr& src) { pointee_ = src.pointee_; src.pointee_ = 0; }
SmartPtr& 运算符=( SmartPtr& src) { if (this != &src
) { 删除 pointee_; pointee_ = src.pointee_; src.pointee_
= 0; }返回*这个; } ... };
```

C++ 规范要求复制构造函数和赋值运算符的右侧是对 `const` 对象的引用。出于显而易见的原因，促进破坏性复制的类打破了这一惯例。因为礼仪的存在是有原因的，所以如果你违反礼仪，你应该预料到会产生负面后果。事实上，它们在这里：

```
void Display(SmartPtr<Something> sp); ... SmartPtr<So
mething> sp(new Something);显示 (sp); // 接收 sp
```

尽管 `Display` 对其参数没有任何损害（按值接受它），但它的行为就像智能指针的漩涡：它会吸收传递给它的所有智能指针。调用 `Display(sp)` 后，`sp` 保存空指针。

由于它们不支持值语义，因此具有破坏性副本的智能指针无法存储在标准容器中，并且通常必须像处理原始指针一样小心处理。

在容器中存储智能指针的能力非常重要。原料容器

pointers make manual ownership management tricky, so many containers of pointers can use smart pointers to good advantage. Smart pointers with destructive copy, however, do not mix with containers.

On the bright side, smart pointers with destructive copy have significant advantages:

- They incur almost no overhead.
- They are good at enforcing ownership transfer semantics. In this case, you use the “maelstrom effect” described earlier to your advantage: You make it clear that your function takes over the passed-in pointer.
- They are good as return values from functions. If the smart pointer implementation uses a certain trick,² you can return smart pointers with destructive copy from functions. This way, you can be sure that the pointee object gets destroyed if the caller doesn’t use the return value.
- They are excellent as stack variables in functions that have multiple return paths. You don’t have to remember to delete the pointee object manually—the smart pointer takes care of this for you.

The destructive copy strategy is used by the standard-provided `std::auto_ptr`. This brings destructive copy another important advantage:

- Smart pointers with destructive copy semantics are the only smart pointers that the standard provides, which means that many programmers will get used to their behavior sooner or later.

For these reasons, the `SmartPtr` implementation should provide optional support for destructive copy semantics.

Smart pointers use various ownership semantics, each having its own trade-offs. The most important techniques are deep copy, reference counting, reference linking, and destructive copy. `SmartPtr` implements all these strategies through an `Ownership` policy, allowing its users to choose the one that best fits an application’s needs. The default strategy is reference counting.

7.6 The Address-of Operator

In striving to make smart pointers as indistinguishable as possible from their native counterparts, designers stumbled upon an obscure operator that is on the list of overloadable operators: unary operator `&`, the *address-of operator*.³

An implementer of smart pointers might choose to overload the address-of operator like this:

```
template <class T>
class SmartPtr
{
public:
```

²Invented by Greg Colvin and Bill Gibbons for `std::auto_ptr`.

³*Unary operator* `&` is to differentiate it from binary operator `&`, which is the bitwise AND operator.

指针使手动所有权管理变得棘手，因此许多指针容器可以使用智能指针来获得很好的优势。然而，具有破坏性复制的智能指针不会与容器混合。

从好的方面来看，具有破坏性复制的智能指针具有显著的优势：

- 它们几乎不产生任何开销。
- 他们擅长强制执行所有权转移语义。在这种情况下，您可以利用前面描述的“漩涡效应”来发挥自己的优势：您清楚地表明您的函数接管了传入的指针。
- 它们非常适合作为函数的返回值。如果智能指针实现使用某种技巧，² 您可以从函数返回带有破坏性副本的智能指针。这样，如果调用者不使用返回值，您可以确保被指针对象被销毁。
- 它们非常适合作具有多个返回路径的函数中的堆栈变量。您不必记住手动删除被指对象——智能指针会为您处理这件事。

标准提供的 `std::auto_ptr` 使用破坏性复制策略。这给破坏性复制带来了另一个重要的优势：

- 具有破坏性复制语义的智能指针是该标准提供的唯一智能指针，这意味着许多程序员迟早会习惯他们的行为。

由于这些原因，`SmartPtr` 实现应该为破坏性复制语义提供可选支持。

智能指针使用各种所有权语义，每种语义都有自己的权衡。最重要的技术是深度复制、引用计数、引用链接和破坏性复制。`SmartPtr` 通过所有权策略实施所有这些策略，允许用户选择最适合应用程序需求的策略。默认策略是引用计数。

7.6 地址运算符

在努力使智能指针与其本机指针尽可能难以区分的过程中，设计者偶然发现了可重载运算符列表中的一个不起眼的运算符：一元运算符`&`，即 *address-of operator*。³

智能指针的实现者可能会选择重载地址运算符，如下所示：

```
模板 <类 T> 类 SmartPtr
{public:
```

²由 Greg Colvin 和 Bill Gibbons 为 `std::auto_ptr` 发明。³*Unary*运算符`&`是为了区别二元运算符`&`，二元运算符`&`是按位与运算符。

```

T** operator&()
{
    return &pointee_;
}
...
};

```

After all, if a smart pointer is to simulate a pointer, then its address must be substitutable for the address of a regular pointer. This overload makes code like the following possible:

```

void Fun(Widget** pWidget);
...
SmartPtr<Widget> spWidget(...);
Fun(&spWidget); // okay, invokes operator& and obtains a
                // pointer to pointer to Widget

```

It seems very desirable to have such an accurate compatibility between smart pointers and dumb pointers, but overloading the unary `operator&` is one of those clever tricks that can do more harm than good. There are two reasons why overloading unary `operator&` is not a very good idea.

One reason is that exposing the address of the pointed-to object implies giving up any automatic ownership management. When a client freely accesses the address of the raw pointer, any helper structures that the smart pointer holds, such as reference counts, become invalid for all purposes. While the client deals directly with the address of the raw pointer, the smart pointer is completely unconscious.

The second reason, a more pragmatic one, is that overloading unary `operator&` makes the smart pointer unusable with STL containers. Actually, overloading unary `operator&` for a type pretty much makes generic programming impossible for that type, because the address of an object is too fundamental a property to play with naively. Most generic code assumes that applying `&` to an object of type `T` returns an object of type `T*`—you see, address-of is a fundamental concept. If you defy this concept, generic code behaves strangely either at compile time or—worse—at runtime.

Thus, it is not recommended that unary `operator&` be overloaded for smart pointers or for any objects in general. `SmartPtr` does not overload unary `operator&`.

7.7 Implicit Conversion to Raw Pointer Types

Consider this code:

```

void Fun(Something* p);
...
SmartPtr<Something> sp(new Something);
Fun(sp); // OK or error?

```

Should this code compile or not? Following the “maximum compatibility” line of thought, the answer is yes.

```
T** 运算符&() { return &poi
ntee_; } ... };
```

毕竟，如果智能指针要模拟指针，那么它的地址必须可以替换常规指针的地址。这种重载使得如下代码成为可能：

```
void Fun(Widget** pWidget); ... SmartPtr<Widget> spWidget(...); 乐趣 (&spWid
get); // 好的，调用operator& 并获取 // 指向Widget 的指针
```

智能指针和哑指针之间具有如此精确的兼容性似乎是非常理想的，但是重载一元运算符&是那些弊大于利的聪明技巧之一。有两个原因导致重载一元运算符&不是一个好主意。

原因之一是公开所指向对象的地址意味着放弃任何自动所有权管理。当客户端自由访问原始指针的地址时，智能指针保存的任何辅助结构（例如引用计数）对于所有目的都将变得无效。当客户端直接处理原始指针的地址时，智能指针是完全无意识的。

第二个原因（更务实的一个）是重载一元运算符&使智能指针无法与 STL 容器一起使用。实际上，为类型重载一元运算符&几乎使得该类型的泛型编程变得不可能，因为对象的地址是一个太基本的属性，无法天真地使用。大多数通用代码假设将 & 应用于 T 类型的对象会返回 T* 类型的对象 - 你看，address-of 是一个基本概念。如果你违背这个概念，通用代码要么在编译时，要么在运行时表现得很奇怪（更糟糕的是）。

因此，一般不建议为智能指针或任何对象重载一元运算符&。SmartPtr 不会重载一元运算符&。

7.7 到原始指针类型的隐式转换

s

考虑这段代码：

```
void Fun(某事* p); ... SmartPtr<Something> sp(new So
mething); 乐趣 (sp); // 正常还是错误？
```

这段代码是否应该编译？按照“最大兼容性”的思路，答案是肯定的。

Technically, it is very simple to render the previous code compilable by introducing a user-defined conversion:

```
template <class T>
class SmartPtr
{
public:
    operator T*() // user-defined conversion to T*
    {
        return pointee_;
    }
    ...
};
```

However, this is not the end of the story.

User-defined conversions in C++ have an interesting history. Back in the 1980s, when user-defined conversions were introduced, most programmers considered them a great invention. User-defined conversions promised a more unified type system, expressive semantics, and the ability to define new types that were indistinguishable from built-in ones. With time, however, user-defined conversions revealed themselves as awkward and potentially dangerous. They might become dangerous especially when they expose handles to internal data (Meyers 1998a, Item 29), which is precisely the case with the operator `T*` in the previous code. That's why you should think carefully before allowing automatic conversions for the smart pointers you design.

One potential danger comes inherently from giving the user unattended access to the raw pointer that the smart pointer wraps. Passing the raw pointer around defeats the inner workings of the smart pointer. Once unleashed from the confines of its wrapper, the raw pointer can easily become a threat to program sanity again, just as it was before smart pointers were introduced.

Another danger is that user-defined conversions pop up unexpectedly, even when you don't need them. Consider the following code:

```
SmartPtr<Something> sp;
...
// A gross semantic error
// However, it goes undetected at compile time
delete sp;
```

The compiler matches operator `delete` with the user-defined conversion to `T*`. At runtime, operator `T*` is called, and `delete` is applied to its result. This is certainly not what you want to do to a smart pointer, because it is supposed to manage ownership itself. An extra unwitting `delete` call throws out the window all the careful ownership management that the smart pointer performs under the covers.

There are quite a few ways to prevent the `delete` call from compiling. Some of them are very ingenious (Meyers 1996). One that's very effective and easy to implement is to make the call to `delete` intentionally *ambiguous*. You can achieve this by providing *two* automatic conversions to types that are susceptible to a call to `delete`. One type is `T*` itself, and the other can be `void*`.

从技术上来说，通过引入用户定义的转换来使之前的代码变得可编译是非常简单的：

```
template <class T> class SmartPtr {public: operator T*() // 用户定义的  
到 T* 的转换 { return pointee_; } } ... };
```

然而，这并不是故事的结局。

C++ 中用户定义的转换有着有趣的历史。早在 20 世纪 80 年代，当用户定义的转换被引入时，大多数程序员都认为它们是一项伟大的发明。用户定义的转换承诺了更统一的类型系统、表达语义以及定义与内置类型没有区别的新类型的能力。然而，随着时间的推移，用户定义的转换暴露出自己的尴尬和潜在的危险。它们可能会变得危险，尤其是当它们公开内部数据的句柄时（Meyers 1998a，第 29 项），这正是前面代码中的运算符 T* 的情况。这就是为什么在允许自动转换您设计的智能指针之前您应该仔细考虑。

一种潜在的危险本质上来来自于让用户在无人值守的情况下访问智能指针包装的原始指针。传递原始指针会破坏智能指针的内部工作原理。一旦从包装器的限制中释放出来，原始指针很容易再次成为对程序健全性的威胁，就像在引入智能指针之前一样。

另一个危险是用户定义的转换会意外弹出，即使您不需要它们。考虑以下代码：

```
SmartPtr<某事> sp; ... // 一个严重的语义错误 // 但是，它在编译  
时未被检测到 delete sp;
```

编译器将删除运算符与用户定义的 T* 转换相匹配。在运行时，调用运算符 T*，并将删除应用于其结果。这当然不是您想要对智能指针执行的操作，因为它应该自行管理所有权。额外的不知情的删除调用将智能指针在幕后执行的所有仔细的所有权管理抛到了九霄云外。

有很多方法可以防止删除调用编译。其中一些非常巧妙（Meyers 1996）。一种非常有效且易于实现的方法是有意调用删除 *ambiguous*。您可以通过提供 *two* 自动转换为易于调用删除的类型来实现此目的。一种类型是 T* 本身，另一种可以是 void*。

```

template <class T>
class SmartPtr
{
public:
    operator T*() // User-defined conversion to T*
    {
        return pointee_;
    }
    operator void*() // Added conversion to void*
    {
        return pointee_;
    }
    ...
};

```

A call to `delete` against such a smart pointer object is ambiguous. The compiler cannot decide which conversion to apply, and the trick above exploits this indecision to good advantage.

Don't forget that disabling the `delete` operator was only a part of the issue. Whether to provide an automatic conversion to a raw pointer remains an important decision in implementing a smart pointer. It's too dangerous just to let it in, yet too convenient to rule it out. The final `SmartPtr` implementation will give you a choice about that.

However, forbidding implicit conversion does not necessarily eliminate all access to the raw pointer; it is often necessary to gain such access. Therefore, all smart pointers do provide *explicit* access to their wrapped pointer via a call to a function:

```

void Fun(Something* p);
...
SmartPtr<Something> sp;
Fun(GetImpl(sp)); // OK, explicit conversion always allowed

```

It's not a matter of whether you can get to the wrapped pointer; it's how easy it is. This may seem like a minor difference, but it's actually very important. An implicit conversion happens without the programmer or the maintainer noticing or even knowing it. An explicit conversion—as is the call to `GetImpl`—passes through the mind, the understanding, and the fingers of the programmer and remains written in the code for everybody to see it.

Implicit conversion from the smart pointer type to the raw pointer type is desirable, but sometimes dangerous. `SmartPtr` provides this implicit conversion as a choice. The default is on the safe side—no implicit conversions. Explicit access is always available through the `GetImpl` function.

7.8 Equality and Inequality

C++ teaches its users that any clever trick such as the one presented in the previous section (intentional ambiguity) establishes a new context, which in turn may have unexpected ripples.

Consider tests for equality and inequality of smart pointers. A smart pointer should

```
template <class T> class SmartPtr {public: operator T*() // 用户定义的  
到 T* 的转换 { return pointee_; } }operator void*() // 添加了对 void*  
的转换 { return pointee_; } ... };
```

针对此类智能指针对象的删除调用是不明确的。编译器无法决定应用哪种转换，而上面的技巧充分利用了这种犹豫不决的优势。

不要忘记禁用删除运算符只是问题的一部分。是否提供到原始指针的自动转换仍然是实现智能指针的一个重要决定。仅仅让它进来太危险，但排除它又太方便了。最终的 SmartPtr 实现将为您提供一个选择。

然而，禁止隐式转换并不一定消除对原始指针的所有访问；通常需要获得此类访问权限。因此，所有智能指针都通过调用函数提供对其包装指针的 *explicit* 访问：

```
void Fun(某事* p); ... SmartPtr<某事> sp;乐趣 (GetImpl (sp) ); // 好的，始终允  
许显式转换
```

这不是能否到达包装指针的问题；而是问题。就是这么简单。这看起来可能是一个很小的差异，但实际上非常重要。隐式转换的发生是在程序员或维护者没有注意到甚至不知道的情况下发生的。显式转换（如对 GetImpl 的调用）会经过程序员的思想、理解和手指，并保留在代码中以供每个人查看。

从智能指针类型到原始指针类型的隐式转换是可取的，但有时很危险。SmartPtr 提供这种隐式转换作为一种选择。默认是为了安全起见——没有隐式转换。始终可以通过 GetImpl 函数进行显式访问。

7.8 平等与不平等

C++ 告诉用户，任何巧妙的技巧（例如上一节中提出的技巧（故意模糊））都会建立一个新的上下文，这反过来可能会产生意想不到的涟漪。

考虑对智能指针的相等和不相等的测试。智能指针应该

support the same comparison syntax that raw pointers support. Programmers expect the following tests to compile and run as they do for a raw pointer.

```
SmartPtr<Something> sp1, sp2;
Something* p;
...
if (sp1)    // Test 1: direct test for non-null pointer
...
if (!sp1)   // Test 2: direct test for null pointer
...
if (sp1 == 0) // Test 3: explicit test for null pointer
...
if (sp1 == sp2) // Test 4: comparison of two smart pointers
...
if (sp1 == p)  // Test 5: comparison with a raw pointer
...
```

There are more tests than depicted here if you consider symmetry and operator!=. If we solve the equality tests, we can easily define the corresponding symmetric and inequality tests.

There is an unfortunate interference between the solution to the previous issue (preventing delete from compiling) and a possible solution to this issue. With one user-defined conversion to the raw pointer type, most of the test expressions (except test 4) compile successfully and run as expected. The downside is that you can accidentally call the delete operator against the smart pointer. With two user-defined conversions (intentional ambiguity), you detect wrongful delete calls, but none of these tests compiles anymore—they have become ambiguous too.

An additional user-defined conversion to bool helps, but this, to nobody's surprise, introduces new trouble. Given this smart pointer:

```
template <class T>
class SmartPtr
{
public:
    operator bool() const
    {
        return pointee_ != 0;
    }
    ...
};
```

the four tests compile, but so do the following nonsensical operations:

```
SmartPtr<Apple> sp1;
SmartPtr<Orange> sp2; // Orange is unrelated to Apple
if (sp1 == sp2)       // Converts both pointers to bool
                      // and compares results
...
if (sp1 != sp2)       // Ditto
...
```

支持与原始指针支持的相同比较语法。程序员希望以下测试能够像原始指针一样编译和运行。

```
SmartPtr<某事> sp1, sp2; 某事* p; ... if (sp1) // 测试 1: 直接测试非空指针 ... if (!sp1)
// 测试 2: 直接测试空指针 ... if (sp1 == 0) // 测试 3: 显式测试空指针 ... if (sp1 ==
sp2) // 测试 4: 比较两个智能指针 ... if (sp1 == p) // 测试 5: 与原始指针比较 ...
```

如果考虑对称性和运算符，还有比此处描述的更多的测试！`==`。如果我们解决了等式检验，我们就可以轻松定义相应的对称和不等式检验。

不幸的是，上一问题的解决方案（防止编译时删除）与此问题的可能解决方案之间存在干扰。通过用户定义的到原始指针类型的转换，大多数测试表达式（测试 4 除外）都可以成功编译并按预期运行。缺点是您可能会意外地对智能指针调用删除运算符。通过两个用户定义的转换（故意模糊），您可以检测到错误的删除调用，但这些测试都不再编译——它们也变得模糊了。

额外的用户定义的 `bool` 转换会有所帮助，但毫无疑问，这会带来新的麻烦。给定这个智能指针：

```
模板 <class T> class SmartPtr {public:
运算符 bool() const { return pointee_ !
= 0; } ... };
```

佛 你的测试可以编译，但是以下无意义的操作也可以编译 化：

```
SmartPtr<Apple> sp1;
SmartPtr<Orange> sp2; // Orange is unrelated to Apple
if (sp1 == sp2)       // Converts both pointers to bool
                      // and compares results
...
if (sp1 != sp2)       // Ditto
...

```

```

bool b = sp1;           // The conversion allows this, too
if (sp1 * 5 == 200)     // Ouch! SmartPtr behaves like an integral
                        // type!
    ...

```

As you can see, it's either not at all or too much: Once you add a user-defined conversion to `bool`, you allow `SmartPtr` to act as a `bool` in many more situations than you actually wanted. For all practical purposes, defining an operator `bool` for a smart pointer is not a smart solution.

A true, complete, rock-solid solution to this dilemma is to go all the way and overload each and every operator separately. This way any operation that makes sense for the bare pointer makes sense for the smart pointer, and nothing else. Here is the code that implements this idea.

```

template <class T>
class SmartPtr
{
public:
    bool operator!() const // Enables "if (!sp) ..."
    {
        return pointee_ == 0;
    }
    inline friend bool operator==(const SmartPtr& lhs,
                                  const T* rhs)
    {
        return lhs.pointee_ == rhs;
    }
    inline friend bool operator==(const T* lhs,
                                  const SmartPtr& rhs)
    {
        return lhs == rhs.pointee_;
    }
    inline friend bool operator!=(const SmartPtr& lhs,
                                  const T* rhs)
    {
        return lhs.pointee_ != rhs;
    }
    inline friend bool operator!=(const T* lhs,
                                  const SmartPtr& rhs)
    {
        return lhs != rhs.pointee_;
    }
    ...
};

```

Yes, it's a pain, but this approach solves the problems with almost all comparisons, including the tests against the literal zero. What the forwarding operators in this code do is to pass operators that client code applies to the smart pointer on to the raw pointer that the smart pointer wraps. No simulation can be more realistic than that.

We still haven't solved the problem completely. If you provide an automatic conversion

```
布尔 b = sp1; // 转换也允许这样做 if (sp1 * 5 == 200) // 哎呀! SmartPtr 的行为类似于整型 // 类型! ...
```

正如您所看到的, 它要么根本不存在, 要么太多: 一旦您将用户定义的转换添加到 `bool` 中, 您就允许 `SmartPtr` 在比您实际想要的更多的情况下充当 `bool`。出于所有实际目的, 为智能指针定义运算符布尔并不是一个明智的解决方案。

解决这一困境的真正、完整、坚如磐石的解决方案是一路走来, 分别让每个操作员超载。这样, 任何对裸指针有意义的操作对智能指针都有意义, 而不是其他。这是实现这个想法的代码。

```
template <class T> class SmartPtr {public: bool operator!() const // 启用 “if (!
sp) ...” { return pointee_ == 0; }内联友元布尔运算符==(const SmartPtr& l
hs, const T* rhs) { return lhs.pointee_ == rhs; }内联友元布尔运算符==(con
st T* lhs, const SmartPtr& rhs) { return lhs == rhs.pointee_; }内联友元布尔
运算符!=(const SmartPtr& lhs, const T* rhs) { return lhs.pointee_ != rhs; }
内联友元布尔运算符!=(const T* lhs, const SmartPtr& rhs) { return lhs != r
hs.pointee_; } ... };
```

是的, 这很痛苦, 但这种方法解决了几乎所有比较的问题, 包括针对文字零的测试。此代码中的转发运算符的作用是将客户端代码应用于智能指针的运算符传递到智能指针包装的原始指针。没有比这更真实的模拟了。

我们还没有完全解决问题。如果您提供自动转换

to the pointee type, there still is the risk of ambiguities. Suppose you have a class `Base` and a class `Derived` that inherits `Base`. Then the following code makes practical sense yet is ill formed due to ambiguity.

```
SmartPtr<Base> sp;
Derived* p;
...
if (sp == p) {} // error! Ambiguity between:
                // '(Base*)sp == (Base*)p'
                // and 'operator==(sp, (Base*)p)'
```

Indeed, smart pointer development is not for the faint of heart.

We're not out of bullets, though. In addition to the definitions of `operator==` and `operator!=`, we can add *templated* versions of them, as you can see in the following code:

```
template <class T>
class SmartPtr
{
public:
    ... as above ...
    template <class U>
    inline friend bool operator==(const SmartPtr& lhs,
        const U* rhs)
    {
        return lhs.pointee_ == rhs;
    }
    template <class U>
    inline friend bool operator==(const U* lhs,
        const SmartPtr& rhs)
    {
        return lhs == rhs.pointee_;
    }
    ... similarly defined operator!= ...
};
```

The templated operators are “greedy” in the sense that they match comparisons with any pointer type whatsoever, thus consuming the ambiguity.

If that's the case, why should we keep the nontemplated operators—the ones that take the pointee type? They never get a chance to match, because the template matches any pointer type, including the pointee type itself.

The rule that “never” actually means “almost never” applies here, too. In the test `if (sp == 0)`, the compiler tries the following matches.

- *The templated operators.* They don't match because the type of literal zero is not a pointer type. A literal zero can be implicitly converted to a pointer type, but template matching does not include conversions.
- *The nontemplated operators.* After eliminating the templated operators, the compiler tries the nontemplated ones. One of these operators kicks in through an implicit conversion from the literal zero to the pointee type. Had the nontemplated operators not existed, the test would have been an error.

对于指针类型来说，仍然存在歧义的风险。假设您有一个类 `Base` 和一个继承 `Base` 的类 `Derived`。那么下面的代码具有实际意义，但由于不明确而格式不正确。

```
SmartPtr<Base> sp; 派生* p; ... if (sp == p) {} // 错误! //(Base*)sp
== (Base*)p' // 和 'operator==(sp, (Base*)p)' 之间存在歧义
```

在确实，智能指针开发不适合胆小的人 艺术。

不过，我们的子弹还没有用完。除了 `operator==` 和 `operator!=` 的定义之外，我们还可以添加它们的 *templated* 版本，如以下代码所示：

```
template <class T> class SmartPtr {public: ... 如上所述 ... template <class U>
内联友元布尔运算符==(const SmartPtr& lhs, const U* rhs) { return lhs.poin
tee_ == rhs; } 模板 <class U> 内联友元布尔运算符==(const U* lhs, const S
martPtr& rhs) { return lhs == rhs.pointee_; }...类似定义的运算符! = ... };
```

模板化运算符是“贪婪的”，因为它们匹配与任何指针类型的比较，从而消除了歧义。

如果是这样的话，我们为什么要保留非模板化运算符——那些采用指针类型的运算符呢？它们永远没有机会匹配，因为模板匹配任何指针类型，包括被指针类型本身。

“从不”实际上意味着“几乎从不”的规则也适用于此。在测试 `if (sp == 0)` 中，编译器尝试以下匹配。

- *The templated operators.* 它们不匹配，因为文字零的类型不是指针类型。文字零可以隐式转换为指针类型，但模板匹配不包括转换。
- *The nontemplated operators.* 消除模板化运算符后，编译器会尝试非模板化运算符。这些运算符之一通过从文字零到指针类型的隐式转换来启动。如果非模板化运算符不存在，测试就会出错。

In conclusion, we need *both* the nontemplated and the templated comparison operators.

Let's see now what happens if we compare two `SmartPtr`s instantiated with different types.

```
SmartPtr<Apple> sp1;
SmartPtr<Orange> sp2;
if (sp1 == sp2)
    ...
```

The compiler chokes on the comparison because of an ambiguity: Each of the two `SmartPtr` instantiations defines an `operator==`, and the compiler does not know which one to choose. We can dodge this problem by defining an “ambiguity buster” as shown:

```
template <class T>
class SmartPtr
{
public:
    // Ambiguity buster
    template <class U>
    bool operator==(const SmartPtr<U>& rhs) const
    {
        return pointee_ == rhs.pointee_;
    }
    // Similarly for operator!=
    ...
};
```

This newly added operator is a member that specializes exclusively in comparing `SmartPtr<...>` objects. The beauty of this ambiguity buster is that it makes smart pointer comparisons act like raw pointer comparisons. If you compare two smart pointers to `Apple` and `Orange`, the code will be essentially equivalent to comparing two raw pointers to `Apple` and `Orange`. If the comparison makes sense, then the code compiles; otherwise, it's a compile-time error.

```
SmartPtr<Apple> sp1;
SmartPtr<Orange> sp2;
if (sp1 == sp2) // Semantically equivalent to
                // sp1.pointee_ == sp2.pointee_
    ...
```

There is one unsatisfied syntactic artifact left, namely, the direct test `if (sp)`. Here life becomes really interesting. The `if` statement applies only to expressions of arithmetic and pointer type. Consequently, to allow `if (sp)` to compile, we must define an automatic conversion to either an arithmetic or a pointer type.

A conversion to arithmetic type is not recommended, as the earlier experience with `operator bool` witnesses. A pointer is not an arithmetic type, period. A conversion to a pointer type makes a lot more sense, and here the problem branches.

If you want to provide automatic conversions to the pointee type (see previous section), then you have two choices: Either you risk unattended calls to `operator delete`, or you

总之，我们需要 *both* 非模板化和模板化比较运算符。现在让我们看看如果比较两个用不同类型实例化的 `SmartPtr` 会发生什么。

```
SmartPtr<苹果> sp1; SmartPtr<
  橙色> sp2; 如果 (sp1 == sp2) ..
```

.

编译器因比较而陷入歧义：两个 `SmartPtr` 实例化中的每一个都定义了一个运算符 `==`，而编译器不知道选择哪一个。我们可以通过定义一个“歧义破坏器”来避免这个问题，如下所示：

```
template <class T> class SmartPtr {public: // 歧义消除器
  template <class
    U> bool operator==(const SmartPtr<U>& rhs) const { return point
      ee_ == rhs.pointee_; } // 运算符类似！ = ... };
```

这个新添加的运算符是专门用于比较 `SmartPtr<...>` 对象的成员。这种歧义消除器的优点在于它使智能指针比较就像原始指针比较一样。如果将两个智能指针与 `Apple` 和 `Orange` 进行比较，则该代码本质上相当于将两个原始指针与 `Apple` 和 `Orange` 进行比较。如果比较有意义，则代码可以编译；否则，这是一个编译时错误。

```
SmartPtr<苹果> sp1; SmartPtr<橙色> sp2; if (sp1 == sp2) // 语义上等
  价于 // sp1.pointee_ == sp2.pointee_ ...
```

剩下一个不令人满意的语法工件，即直接测试 `if (sp)`。在这里，生活变得非常有趣。if 语句仅适用于算术和指针类型的表达式。因此，为了允许 `if (sp)` 编译，我们必须定义一个到算术类型或指针类型的自动转换。

不建议转换为算术类型，因为早期使用运算符 `bool` 的经验证明了这一点。指针不是算术类型，句点。转换为指针类型更有意义，但问题就在这里出现了。

如果您想提供到 `pointee` 类型的自动转换（请参阅上一节），那么您有两种选择：要么冒着无人值守调用操作员删除的风险，要么您

forgo the `if (sp)` test. The tiebreaker is between the lack of convenience and a risky life. The winner is safety, so you cannot write `if (sp)`. Instead, you can choose between `if(sp != 0)` and the more baroque `if (!!sp)`. End of story.

If you don't want to provide automatic conversions to the pointee type, there is an interesting trick you can use to make `if (sp)` possible. Inside the `SmartPtr` class template, define an inner class `Tester` and define a conversion to `Tester*`, as shown in the following code:

```
template <class T>
class SmartPtr
{
    class Tester
    {
        void operator delete(void*);
    };
public:
    operator Tester*() const
    {
        if (!pointee_) return 0;
        static Tester test;
        return &test;
    }
    ...
};
```

Now if you write `if (sp)`, `operator Tester*` enters into action. This operator returns a null value if and only if `pointee_` is null. `Tester` itself disables `operator delete`, so if somebody calls `delete sp`, a compile-time error occurs. Interestingly, `Tester`'s definition itself lies in the private part of `SmartPtr`, so the client code cannot do anything else with it.

`SmartPtr` addresses the issue of tests for equality and inequality as follows:

- Define `operator==` and `operator!=` in two flavors (templated and nontemplated).
- Define `operator!`.
- If you allow automatic conversion to the pointee type, then define an additional conversion to `void*` to ambiguate a call to the `delete` operator intentionally; otherwise, define a private inner class `Tester` that declares a private `operator delete`, and define a conversion to `Tester*` for `SmartPtr` that returns a null pointer if and only if the `pointee_` is null.

7.9 Ordering Comparisons

The ordering comparison operators are `operator<`, `operator<=`, `operator>`, and `operator>=`. You can implement them all in terms of `operator<`.

Whether to allow ordering of smart pointers is an interesting question in and of itself and relates to the dual nature of pointers, which consistently confuses programmers. Pointers are two concepts in one: iterators and monikers. The iterative nature of pointers allows you to walk through an array of objects using a pointer. Pointer arithmetic, including com-

放弃 if (sp) 测试。决定胜负的是缺乏便利和危险的生活。胜者为安全，所以不能写 if(sp)。相反，您可以在 if(sp != 0) 和更巴洛克式的 if (!!sp) 之间进行选择。故事结束。

如果您不想提供到指针类型的自动转换，可以使用一个有趣的技巧来使 if (sp) 成为可能。在 SmartPtr 类模板中，定义一个内部类 Tester 并定义到 Tester* 的转换，如以下代码所示：

```
模板 <类 T> 类 SmartPtr { 类 Tester { void 运算
符删除(void*); }; public: 运算符 Tester*() const
{ if (!pointee_) return 0;静态测试仪测试; 返回&
测试; } ... };
```

现在，如果您编写 if (sp)，运算符 Tester* 就会开始执行。当且仅当 pointee_ 为 null 时，此运算符返回 null 值。测试器本身禁用运算符 delete，因此如果有人调用 delete sp，则会发生编译时错误。有趣的是，Tester 的定义本身位于 SmartPtr 的私有部分中，因此客户端代码不能用它做任何其他事情。

SmartPtr 解决了平等和不平等的测试问题，如下所示：

- 以两种风格（模板化和非模板化）定义 operator== 和 operator!=。
- 定义运算符！
- 如果允许自动转换为指针对象类型，则定义一个额外的 void* 转换，以故意模糊对删除运算符的调用；否则，定义一个声明私有运算符 delete 的私有内部类 Tester，并为 SmartPtr 定义一个到 Tester* 的转换，当且仅当 pointee_ 为 null 时返回一个空指针。

7.9 排序比较

排序比较运算符是 operator<、operator<=、operator> 和 operator>=。您可以根据运算符 < 来实现它们。

是否允许智能指针排序本身就是一个有趣的问题，并且与指针的双重性质有关，这一直让程序员感到困惑。指针是两个概念合二为一的：迭代器和名字对象。指针的迭代性质允许您使用指针遍历对象数组。指针算术，包括 com-

parisons, supports this iterative nature of pointers. At the same time, pointers are monikers—inexpensive object representatives that can travel quickly and access the objects in a snap. The dereferencing operators `*` and `->` support the moniker concept.

The two natures of pointers can be confusing at times, especially when you need only one of them. For operating with a vector, you might use both iteration and dereferencing, whereas for walking through a linked list or for manipulating individual objects, you use only dereferencing.

Ordering comparisons for pointers is defined only when the pointers belong to the same contiguous memory. In other words, you can use ordering comparisons only for pointers that point to elements in the same array.

Defining ordering comparisons for smart pointers boils down to this question: Do smart pointers to the objects in the same array make sense? On the face of it, the answer is no. Smart pointers' main feature is to manage object ownership, and objects with separate ownership do not usually belong to the same array. Therefore, it would be dangerous to allow users to make nonsensical comparisons.

If you really need ordering comparisons, you can always use explicit access to the raw pointer. The issue here is, again, to find the safest and most expressive behavior for most situations.

The previous section concludes that an implicit conversion to a raw pointer type is optional. If `SmartPtr`'s client chooses to allow implicit conversion, the following code compiles:

```
SmartPtr<Something> sp1, sp2;
if (sp1 < sp2) // Converts sp1 and sp2 to raw pointer type,
               // then performs the comparison
...

```

This means that if we want to disable ordering comparisons, we must be proactive, disabling them explicitly. A way of doing this is to declare them and never define them, which means that any use will trigger a link-time error.

```
template <class T>
class SmartPtr
{ ... };

template <class T, class U>
bool operator<(const SmartPtr<T>&, const U&); // Not defined
template <class T, class U>
bool operator<(const T&, const SmartPtr<U>&); // Not defined

```

However, it is wiser to define all other operators in terms of `operator<`, as opposed to leaving them undefined. This way, if `SmartPtr`'s users think it's best to introduce smart pointer ordering, they need only define `operator<`.

```
// Ambiguity buster
template <class T, class U>
bool operator<(const SmartPtr<T>& lhs, const SmartPtr<U>& rhs)
{
    return lhs < GetImpl(rhs);
}

```

parisons, 支持指针的这种迭代性质。同时, 指针也是绰号——廉价的对象代表, 可以快速移动并快速访问对象。解引用运算符 * 和 -> 支持名字概念。

指针的两种性质有时可能会令人困惑, 特别是当您只需要其中一种时。对于向量操作, 您可以同时使用迭代和取消引用, 而对于遍历链接列表或操作单个对象, 您只能使用取消引用。

仅当指针属于同一连续内存时才定义指针比较的顺序。换句话说, 只能对指向同一数组中元素的指针使用排序比较。

定义智能指针的排序比较可以归结为这个问题: 指向同一数组中的对象的智能指针有意义吗? 从表面上看, 答案是否定的。智能指针的主要功能是管理对象所有权, 具有单独所有权的对象通常不属于同一个数组。因此, 允许用户进行无意义的比较是危险的。

如果您确实需要排序比较, 则始终可以使用对原始指针的显式访问。这里的问题是, 在大多数情况下找到最安全、最具表现力的行为。

上一节得出的结论是, 到原始指针类型的隐式转换是可选的。如果 SmartPtr 的客户端选择允许隐式转换, 则可以编译以下代码:

```
SmartPtr<某事> sp1, sp2; if (sp1 < sp2) // 将 sp1 和 sp2 转换为原始指针类型, // 然后执行比较 ...
```

这意味着如果我们想要禁用排序比较, 我们必须主动地显式禁用它们。一种方法是声明它们而不定义它们, 这意味着任何使用都会触发链接时错误。

```
模板 <类 T> 类 SmartPtr
{ ... };
```

```
模板 <类 T, 类 U> 布尔运算符<(const SmartPtr<T>&, const U&); // 未定义模板
板 <class T, class U> bool operator<(const T&, const SmartPtr<U>&); // 未定义
```

然而, 更明智的做法是根据 operator< 定义所有其他运算符, 而不是不定义它们。这样, 如果 SmartPtr 的用户认为最好引入智能指针排序, 他们只需要定义 operator<。

```
// 歧义消除模板 <class T, class U> bool 运算符<(const SmartPtr<T>& lhs, const SmartPtr
<U>& rhs) { return lhs < GetImpl(rhs);
```

```

}
// All other operators
template <class T, class U>
bool operator>(SmartPtr<T>& lhs, const U& rhs)
{
    return rhs < lhs;
}
... similarly for the other operators ...

```

Note the presence, again, of an ambiguity buster. Now if some library user thinks that `SmartPtr<Widget>` should be ordered, the following code is the ticket:

```

inline bool operator<(const SmartPtr<Widget>& lhs,
    const Widget* rhs)
{
    return GetImpl(lhs) < rhs;
}

inline bool operator<(const Widget* lhs,
    const SmartPtr<Widget>& rhs)
{
    return lhs < GetImpl(rhs);
}

```

It's a pity that the user must define two operators instead of one, but it's so much better than defining eight.

This would conclude the issue of ordering, were it not for an interesting detail. Sometimes it is very useful to have an ordering of arbitrarily located objects, not just objects belonging to the same array. For example, you might need to store supplementary per-object information, and you need to access that information quickly. A map ordered by the address of objects is very effective for such a task.

Standard C++ helps in implementing such designs. Although pointer comparison for arbitrarily located objects is undefined, the standard guarantees that `std::less` yields meaningful results for any two pointers of the same type. Because the standard associative containers use `std::less` as the default ordering relationship, you can safely use maps that have pointers as keys.

`SmartPtr` should support this idiom, too; therefore, `SmartPtr` specializes `std::less`. The specialization simply forwards the call to `std::less` for regular pointers:

```

namespace std
{
    template <class T>
    struct less<SmartPtr<T> >
        : public binary_function<SmartPtr<T>, SmartPtr<T>, bool>
    {
        bool operator()(const SmartPtr<T>& lhs,
            const SmartPtr<T>& rhs) const
        {
            return less<T*>()(GetImpl(lhs), GetImpl(rhs));
        }
    };
}

```

```
// 所有其他运算符模板 <class T, class U> bool operator> (SmartPtr<T>& lhs, const U& rhs) { return rhs < lhs; } ...对于其他运算符类似...
```

再次注意歧义消除器的存在。现在，如果某个库用户认为应该订购 `SmartPtr<Widget>`，则以下代码就是票证：

```
内联布尔运算符<(const SmartPtr<Widget>& lhs, const Widget* rhs) { return GetImpl(lhs) < rhs; }内联布尔运算符<(const Widget* lhs, const SmartPtr<Widget>& rhs) { return lhs < GetImpl(rhs); }
```

遗憾的是，用户必须定义两个运算符而不是一个，但这比定义八个要好得多。

如果没有一个有趣的细节，订购问题就到此结束了。有时，对任意位置的对象进行排序非常有用，而不仅仅是属于同一数组的对象。例如，您可能需要存储每个对象的补充信息，并且需要快速访问该信息。按对象地址排序的地图对于此类任务非常有效。

标准 C++ 有助于实现此类设计。尽管未定义任意位置对象的指针比较，但该标准保证 `std::less` 对于相同类型的任何两个指针都会产生有意义的结果。由于标准关联容器使用 `std::less` 作为默认排序关系，因此您可以安全地使用以指针作为键的映射。

`SmartPtr` 也应该支持这个习惯用法；因此，`SmartPtr` 专门针对 `std::less`。对于常规指针，特化只是将调用转发到 `std::less`：

```
命名空间 std { 模板 <class T> struct less<SmartPtr<T>> : 公共 binary_function<SmartPtr<T>, SmartPtr<T>, bool> { bool 运算符 () (const SmartPtr<T>& lhs, const SmartPtr<T>& rhs) const { return less<T*>() (GetImpl(lhs), GetImpl(rhs)); } } }
```

In summary, `SmartPtr` does not define ordering operators by default. It declares—without implementing—two generic `operator<s` and implements all other ordering operators in terms of `operator<`. The user can define either specialized or generic versions of `operator<`.

`SmartPtr` specializes `std::less` to provide an ordering of arbitrary smart pointer objects.

7.10 Checking and Error Reporting

Applications need various degrees of safety from smart pointers. Some programs are computation-intensive and must be optimized for speed, whereas some others (actually, most) are input/output intensive, which allows better runtime checking without degrading performance.

Most often, right inside an application, you might need both models: low safety/high speed in some critical areas, and high safety/lower speed elsewhere.

We can divide checking issues with smart pointers into two categories: initialization checking and checking before dereference.

7.10.1 Initialization Checking

Should a smart pointer accept the null (zero) value?

It is easy to implement a guarantee that a smart pointer cannot be null, and it may be very useful in practice. It means that any smart pointer is always valid (unless you fiddle with the raw pointer by using `GetImplRef`). The implementation is easy with the help of a constructor that throws an exception if passed a null pointer.

```
template <class T>
class SmartPtr
{
public:
    SmartPtr(T* p) : pointee_(p)
    {
        if (!p) throw NullPointerException();
    }
    ...
};
```

On the other hand, the null value is a convenient “not a valid pointer” placeholder and can often be useful.

Whether to allow null values affects the default constructor, too. If the smart pointer doesn’t allow null values, then how would the default constructor initialize the raw pointer? The default constructor could be lacking, but that would make smart pointers harder to deal with. For example, what should you do when you have a `SmartPtr` member variable but don’t have an appropriate initializer for it at construction time? In conclusion, customizing initialization involves providing an appropriate default value.

综上所述，`SmartPtr` 默认情况下没有定义排序运算符。它声明（但没有实现）两个通用运算符`<`，并根据运算符`<`实现所有其他排序运算符。用户可以定义运算符`<`的专用版本或通用版本。

`SmartPtr` 专门为 `std::less` 提供任意智能指针对象的排序。

7.10 检查和错误报告

应用程序需要智能指针提供不同程度的安全性。有些程序是计算密集型的，必须针对速度进行优化，而另一些程序（实际上是大多数）是输入/输出密集型的，这可以在不降低性能的情况下进行更好的运行时检查。

大多数情况下，在应用程序内部，您可能需要两种模型：某些关键区域的低安全性/高速度，以及其他地方的高安全性/低速度。

我们可以将智能指针的检查问题分为两类：初始化检查和解引用前检查。

7.10.1 初始化检查

智能指针是否应该接受空（零）值？

很容易实现智能指针不能为空的保证，并且在实践中可能非常有用。这意味着任何智能指针始终有效（除非您使用 `GetImplRef` 摆弄原始指针）。在构造函数的帮助下，实现很容易，如果传递空指针，该构造函数将引发异常。

```
模板 <class T> class SmartPtr {public: SmartPtr(T* p) : pointee
    _(p) { if (!p) throw NullPointerException(); } } ... };
```

另一方面，空值是一个方便的“不是有效指针”占位符，并且通常很有用。

是否允许空值也会影响默认构造函数。如果智能指针不允许空值，那么默认构造函数将如何初始化原始指针？可能缺少默认构造函数，但这会使智能指针更难处理。例如，当您有一个 `SmartPtr` 成员变量但在构造时没有合适的初始值设定项时，您应该怎么做？总之，自定义初始化涉及提供适当的默认值。

7.10.2 Checking Before Dereference

Checking before dereference is important because dereferencing the null pointer engenders undefined behavior. For many applications, undefined behavior is not acceptable, so checking the pointer for validity before dereference is the way to go. Checks before dereference belong to `SmartPtr`'s operator-`>` and unary operator-`*`.

In contrast to the initialization check, the check before dereference can become a major performance bottleneck in your application, because typical applications use (dereference) smart pointers much more often than they create smart pointer objects. Therefore, you should keep a balance between safety and speed. A good rule of thumb is to start with rigorously checked pointers and remove checks from selected smart pointers as profiling demonstrates a need for it.

Can initialization checking and checking before dereference be conceptually separated? No, because there are links between them. If you enforce strict checking upon initialization, then checking before dereference becomes redundant because the pointer is always valid.

This little efficiency victory brought by encapsulation is, however, amended by the loophole provided by `GetImplRef` which allows arbitrary replacement of the underlying raw pointer.

7.10.3 Error Reporting

The only sensible choice for reporting an error is to throw an exception.

You can do something in the sense of avoiding errors. For example, if a pointer is null upon dereference, you can initialize it on the fly. This is a valid and valuable strategy called *lazy initialization*—you construct the value only when you first need it.

If you want to check things only during debugging, you can use the standard `assert` or similar, more sophisticated macros. The compiler ignores the tests in release mode, so, assuming you remove all null pointer errors during debugging, you reap the advantages of both checking and speed.

`SmartPtr` migrates checking to a dedicated **Checking** policy. This policy implements checking functions (which can optionally provide lazy initialization) and the error reporting strategy.

7.11 Smart Pointers to `const` and `const` Smart Pointers

Raw pointers allow two kinds of constness: the constness of the pointed-to object and that of the pointer itself. The following is an illustration of these two attributes:

```
const Something* pc = new Something; // points to const object
pc->ConstMemberFunction(); // ok
pc->NonConstMemberFunction(); // error
delete pc; // ok (surprisingly)4
Something* const cp = new Something; // const pointer
cp->NonConstMemberFunction(); // ok
cp = new Something; // error, can't assign to const pointer
const Something* const cpc = new Something; // const, points to const
cpc->ConstMemberFunction(); // ok
```

⁴Every once in a while, the question “Why can you apply the `delete` operator to pointers to `const`?” starts a fierce debate in the `comp.std.c++` newsgroup. The fact is, for better or worse, the language allows it.

7.10.2 解引用前检查

在取消引用之前进行检查很重要，因为取消引用空指针会导致未定义的行为。对于许多应用程序来说，未定义的行为是不可接受的，因此在取消引用之前检查指针的有效性是正确的方法。取消引用之前的检查属于 `SmartPtr` 的运算符 `->` 和一元运算符 `*`。

与初始化检查相比，取消引用之前的检查可能成为应用程序中的主要性能瓶颈，因为典型应用程序使用（取消引用）智能指针的频率比创建智能指针对象的频率高得多。因此，您应该在安全和速度之间保持平衡。一个好的经验法则是从严格检查的指针开始，并在分析表明需要时从选定的智能指针中删除检查。

初始化检查和取消引用之前的检查可以在概念上分开吗？不，因为它们之间存在联系。如果在初始化时强制执行严格检查，则在取消引用之前进行检查将变得多余，因为指针始终有效。

然而，封装带来的这一小小的效率胜利被 `GetImplRef` 提供的漏洞所修正，该漏洞允许任意替换底层原始指针。

7.10.3 错误报告

报告错误的唯一明智选择是抛出异常。

你可以做一些避免错误的事情。例如，如果指针在取消引用时空，则可以即时初始化它。这是一个有效且有价值的策略，称为 *lazy initialization* - 仅在第一次需要时才构建该值。

如果您只想在调试期间检查，可以使用标准断言或类似的、更复杂的宏。编译器会忽略发布模式下的测试，因此，假设您在调试期间删除了所有空指针错误，您将获得检查和速度的优势。

`SmartPtr` 将检查迁移到专用检查策略。该策略实现了检查函数（可以选择提供延迟初始化）和错误报告策略。

7.11 指向 `const` 的智能指针和 `const` 智能指针

原始指针允许两种常量：所指向对象的常量和指针本身的常量。下面是这两个属性的说明：

```
const Something* pc = new Something; // 指向 const 对象 pc->ConstMemberFunction(); // 好的 pc-
>NonConstMemberFunction(); // 删除电脑时出错; // 好吧（令人惊讶）4 Something* const cp = n
ew Something; // const 指针 cp->NonConstMemberFunction(); // 好的 cp = new Something; // 错误，
无法分配给 const 指针 const Something* const cpc = new Something; // const, 指向 const cpc->Con
stMemberFunction(); // 好的
```

⁴ 每隔一段时间，就会出现这样的问题：“为什么可以对指向 `const` 的指针应用删除运算符？”在 `comp.std.c++` 新闻组中引发了激烈的争论。事实是，无论好坏，语言都允许这样做。

```
cpc->NonConstMemberFunction(); // error
cpc = new Something; // error, can't assign to const pointer
```

The corresponding uses of `SmartPtr` look like this:

```
// Smart pointer to const object
SmartPtr<const Something> spc(new Something);
// const smart pointer
const SmartPtr<Something> scp(new Something);
// const smart pointer to const object
const SmartPtr<const Something> scpc(new Something);
```

The `SmartPtr` class template can detect the constness of the pointed-to object either through partial specialization or by using the `TypeTraits` template defined in Chapter 2. The latter method is preferable because it does not incur source-code duplication as partial specialization does.

`SmartPtr` imitates the semantics of pointers to const objects, const pointers, and the combinations thereof.

7.12 Arrays

In most cases, instead of dealing with heap-allocated arrays and using `new[]` and `delete[]`, you're better off with `std::vector`. The standard-provided `std::vector` class template provides everything that dynamically allocated arrays provide, plus much more. The extra overhead incurred is negligible in most cases.

However, “most cases” is not “always.” There are many situations in which you don't need and don't want a full-fledged vector; a dynamically allocated array is exactly what you need. It is awkward in these cases to be unable to exploit smart pointer capabilities. There is a certain gap between the sophisticated `std::vector` and dynamically allocated arrays. Smart pointers could close that gap by providing array semantics if the user needs them.

From the viewpoint of a smart pointer to an array, the only important issue is to call `delete[] pointee_` in its destructor instead of `delete pointee_`. This issue is already tackled by the `Ownership` policy.

A secondary issue is providing indexed access, by overloading `operator[]` for smart pointers. This is technically feasible; in fact, a preliminary version of `SmartPtr` did provide a separate policy for optional array semantics. However, only in very rare cases do smart pointers point to arrays. In those cases, there already is a way of providing indexed accessing if you use `GetImpl`:

```
SmartPtr<Widget> sp = ...;
// Access the sixth element pointed to by sp
Widget& obj = GetImpl(sp)[5];
```

It seems like a bad decision to strive to provide extra syntactic convenience at the expense of introducing a new policy.

`SmartPtr` supports customized destruction via the `Ownership` policy. You can therefore

```
cpc->NonConstMemberFunction(); // 错误 cpc = new Something; // 错误, 无法分配给
const 指针
```

SmartPtr的对应用法如下:

```
// 指向 const 对象的智能指针 SmartPtr<const Something> spc(new Something);
// const 智能指针 const SmartPtr<Something> scp(new Something); //
指向 const 对象的 const 智能指针 const SmartPtr<const Something> scpc(new
Something);
```

SmartPtr 类模板可以通过部分特化或使用第 2 章中定义的 TypeTraits 模板来检测所指向对象的常量性。后一种方法更可取, 因为它不会像部分特化那样导致源代码重复。

SmartPtr 模仿指向 const 对象的指针、const 指针及其组合的语义。

7.12 数组

在大多数情况下, 您最好使用 `std::vector`, 而不是处理堆分配的数组并使用 `new[]` 和 `delete[]`。标准提供的 `std::vector` 类模板提供了动态分配数组提供的所有内容, 甚至更多。在大多数情况下, 产生的额外开销可以忽略不计。

然而, “大多数情况”并不“总是”。在很多情况下, 您不需要也不想要成熟的向量; 动态分配的数组正是您所需要的。在这些情况下, 无法利用智能指针功能是很尴尬的。复杂的 `std::vector` 和动态分配的数组之间存在一定的差距。如果用户需要的话, 智能指针可以通过提供数组语义来缩小这一差距。

从指向数组的智能指针的角度来看, 唯一重要的问题是在其析构函数中调用 `delete[] pointee_` 而不是 `delete ppointee_`。所有权政策已经解决了这个问题。

第二个问题是通过为智能指针重载 `operator[]` 来提供索引访问。这在技术上是可行的; 事实上, SmartPtr 的初步版本确实为可选数组语义提供了单独的策略。然而, 只有在极少数情况下, 智能指针才会指向数组。在这些情况下, 如果您使用 `GetImpl`, 则已经有一种提供索引访问的方法:

```
SmartPtr<Widget> sp = ...; // 访问sp指向的第六个元素 Widget
& obj = GetImpl(sp)[5];
```

以引入新策略为代价来努力提供额外的语法便利似乎是一个错误的决定。

SmartPtr 通过所有权策略支持定制销毁。因此你可以

arrange array-specific destruction via `delete[]`. However, `SmartPtr` does not provide pointer arithmetic.

7.13 Smart Pointers and Multithreading

Most often, smart pointers help with sharing objects. Multithreading issues affect object sharing. Therefore, multithreading issues affect smart pointers.

The interaction between smart pointers and multithreading takes place at two levels. One is the pointee object level, and the other is the bookkeeping data level.

7.13.1 Multithreading at the Pointee Object Level

If multiple threads access the same object and if you access that object through a smart pointer, it can be desirable to lock the object during a function call made through operator->. This is possible by having the smart pointer return a proxy object instead of a raw pointer. The proxy object's constructor locks the pointee object, and its destructor unlocks it. The technique is illustrated in Stroustrup (2000). Some code that illustrates this approach is provided here.

First, let's consider a class `Widget` that has two locking primitives: `Lock` and `Unlock`. After a call to `Lock`, you can access the object safely. Any other threads calling `Lock` will block. When you call `Unlock`, you let other threads lock the object.

```
class Widget
{
    ...
    void Lock();
    void Unlock();
};
```

Next, we define a class template `LockingProxy`. Its role is to lock an object (using the `Lock/Unlock` convention) for the duration of `LockingProxy`'s lifetime.

```
template <class T>
class LockingProxy
{
public:
    LockingProxy(T* pObj) : pointee_ (pObj)
    { pointee_->Lock(); }
    ~LockingProxy()
    { pointee_->Unlock(); }
    T* operator->() const
    { return pointee_; }
private:
    LockingProxy& operator=(const LockingProxy&);
    T* pointee_;
};
```

In addition to the constructor and destructor, `LockingProxy` defines an operator-> that returns a pointer to the pointee object.

通过`delete[]`安排特定于数组的销毁。然而，`SmartPtr`不提供指针运算。

7.13 智能指针和多线程

大多数情况下，智能指针有助于共享对象。多线程问题影响对象共享。因此，多线程问题会影响智能指针。

智能指针和多线程之间的交互发生在两个层面。一是受点对象级别，二是记账数据级别。

7.13.1 Pointee对象级别的多线程

如果多个线程访问同一个对象，并且通过智能指针访问该对象，则可能需要在通过`operator->`进行的函数调用期间锁定该对象。这可以通过让智能指针返回代理对象而不是原始指针来实现。代理对象的构造函数锁定被指向对象，其析构函数将其解锁。Stroustrup (2000) 对此技术进行了说明。此处提供了一些说明此方法的代码。

首先，让我们考虑一个具有两个锁定原语的 `Widget` 类：`Lock` 和 `Unlock`。调用 `Lock` 后，您可以安全地访问该对象。任何其他调用 `Lock` 的线程都将阻塞。当您调用 `Unlock` 时，您让其他线程锁定该对象。

```
类 Widget { ... void Lock
();无效解锁 () ; };
```

接下来，我们定义一个类模板 `LockingProxy`。它的作用是在 `LockingProxy` 的生命周期内锁定对象（使用锁定/解锁约定）。

```
模板 <class T> class LockingProxy {public: LockingProxy(T* pObj) :
pointee_(pObj) { pointee_->Lock(); } ~LockingProxy() { pointee_->U
nlock(); } T* 运算符->() const { return pointee_; } private: LockingPr
oxy& 运算符=(const LockingProxy&); T* 指针_; };
```

除了构造函数和析构函数之外，`LockingProxy` 还定义了一个运算符`->`，它返回指向被指针对象的指针。

Although `LockingProxy` looks somewhat like a smart pointer, there is one more layer to it—the `SmartPtr` class template itself.

```
template <class T>
class SmartPtr
{
    ...
    LockingProxy<T> operator->() const
    { return LockingProxy<T>(pointee_); }
private:
    T* pointee_;
};
```

Recall from Section 7.3, which explains the mechanics of `operator->`, that the compiler can apply `operator->` multiple times to one `->` expression, until it reaches a native pointer. Now imagine you issue the following call (assuming `Widget` defines a function `DoSomething`):

```
SmartPtr<Widget> sp = ...;
sp->DoSomething();
```

Here's the trick: `SmartPtr`'s `operator->` returns a temporary `LockingProxy<T>` object. The compiler keeps applying `operator->`. `LockingProxy<T>`'s `operator->` returns a `Widget*`. The compiler uses this pointer to `Widget` to issue the call to `DoSomething`. During the call, the temporary object `LockingProxy<T>` is alive and locks the object, which means that the object is safely locked. As soon as the call to `DoSomething` returns, the temporary `LockingProxy<T>` object is destroyed, so the `Widget` object is unlocked.

Automatic locking is a good application of smart pointer layering. You can layer smart pointers this way by changing the `Storage` policy.

7.13.2 Multithreading at the Bookkeeping Data Level

Sometimes smart pointers manipulate data in addition to the pointee object. As you read in Section 7.5, reference-counted smart pointers share some data—namely the reference count—under the covers. If you copy a reference-counted smart pointer from one thread to another, you end up having two smart pointers pointing to the same reference counter. Of course, they also point to the same pointee object, but that's accessible to the user, who can lock it. In contrast, the reference count is not accessible to the user, so managing it is entirely the responsibility of the smart pointer.

Not only reference-counted pointers are exposed to multithreading-related dangers. Reference-linked smart pointers (Section 7.5.4) internally hold pointers to each other, which are shared data as well. Reference linking leads to communities of smart pointers, not all of which necessarily belong to the same thread. Therefore, every time you copy, assign, and destroy a reference-linked smart pointer, you must issue appropriate locking; otherwise, the doubly linked list might get corrupted.

尽管 `LockingProxy` 看起来有点像智能指针，但它还有一层——`SmartPtr` 类模板本身。

```
模板 <class T> class SmartPtr { ... LockingProxy<T> 运算符->()  
const { return LockingProxy<T>(pointee_); } 私  
有: T* pointee_; };
```

回想一下 7.3 节，它解释了 `operator->` 的机制，编译器可以多次将 `operator->` 应用于一个 `->` 表达式，直到它到达本机指针。现在假设您发出以下调用（假设 `Widget` 定义了函数 `DoSomething()`）：

```
SmartPtr<Widget> sp = ...; sp->DoSomething();
```

技巧如下：`SmartPtr` 的运算符 `->` 返回一个临时 `LockingProxy<T>` 对象。编译器不断应用运算符 `->`。`LockingProxy<T>` 的运算符 `->` 返回一个 `Widget*`。编译器使用这个指向 `Widget` 的指针来发出对 `DoSomething` 的调用。调用过程中，临时对象 `LockingProxy<T>` 处于活动状态，并且锁定了该对象，这意味着该对象被安全锁定。一旦对 `DoSomething` 的调用返回，临时 `LockingProxy<T>` 对象就会被销毁，因此 `Widget` 对象会被解锁。

自动锁定是智能指针分层的一个很好的应用。您可以通过更改存储策略来以这种方式分层智能指针。

7.13.2 簿记数据级别的多线程

有时，智能指针除了操作被指对象之外还操作数据。正如您在第 7.5 节中所读到的，引用计数智能指针在幕后共享一些数据，即引用计数。如果将引用计数智能指针从一个线程复制到另一个线程，最终会得到两个指向同一个引用计数器的智能指针。当然，它们也指向同一个被指对象，但用户可以访问该对象，并且可以锁定它。相反，用户无法访问引用计数，因此管理它完全是智能指针的责任。

不仅引用计数指针面临与多线程相关的危险。引用链接智能指针（第 7.5.4 节）内部保存彼此的指针，它们也是共享数据。引用链接导致智能指针社区，并非所有智能指针都必须属于同一线程。因此，每次复制、分配和销毁引用链接的智能指针时，都必须发出适当的锁定；否则，双向链表可能会损坏。

In conclusion, multithreading issues ultimately affect smart pointers' implementation. Let's see how to address the multithreading issue in reference counting and reference linking.

7.13.2.1 Multithreaded Reference Counting

If you copy a smart pointer between threads, you end up incrementing the reference count from different threads at unpredictable times.

As the appendix explains, incrementing a value is not an atomic operation. For incrementing and decrementing integral values in a multithreaded environment, you must use the type `ThreadingModel<T>::IntType` and the `AtomicIncrement` and `AtomicDecrement` functions.

Here things become a bit tricky. Better said, they become tricky if you want to separate reference counting from threading.

Policy-based class design prescribes that you decompose a class into elementary behavioral elements and confine each of them to a separate template parameter. In an ideal world, `SmartPtr` would specify an `Ownership` policy and a `ThreadingModel` policy and would use them both for a correct implementation.

In the case of multithreaded reference counting, however, things are much too tied together. For example, the counter must be of type `ThreadingModel<T>::IntType`. Then, instead of using `operator++` and `operator--`, you must use `AtomicIncrement` and `AtomicDecrement`. Threading and reference counting melt together; it is unjustifiably hard to separate them.

The best thing to do is to incorporate multithreading in the `Ownership` policy. Then you can have two implementations: `RefCounting` and `RefCountingMT`.

7.13.2.2 Multithreaded Reference Linking

Consider the destructor of a reference-linked smart pointer. It likely looks like this:

```
template <class T>
class SmartPtr
{
public:
    ~SmartPtr()
    {
        if (prev_ == this)
        {
            delete pointee_;
        }
        else
        {
            prev_>next_ = next_;
            next_>prev_ = prev_;
        }
    }
    ...
private:
    T* pointee_;
    SmartPtr* prev_;
```

总之，多线程问题最终会影响智能指针的实现。让我们看看如何解决引用计数和引用链接中的多线程问题。

7.13.2.1 多线程引用计数

如果在线程之间复制智能指针，最终会在不可预测的时间增加来自不同线程的引用计数。

正如附录所解释的，增加值不是原子操作。要在多线程环境中递增和递减整数值，必须使用类型 `ThreadingModel<T>::IntType` 以及 `AtomicIncrement` 和 `AtomicDecrement` 函数。

这里事情变得有点棘手。更好地说，如果您想将引用计数与线程分开，它们会变得很棘手。

基于策略的类设计规定将类分解为基本行为元素，并将每个元素限制在单独的模板参数中。在理想的情况下，`SmartPtr` 将指定所有权策略和 `ThreadingModel` 策略，并使用它们来正确实现。

然而，在多线程引用计数的情况下，事情过于紧密地联系在一起。例如，计数器的类型必须为 `ThreadingModel<T>::IntType`。然后，您必须使用 `AtomicIncrement` 和 `AtomicDecrement`，而不是使用 `operator++` 和 `operator--`。线程和引用计数融为一体；将它们分开是极其困难的。

最好的办法是将多线程纳入所有权策略中。然后您可以有两个实现：`RefCounting` 和 `RefCountingMT`。

7.13.2.2 多线程参考链接

考虑引用链接智能指针的析构函数。它可能看起来像这样：

```
模板 <class T> class SmartPtr {public: ~SmartPtr() { if (prev_ == this) { 删除 pointee_; } }其他{上一个_->下一个_ = 下一个_; 下一个_->上一个_ = 上一个_; }... 私有: T * pointee_; SmartPtr* prev_;
```

```
    SmartPtr* next_;  
};
```

The code in the destructor performs a classic doubly linked list deletion. To make implementation simpler and faster, the list is circular—the last node points to the first node. This way we don't have to test `prev_` and `next_` against zero for any smart pointer. A circular list with only one element has `prev_` and `next_` equal to `this`.

If multiple threads destroy smart pointers that are linked to each other, clearly the destructor must be atomic (uninterruptible by other threads). Otherwise, another thread can interrupt the destructor of a `SmartPtr`, for instance, between updating `prev_->next_` and updating `next_->prev_`. That thread will then operate on a corrupt list.

Similar reasoning applies to `SmartPtr`'s copy constructor and the assignment operator. These functions must be atomic because they manipulate the ownership list.

Interestingly enough, we cannot apply object-level locking semantics here. The appendix divides locking strategies into *class-level* and *object-level* strategies. A class-level locking operation locks all objects in a given class during that operation. An object-level locking operation locks only the object that's subject to that operation. The former technique leads to less memory being occupied (only one mutex per class) but is exposed to performance bottlenecks. The latter is heavier (one mutex per object) but might be speedier.

We cannot apply object-level locking to smart pointers because an operation manipulates up to three objects: the current object that's being added or removed, the previous object, and the next object in the ownership list.

If we want to introduce object-level locking, the starting observation is that there must be one mutex per pointee object—because there's one list per pointee object. We can dynamically allocate a mutex for each object, but this nullifies the main advantage of reference linking over reference counting. Reference linking was more appealing exactly because it didn't use the free store.

Alternatively, we can use an intrusive approach: The pointee object holds the mutex, and the smart pointer manipulates that mutex. But the existence of a sound, effective alternative—reference-counted smart pointers—removes the incentive to provide this feature.

In summary, smart pointers that use reference counting or reference linking are affected by multithreading issues. Thread-safe reference counting needs integer atomic operations. Thread-safe reference linking needs mutexes. `SmartPtr` provides only thread-safe reference counting.

7.14 Putting It All Together

Not much to go! Here comes the fun part. So far we have treated each issue in isolation. It's now time to collect all the decisions into a unique `SmartPtr` implementation.

The strategy we'll use is the one described in Chapter 1: policy-based class design. Each design aspect that doesn't have a unique solution migrates to a policy. The `SmartPtr` class template accepts each policy as a separate template parameter. `SmartPtr` inherits all these template parameters, allowing the corresponding policies to store state.

```
SmartPtr* next_; };
```

析构函数中的代码执行经典的双向链表删除。为了使实现更简单、更快，列表是循环的——最后一个节点指向第一个节点。这样我们就不必针对任何智能指针来测试 `prev_` 和 `next_` 是否为零。只有一个元素的循环列表的 `prev_` 和 `next_` 等于此。

如果多个线程销毁相互链接的智能指针，显然析构函数必须是原子的（不能被其他线程中断）。否则，另一个线程可以中断 `SmartPtr` 的析构函数，例如在更新 `prev_->next_` 和更新 `next_->prev_` 之间。然后该线程将在损坏的列表上运行。

类似的推理也适用于 `SmartPtr` 的复制构造函数和赋值运算符。这些函数必须是原子的，因为它们操纵所有权列表。

有趣的是，我们不能在这里应用对象级锁定语义。附录将锁定策略分为 *class-level* 和 *object-level* 策略。类级锁定操作会在该操作期间锁定给定类中的所有对象。对象级锁定操作仅锁定受该操作影响的对象。前一种技术占用的内存较少（每个类只有一个互斥锁），但会遇到性能瓶颈。后者更重（每个对象一个互斥锁），但速度可能更快。

我们不能对智能指针应用对象级锁定，因为一个操作最多可以操作三个对象：当前正在添加或删除的对象、上一个对象以及所有权列表中的下一个对象。

如果我们想引入对象级锁定，首先观察到每个被指点对象必须有一个互斥锁，因为每个被指点对象都有一个列表。我们可以为每个对象动态分配一个互斥体，但这抵消了引用链接相对于引用计数的主要优势。参考链接更具吸引力，正是因为它不使用免费商店。

或者，我们可以使用侵入式方法：被指针对象保存互斥体，而智能指针操纵该互斥体。但是，一个健全、有效的替代方案（引用计数智能指针）的存在消除了提供此功能的动力。

总之，使用引用计数或引用链接的智能指针会受到多线程问题的影响。线程安全引用计数需要整数原子操作。线程安全的引用链接需要互斥体。`SmartPtr` 仅提供线程安全的引用计数。

7.14 将它们放在一起

没什么可走的！有趣的部分来了。到目前为止，我们已经单独处理了每个问题。现在是时候将所有决策收集到一个独特的 `SmartPtr` 实现中了。

我们将使用的策略是第 1 章中描述的策略：基于策略的类设计。每个没有独特解决方案的设计方面都会迁移到策略中。`SmartPtr` 类模板接受每个策略作为单独的模板参数。`SmartPtr` 继承了所有这些模板参数，允许相应的策略存储状态。

Let's recap the previous sections by enumerating the variation points of `SmartPtr`. Each variation point translates into a policy.

- *Storage policy* (Section 7.3). By default, the stored type is `T*` (`T` is the first template parameter of `SmartPtr`), the pointer type is again `T*`, and the reference type is `T&`. The means of destroying the pointee object is the `delete` operator.
- *Ownership policy* (Section 7.5). Popular implementations are deep copy, reference counting, reference linking, and destructive copy. Note that `Ownership` is not concerned with the mechanics of destruction itself; this is `Storage`'s task. `Ownership` controls the *moment* of destruction.
- *Conversion policy* (Section 7.7). Some applications need automatic conversion to the underlying raw pointer type; others do not.
- *Checking policy* (Section 7.10). This policy controls whether an initializer for `SmartPtr` is valid and whether a `SmartPtr` is valid for dereferencing.

Other issues are not worth dedicating separate policies to them or have an optimal solution:

- The address-of operator (Section 7.6) is best not overloaded.
- Equality and inequality tests are handled with the tricks shown in Section 7.8.
- Ordering comparisons (Section 7.9) are left unimplemented; however, `Loki` specializes `std::less` for `SmartPtr` objects. The user may define an `operator<`, and `Loki` helps by defining all other ordering comparisons in terms of `operator<`.
- `Loki` defines `const`-correct implementations for the `SmartPtr` object, the pointee object, or both.
- There is no special support for arrays, but one of the canned `Storage` implementations can dispose of arrays by using `operator delete[]`.

The presentation of the design issues surrounding smart pointers made these issues easier to understand and more manageable because each issue was discussed in isolation. It would be very helpful, then, if the implementation could decompose and treat issues in isolation instead of fighting with all the complexity at once.

Divide et Impera—this old principle coined by Julius Caesar can be of help even today with smart pointers. (I'd bet money he didn't predict that.) We break the problem into small component classes, called *policies*. Each policy class deals with exactly one issue. `SmartPtr` inherits all these classes, thus inheriting all their features. It's that simple—yet incredibly flexible, as you will soon see. Each policy is also a template parameter, which means you can mix and match existing stock policy classes or build your own.

The pointed-to type comes first, followed by each of the policies. Here is the resulting declaration of `SmartPtr`:

```
template
<
    typename T,
    template <class> class OwnershipPolicy = RefCounted,
    class ConversionPolicy = DisallowConversion,
    template <class> class CheckingPolicy = AssertCheck,
    template <class> class StoragePolicy = DefaultSPStorage
>
class SmartPtr;
```

让我们通过列举 `SmartPtr` 的变化点来回顾一下前面的部分。每个变化点都会转化为一项策略。

- 存储 *policy* (Section 7.3). 默认情况下, 存储类型是 `T*` (`T` 是 `SmartPtr` 的第一个模板参数), 指针类型也是 `T*`, 引用类型是 `T&`。销毁被指对象的方法是删除操作符。
- 所有权 *policy* (Section 7.5). 流行的实现是深度复制、引用计数、引用链接和破坏性复制。请注意, 所有权与破坏本身的机制无关; 这是存储的任务。所有权控制着破坏的 *moment*。
- 转换 *policy* (Section 7.7). 某些应用程序需要自动转换为底层原始指针类型; 其他人则不然。
- 检查 *policy* (Section 7.10). 此策略控制 `SmartPtr` 的初始值设定项是否有效以及 `SmartPtr` 对于取消引用是否有效。

其他问题不值得专门针对它们或有最佳解决方案:

- 地址运算符 (第 7.6 节) 最好不要重载。
- 等式和不等式测试使用第 7.8 节中所示的技巧进行处理。
- 排序比较 (第 7.9 节) 未实现; 然而, `Loki` 专门针对 `SmartPtr` 对象使用 `std::less`。用户可以定义一个运算符 `<`, `Loki` 通过根据运算符 `<` 定义所有其他排序比较来提供帮助。
- `Loki` 为 `SmartPtr` 对象、`pointee` 对象或两者定义了常量正确的实现。
- 没有对数组的特殊支持, 但其中一种固定存储实现可以通过使用运算符 `delete[]` 来处理数组。

围绕智能指针的设计问题的呈现使这些问题更容易理解并且更易于管理, 因为每个问题都是单独讨论的。那么, 如果实现能够分解并孤立地处理问题, 而不是同时处理所有的复杂性, 那将非常有帮助。

Divide et Impera— 朱利叶斯·凯撒 (Julius Caesar) 提出的这一古老原则即使在今天对于智能指针也能有所帮助。(我敢打赌他没有预测到这一点。) 我们将问题分解为小的组件类, 称为 *policies*. 每个策略类只处理一个问题。 `SmartPtr` 继承了所有这些类, 从而继承了它们的所有功能。就这么简单, 但又非常灵活, 您很快就会看到。每个策略也是一个模板参数, 这意味着您可以混合和匹配现有的股票策略类或构建自己的策略类。

指向的类型首先出现, 然后是每个策略。这是 `SmartPtr` 的声明结果:

```
<
模板  类型名称 T, 模板 <class> 类 OwnershipPolicy = RefCounted, 类 Conversi
onPolicy = DisallowConversion, 模板 <class> 类 CheckingPolicy = AssertCheck,
模板 <类> 类 StoragePolicy = DefaultSPStorage > 类智能指针;
```

The order in which the policies appear in `SmartPtr`'s declaration puts the ones that you customize most often at the top.

The following four subsections discuss the requirements of the four policies we have defined. A rule for all policies is that they must have value semantics; that is, they must define a proper copy constructor and assignment operator.

7.14.1 The Storage Policy

The `Storage` policy abstracts the structure of the smart pointer. It provides type definitions and stores the actual `pointee_` object.

If `StorageImpl` is an implementation of the `Storage` policy and `storageImpl` is an object of type `StorageImpl<T>`, then the constructs in Table 7.1 apply.

Here is the default `Storage` policy implementation:

```
template <class T>
class DefaultSPStorage
{
protected:
    typedef T* StoredType;    //the type of the pointee_ object
    typedef T* PointerType;   //type returned by operator->
    typedef T& ReferenceType; //type returned by operator*
public:
    DefaultSPStorage() : pointee_(Default())
    {}
    DefaultSPStorage(const StoredType& p): pointee_(p) {}
    PointerType operator->() const { return pointee_; }
    ReferenceType operator*() const { return *pointee_; }
    friend inline PointerType GetImpl(const DefaultSPStorage& sp)
    { return sp.pointee_; }
    friend inline const StoredType& GetImplRef(
        const DefaultSPStorage& sp)
    { return sp.pointee_; }
    friend inline StoredType& GetImplRef(DefaultSPStorage& sp)
    { return sp.pointee_; }
protected:
    void Destroy()
    { delete pointee_; }
    static StoredType Default()
    { return 0; }
private:
    StoredType pointee_;
};
```

In addition to `DefaultSPStorage`, other sensible policies to define are:

- `ArrayStorage`, which uses `operator delete[]` inside `Destroy`
- `LockedStorage`, which uses layering to provide a smart pointer that locks data while dereferenced (see Section 7.13.1)
- `HeapStorage`, which uses an explicit destructor call followed by `std::free` to release the data

策略在 `SmartPtr` 声明中出现的顺序将您最常自定义的策略放在顶部。

以下四小节讨论了我们定义的四项政策的要求。所有策略的一条规则是它们必须具有值语义；也就是说，它们必须定义适当的复制构造函数和赋值运算符。

7.14.1 存储策略

存储策略抽象了智能指针的结构。它提供类型定义并存储实际的 `pointee_` 对象。

如果 `StorageImpl` 是 `Storage` 策略的实现，并且 `storageImpl` 是 `StorageImpl<T>` 类型的对象，则表 7.1 中的构造适用。

这是默认的存储策略实现：

```
模板 <class T> class DefaultSPStorage {protected: typedef T* StoredType; //pointee_对象的
类型 typedef T* PointerType; //operator-> 返回的类型 typedef T& ReferenceType; // 运算符
返回的类型* public: DefaultSPStorage() : pointee_(Default()) {} DefaultSPStorage(const Stor
edType& p): pointee_(p) {} PointerType operator->() const { return pointee_; } ReferenceTyp
e 运算符*() const { return *pointee_; } 朋友内联 PointerType GetImpl(const DefaultSPStora
ge& sp) { return sp.pointee_; } 朋友内联 const StoredType& GetImplRef( const DefaultSPStora
ge& sp) { return sp.pointee_; } 朋友内联 StoredType& GetImplRef(DefaultSPStorage& sp) { r
eturn sp.pointee_; } protected: void Destroy() { 删除 pointee_; } 静态 StoredType Default() { r
eturn 0; } 私有: StoredType pointee_; };
```

除了 `DefaultSPStorage` 之外，其他需要定义的合理策略还有：

- `ArrayStorage`，它在 `Destroy` 中使用运算符 `delete[]`
- `LockedStorage`，它使用分层来提供在取消引用时锁定数据的智能指针（请参阅第 7.13.1 节）
- `HeapStorage`，它使用显式析构函数调用，然后使用 `std::free` 来释放数据

Table 7.1: Storage Policy Constructs

<i>Expression</i>	<i>Semantics</i>
<code>StorageImpl<T>::StoredType</code>	The type actually stored by the implementation. Default: <code>T*</code> .
<code>StorageImpl<T>::PointerType</code>	The pointer type defined by the implementation. This is the type returned by <code>SmartPtr</code> 's operator <code>-></code> . Default: <code>T*</code> . Can be different from <code>StorageImpl<T>::StoredType</code> when you're using smart pointer layering (see Sections 7.3 and 7.13.1).
<code>StorageImpl<T>::ReferenceType</code>	The reference type. This is the type returned by <code>SmartPtr</code> 's operator <code>*</code> . Default: <code>T&</code> .
<code>GetImpl(storageImpl)</code>	Returns an object of type <code>StorageImpl<T>::StoredType</code> .
<code>GetImplRef(storageImpl)</code>	Returns an object of type <code>StorageImpl<T>::StoredType&</code> , qualified with <code>const</code> if <code>storageImpl</code> is <code>const</code> .
<code>storageImpl.operator->()</code>	Returns an object of type <code>StorageImpl<T>::PointerType</code> . Used by <code>SmartPtr</code> 's own operator <code>-></code> .
<code>storageImpl.operator*()</code>	Returns an object of type <code>StorageImpl<T>::ReferenceType</code> . Used by <code>SmartPtr</code> 's own operator <code>*</code> .
<code>StorageImpl<T>::StoredType p;</code> <code>p = storageImpl.Default();</code>	Returns the default value (usually zero).
<code>storageImpl.Destroy()</code>	Destroys the pointee object.

7.14.2 The Ownership Policy

The Ownership policy must support intrusive as well as nonintrusive reference counting. Therefore, it uses explicit function calls rather than constructor/destructor techniques, as Koenig (1996) does. The reason is that you can call member functions at any time, whereas constructors and destructors are called automatically and only at specific times.

The Ownership policy implementation takes one template parameter, which is the corresponding pointer type. `SmartPtr` passes `StoragePolicy<T>::PointerType` to `OwnershipPolicy`. Note that `OwnershipPolicy`'s template parameter is a pointer type, not an object type.

If `OwnershipImpl` is an implementation of `Ownership` and `ownershipImpl` is an object of type `OwnershipImpl<P>`, then the constructs in Table 7.2 apply.

表 7.1: 存储策略结构

<i>Expression</i>	<i>Semantics</i>
<code>StorageImpl<T>::StoredType</code>	The type actually stored by the implementation. Default: <code>T*</code> .
<code>StorageImpl<T>::PointerType</code>	The pointer type defined by the implementation. This is the type returned by <code>SmartPtr</code> 's <code>operator-></code> . Default: <code>T*</code> . Can be different from <code>StorageImpl<T>::StoredType</code> when you're using smart pointer layering (see Sections 7.3 and 7.13.1).
<code>StorageImpl<T>::ReferenceType</code>	The reference type. This is the type returned by <code>SmartPtr</code> 's <code>operator*</code> . Default: <code>T&</code> .
<code>GetImpl(storageImpl)</code>	Returns an object of type <code>StorageImpl<T>::StoredType</code> .
<code>GetImplRef(storageImpl)</code>	Returns an object of type <code>StorageImpl<T>::StoredType&</code> , qualified with <code>const</code> if <code>storageImpl</code> is <code>const</code> .
<code>storageImpl.operator->()</code>	Returns an object of type <code>StorageImpl<T>::PointerType</code> . Used by <code>SmartPtr</code> 's own <code>operator-></code> .
<code>storageImpl.operator*()</code>	Returns an object of type <code>StorageImpl<T>::ReferenceType</code> . Used by <code>SmartPtr</code> 's own <code>operator*</code> .
<code>StorageImpl<T>::StoredType p; p = storageImpl.Default();</code>	Returns the default value (usually zero).
<code>storageImpl.Destroy()</code>	Destroys the pointee object.

7.14.2 所有权政策

所有权策略必须支持侵入式和非侵入式引用计数。因此，它使用显式函数调用，而不是像 Koenig (1996) 那样的构造函数/析构函数技术。原因是您可以随时调用成员函数，而构造函数和析构函数只会在特定时间自动调用。

所有权策略实现采用一个模板参数，该参数是相应的指针类型。 `SmartPtr` 将 `StoragePolicy<T>::PointerType` 传递给 `OwnershipPolicy`。请注意，`OwnershipPolicy` 的模板参数是指针类型，而不是对象类型。

如果 `OwnershipImpl` 是 `Ownership` 的实现，并且 `OwnershipImpl` 是 `OwnershipImpl<P>` 类型的对象，则表 7.2 中的构造适用。

Table 7.2: Ownership Policy Constructs

<i>Expression</i>	<i>Semantics</i>
<code>P val1;</code> <code>P val2 = OwnershipImpl.</code> <code> Clone(val1);</code>	Clones an object. It can modify the source value if <code>OwnershipImpl</code> uses destructive copy.
<code>const P val1;</code> <code>P val2 = ownershipImpl.</code> <code> Clone(val1);</code>	Clones an object.
<code>P val;</code> <code>bool unique = ownershipImpl.</code> <code> Release(val);</code>	Releases ownership of an object. Returns true if the last reference to the object was released.
<code>bool dc = OwnershipImpl<P></code> <code> ::destructiveCopy;</code>	States whether <code>OwnershipImpl</code> uses destructive copy. If that's the case, <code>SmartPtr</code> uses the Colvin/Gibbons trick (Meyers 1999) used in <code>std::auto_ptr</code> .

An implementation of `Ownership` that supports reference counting is shown in the following:

```
template <class P>
class RefCounted
{
    unsigned int* pCount_;
protected:
    RefCounted() : pCount_(new unsigned int(1)) {}
    P Clone(const P & val)
    {
        ++*pCount_;
        return val;
    }
    bool Release(const P&)
    {
        if (!--*pCount_)
        {
            delete pCount_;
            return true;
        }
        return false;
    }
    enum { destructiveCopy = false }; // see below
};
```

表 7.2: 所有权政策结构

<i>Expression</i>	<i>Semantics</i>
<code>P val1;</code> <code>P val2 = OwnershipImpl.</code> <code>Clone(val1);</code>	Clones an object. It can modify the source value if <code>OwnershipImpl</code> uses destructive copy.
<code>const P val1;</code> <code>P val2 = ownershipImpl.</code> <code>Clone(val1);</code>	Clones an object.
<code>P val;</code> <code>bool unique = ownershipImpl.</code> <code>Release(val);</code>	Releases ownership of an object. Returns true if the last reference to the object was released.
<code>bool dc = OwnershipImpl<P></code> <code>::destructiveCopy;</code>	States whether <code>OwnershipImpl</code> uses destructive copy. If that's the case, <code>SmartPtr</code> uses the Colvin/Gibbons trick (Meyers 1999) used in <code>std::auto_ptr</code> .

支持引用计数的 Ownership 实现如下所示:

```
模板 <class P> class RefCounted { unsigned int* pCount_; protected: Re
fCounted(): pCount_(new unsigned int(1)) {} P Clone(const P & val) {
++*pCount_;返回值; }bool Release(const P&) { if (!--*pCount_) { 删
除 pCount_;返回真; }返回错误; }枚举{ structuralCopy = false };//
见下文};
```

Implementing a policy for other schemes of reference counting is very easy. Let's write an **Ownership** policy implementation for COM objects. COM objects have two functions: `AddRef` and `Release`. Upon the last `Release` call, the object destroys itself. You need only direct `Clone` to `AddRef` and `Release` to COM's `Release`:

```
template <class P>
class COMRefCounted
{
public:
    static P Clone(const P& val)
    {
        val->AddRef();
        return val;
    }
    static bool Release(const P& val)
    {
        val->Release();
        return false;
    }
    enum { destructiveCopy = false }; // see below
};
```

Loki defines the following **Ownership** implementations:

- `DeepCopy`, described in Section 7.5.1. `DeepCopy` assumes that pointee class implements a member function `Clone`.
- `RefCounted`, described in Section 7.5.3 and in this section.
- `RefCountedMT`, a multithreaded version of `RefCounted`.
- `COMRefCounted`, a variant of intrusive reference counting described in this section.
- `RefLinked`, described in Section 7.5.4.
- `DestructiveCopy`, described in Section 7.5.5.
- `NoCopy`, which does not define `Clone`, thus disabling any form of copying.

7.14.3 The Conversion Policy

Conversion is a simple policy: It defines a Boolean compile-time constant that says whether or not `SmartPtr` allows implicit conversion to the underlying pointer type.

If `ConversionImpl` is an implementation of **Conversion**, then the construct in Table 7.3 applies.

The underlying pointer type of `SmartPtr` is dictated by its **Storage** policy and is `StorageImpl<T>::StorageType`.

As you would expect, Loki defines precisely two **Conversion** implementations:

- `AllowConversion`
- `DisallowConversion`

为其他引用计数方案实施策略非常容易。让我们为 COM 对象编写一个所有权策略实现。COM 对象有两个函数：AddRef 和 Release。在最后一次 Release 调用时，对象会自行销毁。你只需要直接 Clone 到 AddRef 并 Release 到 COM 的 Release：

```
模板 <class P> class COMRefCounted {public: static P Clone(const P&
val) { val->AddRef();返回值; } static bool Release(const P& val) { val-
>Release();返回假; }枚举{ structuralCopy = false }; // 见下文 };
```

Loki 定义了以下所有权实现：

- DeepCopy，第 7.5.1 节中描述。DeepCopy 假设 pointee 类实现了一个成员函数 Clone。
- RefCounted，在第 7.5.3 节和本节中描述。
- RefCountedMT，RefCounted 的多线程版本。
- COMRefCounted，本节中描述的侵入式引用计数的一种变体。
- RefLinked，第 7.5.4 节中描述。
- DestructiveCopy，第 7.5.5 节中描述。
- NoCopy，它不定义克隆，因此禁用任何形式的复制。

7.14.3 转换策略

转换是一个简单的策略：它定义一个布尔编译时常量，表示 SmartPtr 是否允许隐式转换为基础指针类型。

如果 ConversionImpl 是 Conversion 的实现，则表 7.3 中的构造适用。

SmartPtr 的底层指针类型由其存储策略决定，为 StorageImpl<T>::StorageType。

正如您所期望的，Loki 精确定义了两种 Conversion 实现：

- 允许转换
- 禁止转换

Table 7.3: Conversion Policy Construct

Expression	Semantics
<code>bool allowConv = ConversionImpl<P>::allow;</code>	If <code>allow</code> is true, <code>SmartPtr</code> allows implicit conversion to its underlying pointer type.

Table 7.4: Checking Policy Constructs

Expression	Semantics
<code>S value; checkingImpl.OnDefault(value);</code>	<code>SmartPtr</code> calls <code>OnDefault</code> in the default constructor call. If <code>CheckingImpl</code> does not define this function, it disables the default constructor at compile time.
<code>S value; checkingImpl.OnInit(value);</code>	<code>SmartPtr</code> calls <code>OnInit</code> upon a constructor call.
<code>S value; checkingImpl.OnDereference (value);</code>	<code>SmartPtr</code> calls <code>OnDereference</code> before returning from <code>operator-></code> and <code>operator*</code> .
<code>const S value; checkingImpl.OnDereference (value);</code>	<code>SmartPtr</code> calls <code>OnDereference</code> before returning from the <code>const</code> versions of <code>operator-></code> and <code>operator*</code> .

7.14.4 The Checking Policy

As discussed in Section 7.10, there are two main places to check a `SmartPtr` object for consistency: during initialization and before dereference. The checks themselves might use `assert`, exceptions, or lazy initialization or not do anything at all.

The `Checking` policy operates on the `StoredType` of the `Storage` policy, not on the `PointerType`. (See Section 7.14.1 for the definition of `Storage`.)

If `S` is the stored type as defined by the `Storage` policy implementation, and if `CheckingImpl` is an implementation of `Checking`, and if `checkingImpl` is an object of type `CheckingImpl<S>`, then the constructs in Table 7.4 apply.

Loki defines the following implementations of `Checking`:

- `AssertCheck`, which uses `assert` for checking the value before dereferencing.
- `AssertCheckStrict`, which uses `assert` for checking the value upon initialization.
- `RejectNullStatic`, which does not define `OnDefault`. Consequently, any use of `SmartPtr`’s default constructor yields a compile-time error.

表 7.3: 转换策略结构

Expression	Semantics
<code>bool allowConv = ConversionImpl<P>::allow;</code>	If <code>allow</code> is true, <code>SmartPtr</code> allows implicit conversion to its underlying pointer type.

表 7.4: 检查策略结构

Expression	Semantics
<code>S value; checkingImpl.OnDefault(value);</code>	<code>SmartPtr</code> calls <code>OnDefault</code> in the default constructor call. If <code>CheckingImpl</code> does not define this function, it disables the default constructor at compile time.
<code>S value; checkingImpl.OnInit(value);</code>	<code>SmartPtr</code> calls <code>OnInit</code> upon a constructor call.
<code>S value; checkingImpl.OnDereference (value);</code>	<code>SmartPtr</code> calls <code>OnDereference</code> before returning from <code>operator-></code> and <code>operator*</code> .
<code>const S value; checkingImpl.OnDereference (value);</code>	<code>SmartPtr</code> calls <code>OnDereference</code> before returning from the <code>const</code> versions of <code>operator-></code> and <code>operator*</code> .

7.14.4 检查策略

正如第 7.10 节中所讨论的，有两个主要位置可以检查 `SmartPtr` 对象的一致性：初始化期间和取消引用之前。检查本身可能使用断言、异常或延迟初始化，或者根本不执行任何操作。

检查策略在存储策略的 `StoredType` 上运行，而不是在 `PointerType` 上运行。（有关存储的定义，请参见第 7.14.1 节。）

如果 `S` 是存储策略实现定义的存储类型，并且 `CheckingImpl` 是 `Checking` 的实现，并且 `checkingImpl` 是 `CheckingImpl<S>` 类型的对象，则表 7.4 中的构造适用。

Loki 定义了以下检查的实现：

- `AssertCheck`，它使用断言在取消引用之前检查值。
- `AssertCheckStrict`，它使用断言来检查初始化时的值。
- `RejectNullStatic`，未定义 `OnDefault`。因此，任何使用 `SmartPtr` 的默认构造函数都会产生编译时错误。

- `RejectNull`, which throws an exception if you try to dereference a null pointer.
- `RejectNullStrict`, which does not accept null pointers as initializers (again, by throwing an exception).
- `NoCheck`, which handles errors in the grand C and C++ tradition—that is, it does no checking at all.

7.15 Summary

Congratulations! You have just read one of the longest, wildest chapters of this book—an effort that we hope has paid off. Now you know a lot of things about smart pointers and are equipped with a pretty comprehensive and configurable `SmartPtr` class template.

Smart pointers imitate built-in pointers in syntax and semantics. In addition, they perform a host of tasks that built-in pointers cannot. These tasks might include ownership management and checking against invalid values.

Smart pointer concepts go beyond actual pointer behavior; they can be generalized into smart resources, such as monikers (handles that don't have pointer syntax, yet resemble pointer behavior in the way they enable resource access).

Because they nicely automate things that are very hard to manage by hand, smart pointers are an essential ingredient of successful, robust applications. As small as they are, they can make the difference between a successful project and a failure—or, more often, between a correct program and one that leaks resources like a sieve.

That's why a smart pointer implementer should invest as much attention and effort in this task as possible; the investment is likely to pay in the long term. Similarly, smart pointer users should understand the conventions that smart pointers establish and use them in accordance with those conventions.

The presented implementation of smart pointers focuses on decomposing the areas of functionality into independent policies that the main class template `SmartPtr` mixes and matches. This is possible because each policy implements a well-defined interface.

7.16 SmartPtr Quick Facts

- `SmartPtr` declaration:

```
template
<
    typename T,
    template <class> class OwnershipPolicy = RefCounted,
    class ConversionPolicy = DisallowConversion,
    template <class> class CheckingPolicy = AssertCheck,
    template <class> class StoragePolicy = DefaultSPStorage
>
class SmartPtr;
```

- `T` is the type to which `SmartPtr` points. `T` can be a primitive type or a user-defined type. The void type is allowed.
- For the remaining class template parameters (`OwnershipPolicy`, `ConversionPolicy`, `CheckingPolicy`, and `StoragePolicy`), you can implement your own policies or choose from the defaults mentioned in Sections 7.14.1 through 7.14.4.

- `RejectNull`，如果您尝试取消引用空指针，则会引发异常。
- `RejectNullStrict`，它不接受空指针作为初始化器（同样，通过抛出异常）。
- `NoCheck`，它按照 C 和 C++ 传统处理错误，也就是说，它根本不进行检查。

7.15 总结

恭喜！您刚刚阅读了本书中最长、最狂野的章节之一——我们希望这一努力得到回报。现在您已经了解了很多有关智能指针的知识，并且配备了相当全面且可配置的 `SmartPtr` 类模板。

智能指针在语法和语义上模仿内置指针。此外，它们还执行许多内置指针无法执行的任务。这些任务可能包括所有权管理和检查无效值。

智能指针概念超越了实际的指针行为；它们可以概括为智能资源，例如名字（没有指针语法的句柄，但在支持资源访问的方式上类似于指针行为）。

由于智能指针可以很好地实现手动管理很难的事情的自动化，因此它们是成功、强大的应用程序的重要组成部分。尽管它们很小，但它们可以决定项目的成功与失败，或者更常见的是，它们可以决定正确的程序与像筛子一样泄漏资源的程序。

这就是为什么智能指针实现者应该在这项任务上投入尽可能多的注意力和精力；从长远来看，这项投资可能会带来回报。同样，智能指针用户应该了解智能指针建立的约定并按照这些约定使用它们。

所提出的智能指针的实现侧重于将功能区域分解为主类模板 `SmartPtr` 混合和匹配的独立策略。这是可能的，因为每个策略都实现了一个定义良好的接口。

7.16 SmartPtr 要闻速览

- 智能指针声明：

```
template <
    类型名称 T, 模板 <class> 类 OwnershipPolicy = RefCounted, 类 ConversionPolicy = DisallowConversion,
    模板 <类> 类 CheckingPolicy = AssertCheck, 模板 <类> 类 StoragePolicy = DefaultSPStorage >
    类 SmartPtr;
```

- T 是 `SmartPtr` 指向的类型。T 可以是原始类型或用户定义类型。允许使用 `void` 类型。
- 对于其余的类模板参数（`OwnershipPolicy`、`ConversionPolicy`、`CheckingPolicy` 和 `StoragePolicy`），您可以实现自己的策略或从第 7.14.1 节到 7.14.4 节中提到的默认值中进行选择。

- `OwnershipPolicy` controls the ownership management strategy. You can select from the predefined classes `DeepCopy`, `RefCounted`, `RefCountedMT`, `COMRefCounted`, `RefLinked`, `DestructiveCopy`, and `NoCopy`, described in Section 7.14.2.
- `ConversionPolicy` controls whether implicit conversion to the pointee type is allowed. The default is to forbid implicit conversion. Either way, you can still access the pointee object by calling `GetImpl`. You can use the `AllowConversion` and `DisallowConversion` implementations (Section 7.14.3).
- `CheckingPolicy` defines the error checking strategy. The defaults provided are `AssertCheck`, `AssertCheckStrict`, `RejectNullStatic`, `RejectNull`, `RejectNullStrict`, and `NoCheck` (Section 7.14.4).
- `StoragePolicy` defines the details of how the pointee object is stored and accessed. The default is `DefaultSPStorage`, which, when instantiated with a type `T`, defines the reference type as `T&`, the stored type as `T*`, and the type returned from `operator->` as `T*` again. Other storage types defined by Loki are `ArrayStorage`, `LockedStorage`, and `HeapStorage` (Section 7.14.1).

- **OwnershipPolicy** 控制所有权管理策略。您可以从预定义的类 `DeepCopy`、`RefCounted`、`RefCountedMT`、`COMRefCounted`、`RefLinked`、`DestructiveCopy` 和 `NoCopy` 中进行选择，如第 7.14.2 节所述。
- **ConversionPolicy** 控制是否允许隐式转换为指针对象类型。默认是禁止隐式转换。无论哪种方式，您仍然可以通过调用 `GetImpl` 来访问被指对象。您可以使用 `AllowConversion` 和 `DisallowConversion` 实现（第 7.14.3 节）。
- **CheckingPolicy** 定义错误检查策略。提供的默认值是 `AssertCheck`、`AssertCheckStrict`、`RejectNullStatic`、`RejectNull`、`RejectNullStrict` 和 `NoCheck`（第 7.14.4 节）。
- **StoragePolicy** 定义了如何存储和访问被指对象的详细信息。默认值是 `DefaultSPStorage`，当用类型 `T` 实例化时，它将引用类型定义为 `T&`，存储类型为 `T*`，并且从 `operator->` 返回的类型再次定义为 `T*`。Loki 定义的其他存储类型有 `ArrayStorage`、`LockedStorage` 和 `HeapStorage`（第 7.14.1 节）。

This page intentionally left blank

This page intentionally left blank

Index

A

- Abstract Factory design pattern, 69, 40–51
 - architectural role of, 219–222
 - basic description of, 219–234
 - implementing, 226–233
 - interface, generic, 223–226
 - quick facts, 233–234
- AbstractEnemyFactory, 221–222, 224–227
- AbstractFactory, 219–234
- AbstractProduct, 209–214, 216–217
- abstract products, 209–214, 216–217, 222, 231
- Accept, 240–251, 254
- AcceptImpl, 252, 256, 259
- ACE, 309
- Acquire, 161–162, 305, 306
- Action, 100
- Adapter, 284
- Add, 278–282, 284, 300
- address-of, 170–171
- after, 16
- AbstactFactoryUnit, 223–226
- Alexandrescu, Andrei, 29
- algorithms
 - copy, 40
 - compile-time, 76
 - linear search, 56
 - operating on typelists, 76
- Allocate, 83, 85
- Allocated, 82
- allocation, small-object. *See also* allocators
 - basic description of, 77–96
 - default free store allocators and, 78
 - fixed-size allocators and, 84–87
 - hat trick and, 89–91
 - memory chunks and, 81–84
 - quick facts, 94–95
- allocators. *See also* allocation, small-object
 - default free store, 78
 - fixed-size, 84–87
 - memory chunks and, 81–84
 - workings of, 78–79
- allocChunk_, 85, 87
- AllowConversion, 192–193
- ALU (arithmetic logic unit), 303
- AnotherDerived, 197
- APIs (application program interfaces), 161, 306
- Append, 57–58, 76
- Application, 100
- arguments, 114–115, 285–290
- Array policy, 19–20
- arrays, 19–20, 183–184
- ArrayStorage, 189–190
- assert, 193
- AssertCheck, 193
- AssertCheckStrict, 193
- assertions, compile-time, 23–26
- associative collections, 203
- AssocVector, 210, 277–278
- asynchronous execution, 302
- atexit, 134–139, 142–143, 149
- ATEXIT_FIXED, 139
- AtExitFn, 144–145
- AtomicAdd, 305
- AtomicAssign, 305
- AtomicDecrement, 186
- AtomicIncrement, 186
- auto_ptr, 108, 159, 170
- available_, 79

指数

一个

抽象工厂设计模式, 69、40–51 的架构角色、219–222 的基本描述、219–234 实现、226–233 接口、通用、223–226 快速事实、233–234 AbstractEnemyFactory、221–222、224–227 AbstractFactory、219–234 AbstractProduct、209–214、216–217 抽象产品、209–214、216–217、222、231 接受、240–251、254 AcceptImpl、252、256、259 ACE、309 获取、161–162、305、306 操作、100 适配器、284 Add、278–282、284、300 地址、170–171 之后、16 AbstractFactoryUnit、223–226 Alexandrescu、Andrei、29 算法复制、40 编译时、76 线性搜索、56 类型列表操作、76 分配、83、85 分配、82 分配、小物体。See also 分配器的基本描述, 77–96 默认自由存储分配器和, 78 固定大小分配器和, 84–87 帽子戏法和, 89–91

内存块以及 81–84 个速览、94–95 个分配器。See also 分配、小对象默认自由存储、78 个固定大小、84–87 个内存块和 81–84 个工作原理、78–79 allocChunk_、85、87 Allow Conversion、192–193 ALU (算术逻辑单元)、303 AnotherDerived、197 API (应用程序接口)、161、306 Append、57–58、76 应用程序、100 个参数、114–115、285–290 数组策略、19–20 个数组、19–20、183–184 ArrayStorage、189–190 断言、193 AssertCheck、193 AssertCheckStrict、193 个断言、编译时、23–26 关联集合、203 AssocVector、210、277–278 异步执行、302 atexit、134–139、142–143、149 ATEXIT_FIXED、139 AtExitFn、144–145 AtomicAdd、305 AtomicAssign、305 原子减量、186 原子增量、186 auto_ptr、108、159、170 available_、79

B

backEnd_, 281, 294
 BackendType, 294
 BadMonster, 219–222
 BadSoldier, 219–222, 230
 BadSuperMonster, 219–221
 BankAccount, 306
 Bar, 139
 Base, 63, 90, 91, 176
 BASE_IF, 37
 BaseLhs, 272, 299
 BaseProductList, 227
 BaseRhs, 272, 284, 299
 BaseSmartPtr, 46
 BaseVisitable, 249, 251, 254
 BaseVisitor, 245, 249–250, 252, 254, 256, 261
 BaseVisitorImpl, 260, 262
 BasicDispatcher, 277–284, 292–294, 297–299
 BasicFastDispatcher, 291–294, 297, 298–299
 before, 16
 BinderFirst, 121
 BindFirst, 121, 127–128
 binding, 119–121, 127–128
 BitBlast, 40–41, 44–46
 blocksAvailable_, 81–83
 blockSize_, 82
 bookkeeping data level, 185–187
 bool, 105, 174–175, 177, 191
 _buffer, 135
 bulk allocation, 86
 butterfly allocation, 86
 Button, 50, 61–62, 219–221

C

callable entities, 103–104
 callbackMap_, 280
 callbacks, 103–104, 205, 280–281
 callbacks_, 205, 281
 CallbackType, 277, 280, 299
 CastingPolicy policy, 288, 299
 CatchAll policy, 260, 262
 Chain, 122, 128
 chaining requests, 122
 char, 35, 38, 114–115
 checking issues, 181–182
 checkingImpl, 193
 Checking policy, 15–16, 188, 193–194
 chunks, of memory, 81–87
 chunkSize, 88

Circle, 203
 class(es). *See also* inheritance; policy classes
 base, 228
 client, 153–154
 decomposing, 19–20
 derived, 228
 final, 29
 generating, with typelists, 64–75
 -level locking operations, 187
 local, 28–29
 object factories and, 200–201
 visitable, 248–255
 visitor, 248–255
 Class, 200
 ClassLevelLockable, 153, 307–309
 Clone, 30, 107, 123, 164, 211–213, 230, 232, 234, 284
 CloneFactory, 214–218
 clone object factories, 211–215
 CLOS, 263
 columns_, 292
 COM (Component Object Model), 133, 192
 command(s). *See also* Command design
 pattern
 active, 102
 forwarding, 102
 Command, 100
 Command design pattern, 99–104
 basic description of, 100–102
 in the real world, 102–103
 comparison operators, 178–181
 compile-time
 assertions, 23–26
 detecting convertibility and inheritance at, 34–37
 CompileTimeChecker, 25–26
 CompileTimeError, 25
 COMRefCounted, 192
 ConcreteCommand, 100, 102
 ConcreteFact, 232, 234
 ConcreteFactory, 226–228, 230–231, 233–234
 ConcreteLifetimeTracker, 144–145
 concrete products, 222, 227
 const, 44, 114–115, 182–183
 constant(s)
 mapping integral, to types, 29–31
 -time multimethods, 290–293
 ConventionalDialog, 219–221
 conversion
 argument, 114–115, 285–290
 binding as, 119–121
 implicit, to raw pointer types, 171–173
 return type, 114–114
 user-defined, 172

乙

backEnd_、281、294 BackendType、294 Bad Monster、219–222 BadSoldier、219–222、230 BadSuperMonster、219–221 BankAccount、306 Bar、139 Base、63、90、91、176 BASE_IF、37 BaseLhs、272、299 BaseProductList、227 BaseRhs、272、284、299 BaseSmartPtr、46 BaseVisitable、249、251、254 BaseVisitor、245、249–250、252、254、256、261 BaseVisitorImpl、260、262 BasicDispatcher、277–284、292–294、297–299 BasicFastDispatcher、291–294、297、298–299 之前、16 Binder First、121 BindFirst、121、127–128 绑定、119–121、127–128 BitBlast、40–41、44–46 blockAvailable_、81–83 blockSize_、82 簿记数据级别、185–187 bool、105、174–175、177、191_buffer、135 批量分配、86 蝴蝶分配、86 Button、50、61–62、219–221

C

可调用实体、103–104 callbackMap_、280 个回调、103–104、205、280–281 calls_、205、281 CallbackType、277、280、299 CastingPolicy 策略、288、299 Catch All 策略、260、262 Chain、122、128 链接请求、122 个字符、35、38、114–115 检查问题、181–182 checkImpl、193 检查策略、15–16、188、193–194 内存块、81–87 chunkSize、88

圆圈、203 班。See also 继承；策略类基础、228 个客户端、153–154 个分解、19–20 个派生、228 个最终、29 个生成、带类型列表、64–75 级锁定操作、187 个本地、28–29 个对象工厂、200–201 个可访问、248–255 个访问者、248–255 个类、200 个 ClassLevelLockable、153、307–309 克隆、30、107、123、164、211–213、230、232、234、284 CloneFactory、214–218 克隆对象工厂、211–215 CLOS、263 columns_、292 COM（组件对象模型）、133、192 命令。See also 命令设计模式活动、102 转发、102 命令、100 命令设计模式、99–104 基本描述、100–102 现实世界、102–103 比较运算符、178–181 编译时断言、23–26 检测可转换性和继承、34–37 CompileTimeChecker、25–26 CompileTimeError、25 COMRefCounted、192 ConcreteCommand、100、102 ConcreteFact、232、234 ConcreteFactory、226–228、230–231、233–234 ConcreteLifetimeTracker、144–145 混凝土产品、222、227 const、44、114–115、182–183 常量映射积分、到类型、29–31 时间多方法、290–293 常规对话框、219–221 转换参数、114–115、285–290 绑定为、119–121 隐式、到原始指针类型、171–173 返回类型、114–114 用户定义、172

Conversion policy, 36–37, 188, 192–193
 convertibility, detecting, at compile time, 35–37

copy

construction, eliding of, 123
 deep, 123, 162–164, 192
 destructive, 168–170
 on write (COW), 165

Copy, 40, 45

copy_backward, 144

copyAlgo, 45

CORBA (Common Object Request Broker Architecture), 115, 133

counting, reference, 165–167

covariant return types, 212–213

Create, 9, 11–12, 14, 31–32, 198, 224, 234

CreateButton, 50

CreateDocument, 199

CreateObject, 208–209

CreateScrollbar, 50

CreateShape, 205–206

CreateShapeCallback, 205

CreateStatic, 153

CreateT, 223

CreateUsingMalloc, 153

CreateUsingNew, 153

CreateWindow, 50

Creation policy, 151, 153

Creator policy, 7–9, 11–14, 149, 154, 156

cyclic

dependencies, 243–248
 references, 168

CyclicVisitor, 255–257, 261

Czarnecki, Krzysztof, 54

D

dead reference problem, 135–142

Deallocate, 82–87

deallocChunk_, 86–87

deep copy, 123, 163–164, 192

DeepCopy, 192

DEFAULT_CHUNK_SIZE, 93–94, 95

default free store allocator, 78

DefaultLifetime, 153

DEFAULT_THREADING, 94

#define preprocessor directive, 93, 139

DEFINE_CYCLIC_VISITABLE, 257

DEFINE_VISITABLE, 252, 254, 256

delete, 12, 89–91, 94, 108, 132, 143, 159, 172–178

delete[], 184, 189–190

DeleteChar, 125

dependency, circular, 141

DependencyManager, 141

Deposit, 306

dereference, checking before, 182

Derived, 90, 197–198

DerivedToFront algorithm, 63–65, 76, 272

design patterns

Abstract Factory pattern, 69, 49–51, 219–234

Command pattern, 99–104

Double-Checked Locking pattern, 146–147, 149

Prototype design pattern, 228–233

Strategy design pattern, 8

Destroy policy, 20

destroyed_, 136, 138

_DestroySingleton, 135

DestructiveCopy, 192

destructors, 12–13

detection, dead-reference, 135–137

Dialog, 219–221

Dijkstra, Edgar, 305–306

DisallowConversion, 192–193

DispatcherBackend policy, 294

DispatchRhs, 269–270

Display, 135–142, 169

DisplayStatistics, 230

do-it-all interface, failure of, 4–5

DocElement, 236–248, 249, 251, 256, 259

DocElementVisitor, 239–248

DocElementVisitor.h, 243–244

DoClone, 213

DoCreate, 223–224, 227, 232

DocStats, 236–237, 240–242, 246–247, 248

Document, 198

DocumentManager, 198–199

DottedLine, 212

double, 306

Double-Checked Locking pattern, 146–147, 149

DoubleDispatch, 267, 268

double dispatcher, logarithmic, 263, 276–285, 297–300

double switch-on-type, 264–268

Dr. Dobb's Journal, 125

Drawing, 201–203

DrawingDevices, 290

DrawingType, 203

Dylan, 263

dynamic_cast, 238, 243, 246, 251, 255, 267, 285–290, 299

cost of, 255–256

DynamicCaster, 288, 289

转换策略, 36–37, 188, 192–193 可转换性, 在编译时检测, 35–37 复制构造, 删除, 123 深度, 123, 162–164, 192 破坏性, 168–170 写入时 (COW), 165 复制, 40, 45 `copy_backward`, 144 `copyAlgo`, 45 CORBA (通用对象请求代理架构)、115、133 计数、引用、165–167 协变返回类型、212–213 创建、9、11–12、14、31–32、198、224、234 `CreateButton`、50 `CreateDocument`、199 `CreateObject`、208–209 `CreateScrollBar`、50 `CreateShape`、205–206 `CreateShapeCallback`、205 `CreateStatic`、153 `CreateT`、223 `CreateUsingMalloc`、153 `CreateUsingNew`、153 `CreateWindow`、50 创建策略、151、153 创建者策略、7–9、11–14、149、154、156 循环依赖项、243–248 参考文献、168 `CyclicVisitor`、255–257、261 Czarnecki、Krzysztof、54

D

死引用问题, 135–142 `Deallocate`、82–87 `deallocChunk_`、86–87 深层复制、123、163–164、192 `DeepCopy`、192 `DEFAULT_CHUNK_SIZE`、93–94、95 默认自由存储分配器、78 `DefaultLifetime`、153 `DEFAULT_THREADING`、94 `#define` 预处理器指令, 93, 139 `DEFINE_CYCLIC_VISITABLE`、257 `DEFINE_VISITABLE`、252, 254, 256 删除, 12, 89–91, 94, 108, 132, 143, 159, 172–178 删除[] , 184, 189–190 删除字符, 125

依赖、循环、141 `DependencyManager`、141 `D eposit`、306 解引用、检查前、182 `Derived`、90、197–198 `DerivedToFront` 算法、63–65、76、272 设计模式抽象工厂模式、69、49–51、219–234 命令模式、99–104 双重检查锁定模式、146–147、149 原型设计模式、228–233 策略设计模式、8 销毁策略、20 `destroy_`、136, 138 `_D estroySingleton`、135 `DestructiveCopy`、192 析构函数、12–13 检测, 死引用、135–137 `Dialog`、219–221 Dijkstra, Edgar、305–306 `DisallowConversion`、192–193 `DispatcherBackend` 策略、294 `DispatchRhs`、269–270 `Display`、135–142、169 `DisplayStatistics`、230 全能接口、故障、4–5 `DocElement`、236–248、249、251、256、259 `DocElementVisitor`、239–248 `DocElementVisitor.h`、243–244 `DoClone`、213 `DoCreate`、223–224、227、232 `DocStats`、236–237、240–242、246–247、248 文档、198 `DocumentManager`、198–199 `DottedLine`、212 双、306 双检查锁定模式、146–147、149 `DoubleDispatch`、267、268 双调度器、对数、263、276–285、297–300 双开关型、264–268 Dr. Dobb's Journal、125 绘图、201–203 `DrawingDevices`、290 `DrawingType`、203 Dylan、263 `Dynamic_cast`、238、243、246、251、255、267、285–290、299 成本、255–256 `DynamicCaster`、288、289

E

EasyLevelEnemyFactory, 227, 229, 231
Effective C++ (Meyers), 132
 efficient resource use, 302
 Eisenecker, Ulrich, 54
 ElementAt, 20
 Ellipse, 203, 271, 273
 else, 238
 EmptyType, 39–40, 48, 110
 encapsulation, 99
 EnforceNotNull, 15, 18
 enum, 45
 equality, 173–178
 erase, 278
 Erase, 58–59, 76
 EraseAll, 76, 59
 error(s)
 messages, compile-time assertions and, 24
 reporting, 181–182
 EventHandler, 71
 events, 71, 309
 exceptions, 209
 Execute, 100, 102
 Executor, 269–272
 exists2Way, 36
 ExtendedWidget, 18, 164

F

factor(ies)
 basic description of, 197–218
 classes and, 200–201
 generalization and, 207–210
 implementing, 201–206
 need for, 198–200
 quick facts, 216–217
 templates, 216–218
 type identifiers and, 206–207
 using, with generic components, 215
 Factory, 207–208, 215–217
 FactoryErrorImpl, 209
 FactoryError policy, 208–209, 217–218, 234
 FactoryErrorPolicy, 217–218, 234
 FastWidgetPtr, 17
 Field, 67–70, 74–75
 Fire, 275, 270, 271
 firstAvailableBlock_, 81–83
 FixedAllocator, 80–81, 84–95
 FnDispatcher, 280–282, 285, 288, 293–294, 299–300

FnDoubleDispatcher, 280
 Foo, 103, 139
 forwarding functions, cost of, 122–124
 free, 143, 189–190
 fun_, 113, 115
 functionality, optional, through incomplete instantiation, 13–14
 functions
 forwarding, cost of, 122–124
 static, singletons and, 130
 functor(s)
 argument type conversions and, 114–115
 basic description of, 99–128
 binding and, 119–121
 chaining requests and, 122
 command design pattern and, 100–102
 double dispatch to, 282–285
 generalized, 99–128
 handling, 110–112
 heap allocation and, 124–125
 implementing Undo and Redo with, 125–126
 multimethods and, 282–285
 quick facts, 126–128
 real-world issues and, 122–125
 return type conversions and, 114–115
 Functor1, 106
 Functor2, 106
 FunctorDispatcher, 283–285, 288, 293, 299–300
 FunctorHandler, 110–112, 117–119, 126
 Functor template, 99–108, 114, 117, 120, 125–126, 215
 FunctorImpl, 107–112, 123–124, 128, 283, 284
 FunctorType, 284
 FunkyDialog, 219–221

G

GameApp, 230
 Gamma, Ralph, 248
 generalization, 207–210. *See also* generalized functors
 generalized functors. *See also* functors
 argument type conversions and, 114–115
 basic description of, 99–128
 binding and, 119–121
 chaining requests and, 122
 Command design pattern and, 100–102
 handling, 110–112
 heap allocation and, 124–125
 implementing Undo and Redo with, 125–126

乙

EasyLevelEnemyFactory, 227, 229, 231
Effective C++ (Meyers), 132 高效资源利用, 3
 02 Eisenecker, Ulrich, 54 ElementAt, 20 Ellipse
 , 203, 271, 273 else, 238 EmptyType, 39–40, 48
 , 110 封装、99 EnforceNotNull、15、18 枚举
 、45 相等、173–178 擦除、278 擦除、58–5
 9、76 EraseAll、76、59 错误消息、编译时
 断言和、24 报告、181–182 EventHandler、7
 1 事件、71、309 异常、209 执行、100、10
 2 执行器、269–272 存在 2Way、36 扩展Wid
 get、18、164

F

因素 基本描述, 197–218 类, 200–201 概括
 , 207–210 实施, 201–206 需求, 198–200
 速览, 216–217 模板, 216–218 类型标识符
 , 206–207 使用通用组件, 215 工厂, 207
 –208、215–217 FactoryErrorImpl、209 Facto
 ryError 策略、208–209、217–218、234 Facto
 ryErrorPolicy、217–218、234 FastWidgetPt
 r、17 字段、67–70、74–75 火灾、275、270
 、271 FirstAvailableBlock_、81–83 固定分
 配器、80–81、84–95 FnDispatcher、280–28
 2、285、288、293–294、299–300

FnDoubleDispatcher、280 Foo、103、139 转发函
 数、成本、122–124 免费、143、189–190 fun_
 113、115 功能、可选, 通过不完整实例化、13–1
 4 函数转发、成本、122–124 静态、单例和 130
 函子参数类型转换和, 114–115 基本描述, 99–1
 28 绑定和, 119–121 链接请求和, 122 命令设计
 模式和, 100–102 双分派到, 282–285 广义, 99
 –128 处理, 110–112 堆分配和, 124–125 实现
 撤消和重做, 125–126 多方法, 282–285 个快速
 事实, 126–128 个实际问题, 122–125 个返回类
 型转换, 114–115 Functor1, 106 Functor2, 106 Fun
 ctorDispatcher, 283–285, 288, 293, 299–300 Funct
 orHandler, 110–112, 117–119、126 Functor 模板、9
 9–108、114、117、120、125–126、215 FunctorI
 mpl、107–112、123–124、128、283、284 Funct
 orType、284 FunkyDialog、219–221

G

GameApp, 230 Gamma, Ralph, 248 泛化, 20
 7–210. *See also* 泛化函子泛化函子. *See also* 函子
 参数类型转换和, 114–115 基本描述, 99–128
 绑定和, 119–121 链接请求和, 122 命令设计
 模式和, 100–102 处理, 110–112 堆分配, 124
 –125 实现撤消和重做, 125–126

- quick facts, 126–128
- real-world issues and, 122–125
- return type conversions and, 114–115
- GenLinearHierarchy, 71–75, 227, 230, 231
- GenScatterHierarchy, 64–75, 223–226, 227–228
- geronimosWork, 117
- GetClassIndex, 292–293
- GetClassIndexStatic, 292
- GetImpl, 162, 173, 183, 190
- GetImplRef, 162, 190
- GetLongevity, 154
- GetPrototype, 12, 14
- Go, 269, 270, 272, 279
- GoF (Gang of Four) book, 100, 122, 125, 199, 235, 248–249, 255–262
- granular interfaces, 224
- GraphicButton, 61–62
- GUI (graphical user interface), 102

H

- handles, 161
- Harrison, Tim, 146
- Haskell, 263
- hat trick, 89–91
- HatchingDispatcher, 272
- HatchingExecutor, 271
- HatchRectanglePoly, 279
- header files
 - DocElementVisitor.h, 243–244
 - Multimethods.h, 294
 - TypeList.h, 51, 55, 75
 - SmallAlloc.h, 93–95
- heaps, 124–125, 189–190
- HeapStorage, 189–190
- hierarchies
 - linear, 70–74
 - scattered, 64–70
- HTMLDocument, 198

I

- IdentifierType, 209, 213, 215
- IdToProductMap, 214
- #ifdef preprocessor directive, 138–139
- if-else statements, 29, 267, 268, 270
- if statements, 238, 267, 305
- IMPLEMENT_INDEXABLE_CLASS, 293

- implicit conversion, to raw pointer types, 171–173
- IncrementFontSize, 240, 241
- indexed access, 55
- IndexOf, 56, 76, 275
- inequality, 173–178
- inheritance
 - detecting, at compile time, 34–37
 - logarithmic dispatcher and, 279
 - multiple, 5–6
- INHERITS, 37
- InIt, 40, 82
- initialization
 - checking, 181–182
 - dynamic, 132
 - lazy, 182
 - object factories and, 197
 - static, 132
- insert, 205
- InsertChar, 120, 122, 125–126
- Instance, 131–132, 135–137, 146, 151
- instantiation, 13–14, 120, 272, 274
- int, 159
- Int2Type, 29–31, 68–69
- interface(s)
 - Abstract Factory design pattern, 223–226
 - application program (APIs), 161, 306
 - do-it-all, failure of, 4–5
 - granular, 224
 - graphical user (GUI), 102
 - separation, 101
- intrusive reference counting, 167. *See also* reference counting
- IntType, 186, 304
- InvocationTraits, 275
- isConst, 47
- isPointer, 47
- isReference, 41, 47
- isStdArith, 47
- isStdFloat, 47
- isStdFundamental, 42–43, 47
- isStdIntegral, 47
- isStdSignedInt, 47
- isStdUnsignedInt, 47
- isVolatile, 47

K

- KDL problem, 135–142, 155
- Keyboard, 135–142, 155
- KillPhoenixSingleton, 138
- Knuth, Donald E., 77, 78

速览、126–128 个实际问题、122–125 个返回
类型转换和 114–115 GenLinearHierarchy、71–75
、227、230、231 GenScatterHierarchy、64–75、
223–226、227–228 geronimosWork、117 GetClas
sIndex、292–293 GetClassIndexStatic、292 GetI
mpl、162、173、183、190 GetImplRef、162、19
0 GetLongevity、154 GetPrototype、12、14 Go、
269、270、272、279 GoF（四人帮）书、100、1
22、125、199、235、248–249、255–262 粒度
界面、224 GraphicButton、61–62 GUI（图形用
户界面）、102

H

句柄、161 Harrison、Tim、146
Haskell、263 帽子戏法、89–91
HatchingDispatcher、272 Hatchin
gExecutor、271 HatchRectangleP
oly、279 头文件 DocElementVisit
or.h、243–244 Multimethods.h、2
94 Typelist.h、51、55、75 Small
Alloc.h、93–95 堆、124–125、1
89–190 HeapStorage、189–190 层
次结构线性、70–74 分散、64–7
0 HTMLDocument、198

我

IdentifierType、209、213、215 IdToPro
ductMap、214 #ifdef 预处理器指令、13
8–139 if-else 语句、29、267、268、270
if 语句、238、267、305 IMPLEMENT_I
NDEXABLE_CLASS、293

隐式转换，到原始指针类型、171–173 Incr
ementFontSize、240、241 索引访问、55 Ind
exOf、56、76、275 不等式、173–178 继承检
测，编译时、34–37 对数调度程序、279 倍
数、5–6 INHERITS、37 InIt、40、82 初始
化检查、181–182 动态、132 惰性、182 对
象工厂和、197 静态、132 插入、205 Insert
Char、120、122、125–126 实例、131–132
、135–137、146、151 实例化、13–14、12
0、272、274 int、159 Int2Type、29–31、68–6
9 接口 抽象工厂设计模式、223–226 应用程
序 (API)、161、306 全能、失败、4–5 粒度、224
图形用户 (GUI)、102 分离、101 侵入式引用
计数、167. *See also* 引用计数 IntType、186、
304 调用特征、275 isConst、47 isPointer
、47 isReference、41、47 isStdArith、47
isStdFloat、47 isStdFundamental、42–43、
47 isStdIntegral、47 isStdSignedInt、47 isS
tdUnsignedInt、47 isVolatile、47

K

KDL 问题、135–142、155 键
盘、135–142、155 KillPhoenix
Singleton、138 Knuth、Donald
E.、77、78

L

Lattanzi, Len, 77
 Length, 76
 less, 180–181
 LhsTypes, 272
 Lifetime policy, 149–153
 LifetimeTracker, 142–143
 LIFO (last in, first out), 134
 Line, 203, 204, 212–213
 linking, reference, 167–168
 LISP, 52
 ListOfTypes, 295
 Lock, 146, 184, 306
 LockedStorage, 189–190
 locking
 class-level, 307
 object-level, 306–308
 pattern, double-checked, 146–147, 149
 semantics, 306–308
 LockingProxy, 184–185
 Log, 135–142
 logarithmic double dispatcher, 263, 276–285,
 297–300
 logic_error, 152
 Loki, 70, 77, 91–92, 210, 303, 309
 multimethods and, 263, 268, 277–278, 288,
 295–300
 mutexes and, 306
 smart pointers and, 163
 long double data type, 35
 longevity, 139–145, 149, 151
 lower_bound, 277

M

MacroCommand, 122
 macros, 122, 251–252, 254, 256–257, 261
 “maelstrom effect,” 170
 MAKE_VISITABLE, 261
 MakeAdapter, 28–29
 MakeCopy, 164
 MakeT, 223
 malloc, 143
 map, 204–205, 210, 277, 278
 mapping
 integral constants to types, 31–32
 type-to-type, 31–33
 Martin, Robert, 245
 maxObjectSize, 88
 MAX_SMALL_OBJECT_SIZE, 93–94, 95

MemControlBlock, 78–79
 MemFunHandler, 117–119, 126
 memory
 allocators, workings of, 78–80
 chunks of, 81–87
 heaps, 124–125, 189–190
 RMW (read-modify-write) operation, 303–304
 Meyers, Scott, 77, 133–134, 276, 280
 Meyers singleton, 133–134
 ML, 263
 ModalDialog, 27
 Monster, 219–222, 224, 229
More Effective C++ (Meyers), 276, 280
 MostDerived, 63, 76
 multimethods
 arguments and, 285–290
 basic description of, 263–300
 constant-time, 290–293
 double switch-on-type and, 265–268
 logarithmic double dispatcher and, 276–285
 need for, 264–265
 quick facts, 297–300
 symmetry and, 273–74
 Multimethods.h, 294
 MultiThreaded, 6
 MultiThreadedRefCounting, 186–187
 multithreading, 145–148, 302–306
 at the bookkeeping data level, 185–187
 critique of, 302–303
 library, 301–309
 mutexes and, 305–306
 at the pointee object level, 184–185
 reference counting and, 186
 reference tracking and, 186–187
 smart pointers and, 184–187
 mutex_, 146
 mutexes, 146–147, 305–306
 MyController, 27
 MyOnlyPrinter, 130
 MyVisitor, 257

N

name, 206
 name cyclic dependency, 243
 new[], 183
 next_, 187
 NiftyContainer, 28–30, 33–34
 NoChecking, 15, 18–19
 NoCopy, 192
 NoDestroy, 153
 NoDuplicates, 60–61, 76

L

Lattanzi, Len, 77 长度、76 少、180–181 LhsTypes、272 生命周期策略、149–153 LifetimeTracker、142–143 LIFO（后进先出）、134 行、203、204、212–213 链接、参考、167–168 LISP、52 ListOfTypes、295 Lock、146、184、306 LockedStorage、189–190 锁定类级别、307 对象级别、306–308 模式、双重检查、146–147、149 语义、306–308 LockingProxy、184–185 Log、135–142 对数双调度程序、263、276–285、297–300 逻辑错误、152 Loki、70、77、91–92、210、303、309 多方法和、263、268、277–278、288、295–300 互斥体和、306 智能指针和、163 long double 数据类型、35 寿命、139–145、149、151 lower_bound、277

MemControlBlock、78–79 MemFunHandler、117–119、126 内存分配器、工作原理、78–80 块、81–87 堆、124–125、189–190 RMW（读-修改-写）操作、303–304 Meyers、Scott、77、133–134、276、280 Meyers 单例、133–134 ML、263 ModalDialog、27 Monster、219–222、224、229 *More Effective C++* (Meyers)、276、280 MostDerived、63、76 多方法参数以及、285–290 的基本描述、263–300 恒定时间、290–293 双开关型和、265–268 对数双调度程序和、276–285 需要、264–265 快速事实、297–300 对称性和、273–74 Multimethods.h、294 MultiThreaded、6 MultiThreadedRefCounting、186–187 多线程、145–148、302–306 簿记数据级别、185–187 批判、302–303 库、301–309 互斥体、305–306 受指点对象级别、184–185 引用计数和、186 引用跟踪和、186–187 智能指针和、184–187 mutex_、146 互斥体、146–147、305–306 MyController、27 MyOnlyPrinter、130 MyVisitor、257

中号

MacroCommand、122 个宏、122、251–252、254、256–257、261 “漩涡效应”、170 MAKE_VISITABLE、261 MakeAdapter、28–29 MakeCopy、164 MakeT、223 malloc、143 地图、204–205、210、277、278 将整型常量映射到类型、31–32 类型到类型、31–33 Martin、Robert、245 maxObjectSize、88 MAX_SMALL_OBJECT_SIZE、93–94、95

氮

name、206 名称循环依赖、243 new[], 183 next_、187 NiftyContainer、28–30、33–34 NoChecking、15、18–19 NoCopy、192 NoDestroy、153 NoDuplicates、60–61、76

NonConstType, 47
 nontemplated operators, 176–177
 NonVolatileType, 47
 NoQualifiedType, 47
 NullType, 39–41, 48, 52, 54–56, 62–63, 270

O

object factor(ies)
 basic description of, 197–218
 classes and, 200–201
 generalization and, 207–210
 implementing, 201–206
 need for, 198–200
 quick facts, 216–217
 templates, 216–218
 type identifiers and, 206–207
 using, with generic components, 215
 ObjectLevelLockable, 307–309
 OnDeadReference, 136–139, 150
 OnError, 272
 OnEvent, 70–71
 OnUnknownVisitor, 260, 262
 operators
 operator.*, 104, 116–117
 operator!=, 174, 176, 178
 operator(), 103–113, 116, 122–127, 283, 285
 operator→, 157, 160–162, 165, 182–185
 operator→*, 104, 116–117
 operator*, 157, 161–162, 178–179, 182
 operator<, 144
 operator<=, 178–179
 operator=, 162
 operator==, 176–177, 178
 operator>, 178–179
 operator>=, 178–179
 operator[], 55, 183, 278
 operator T*, 172
 OpNewCreator, 10
 OpNewFactoryUnit, 226, 227, 231, 234
 OrderedTypeInfo, 217, 276–277
 orthogonal policies, 20
 OutIt, 40
 ownership-handling strategies, 163–170
 Ownership policy, 183–184, 186, 188, 190–192

P

Paragraph, 237, 241, 247, 249, 254, 256
 ParagraphVisitor, 246, 247, 248, 251

parameter(s)
 template, 10–11, 64, 105
 types, optimized, 43–44
 ParameterType, 43–44, 47, 123
 ParentFunctor, 111, 120–121
 Parm1, 111
 Parm2, 111
 ParmN, 109
 Parrot, 117–119
 pattern(s)
 Abstract Factory pattern, 69, 49–51, 219–234
 Command pattern, 99–104
 Double-Checked Locking pattern, 146–147, 149
 Prototype design pattern, 228–233
 Strategy design pattern, 8
Pattern Hatching (Vlissides), 133
 pData_, 86
 pDocElem, 246
 pDuplicateShape, 212
 pDynObject, 140
 pFactory_, 222
 Phoenix Singleton, 137–142, 149, 153
 pimpl idiom, 78
 pInstance_, 131–132, 136, 138, 146–148, 151
 placement new, 138
 pLastAlloc_, 89
 POD (plain old data) structure, 45–46
 Point3D, 70
 pointee_, 160, 161, 178, 183
 PointeeType, 41–42, 47, 160–161
 pointee object level, 184–185
 pointer(s)
 address-of operator and, 170–171
 arrays and, 183–184
 basic description of, 157–195
 checking issues and, 181–182
 copy on write (COW) and, 165
 deep copy and, 163–164
 destructive copy and, 168–170
 equality and, 173–178
 error reporting and, 181–182
 failure of the do-it-all interface and, 5
 handling, to member functions, 115–119
 implicit conversion and, 171–173
 inequality and, 173–178
 multithreading and, 184–187
 ordering comparison operators and, 178–181
 ownership-handling strategies, 163–170
 quick facts, 194–195
 raw, 171–173
 reference counting and, 165–167, 186
 reference linking and, 166–168
 reference tracking and, 186–187

NonConstType、47 个非模板化运算符、176–177 NonVolatileType、47 NoQualifiedType、47 NullType、39–41、48、52、54–56、62–63、270

氧

对象因素 197–218 类的基本描述, 200–201 泛化, 207–210 实施, 201–206 需求, 198–200 速览, 216–217 模板, 216–218 类型标识符, 206–207 使用通用组件, 215 ObjectLevelLockable, 307–309 OnDeadReference, 136–139, 150 OnError, 272 OnEvent, 70–71 OnUnknownVisitor, 260, 262 运算符operator.*, 104, 116–117 运算符!=, 174, 176, 178 运算符(), 103–113, 116, 122–127、283、285 运算符->, 157、160–162、165、182–185 运算符->*, 104、116–117 运算符*、157、161–162、178–179、182 运算符<, 144 运算符<=、178–179 运算符=、162 运算符==、176–177、178 运算符>, 178–179 运算符>=、178–179 运算符[], 55、183、278 运算符T*、172 OpNewCreator、10 OpNewFactoryUnit、226、227、231, 234 OrderedTypeInfo, 217, 276–277 正交策略, 20 OutIt, 40 所有权处理策略, 163–170 所有权策略, 183–184, 186, 188, 190–192

参数模板, 10–11, 64, 105 类型, 优化, 43–44 ParameterType, 43–44, 47, 123 ParentFunctor, 111, 120–121 Parm1, 111 Parm2, 111 ParmN, 109 Parrot, 117–119 模式 抽象工厂模式, 69, 49–51、219–234 命令模式、99–104 双重检查锁定模式、146–147、149 原型设计模式、228–233 策略设计模式、8 Pattern Hatching (Vlissides)、133 pData_、86 pDocElem、246 pDuplicateShape、212 pDynObject、140 pFactory_、222 Phoenix Singleton、137–142、149、153 pimpl idiom、78 pInstance_、131–132、136、138、146–148、151 新放置、138 pLastAlloc_、89 POD (纯旧数据) 结构、45–46 Point3D、70 pointee_、160, 161, 178, 183 PointeeType, 41–42, 47, 160–161 pointee 对象级别, 184–185 指针地址运算符 and, 170–171 数组 and, 183–184 基本描述, 157–195 检查问题 and, 181–182 写时复制 (COW) 和, 165 深度复制和, 163–164 破坏性复制和, 168–170 相等和, 173–178 错误报告和, 181–182 全能接口失败和, 5 成员函数的处理, 115–119 隐式转换和, 171–173 不等式和, 173–178 多线程 and, 184–187 排序比较运算符 and, 178–181 所有权处理策略, 163–170 速览, 194–195 raw, 171–173 引用计数 and, 165–167, 186 引用链接 and, 166–168 引用跟踪 and, 186–187

磷

Paragraph, 237, 241, 247, 249, 254, 256 ParagraphVisitor, 246, 247, 248, 251

- traits of, implementing, 41–42
- types, implicit conversion, 171–173
- `PointerToObj`, 118
- `PointerTraits`, 41–42
- `PointerType`, 160, 190–191
- policies. *See also* policy classes
 - basic description of, 3, 7–11
 - `BasicDispatcher` and, 293–294
 - `BasicFastDispatcher` and, 293–294
 - compatible, 17–18
 - decomposing classes in, 19–20
 - enriched, 12
 - multimethods and, 293–294
 - noncompatible, 17–18
 - orthogonal, 20
 - singletons and, 149–150, 152–153
 - stock, 152–153
- policies (listed by name). *See also* policies
 - Array policy, 19–20
 - `CastingPolicy` policy, 288, 299
 - `CatchAll` Policy, 260, 262
 - Checking policy, 15–16, 188, 193–194
 - Conversion policy, 36–37, 188, 192–193
 - Creation policy, 151, 153
 - Creator policy, 8–9, 11–14, 149, 155, 156
 - Destroy policy, 20
 - `DispatcherBackend` policy, 294
 - `FactoryError` policy, 208–209, 217–218, 234
 - Lifetime policy, 149–153
 - Ownership policy, 183–184, 186, 188, 190–192
 - Storage policy, 17, 185, 188, 189–190
 - Structure policy, 16–17
 - `ThreadingModel` policy, 16, 149, 151–153, 186, 303–309
- policy classes. *See also* classes
 - basic description of, 3–22
 - combining, 14–16
 - customizing structure with, 16–17
 - destructors of, 12–13
 - implementing, 10–11
- `Poly`, 271, 279
- `Polygon`, 203, 212
- polymorphism, 78, 163–164
 - `Abstract Factory` implementation and, 228–229
 - multimethods and, 264
 - object factories and, 197–200, 211–212
- `prev_`, 187
- `Printer`, 161–162
- `printf`, 105
- `printingPort_`, 130
- `priority_queue`, 142–145

- `ProductCreator`, 210–211, 216–217
- Prototype design pattern, 228–233
- `PrototypeFactoryUnit`, 231–232, 234
- prototypes, 228–233
- `pTrackerArray`, 143

Q

- qualifiers, stripping, 44

R

- `RasterBitmap`, 237, 241, 247, 249, 256
- `RasterBitmapVisitor`, 247, 251
- `realloc`, 143
- `Receiver`, 100
- `Rectangle`, 267, 279, 289
- `RectanglePoly`, 279
- `Redo`, 125–126
- `RefCounted`, 6, 192
- `RefCountedMT`, 192
- reference(s)
 - counting, 165–167, 186
 - linking, 167–168, 186–187
- `ReferencedType`, 41–42, 47
- `RefLinked`, 192
- `RegisterShape`, 206
- `RejectNull`, 194
- `RejectNullStatic`, 193–194
- `RejectNullStrict`, 194
- `Release`, 161–162, 192, 305, 306
- `Replace`, 76
- `ReplaceAll`, 61, 76
- `Reset`, 162
- resource leaks, 133
- `Result`, 55
- `ResultType`, 111, 269, 284
- return type(s)
 - conversion, 114–115
 - covariant, 228
 - generalized functors and, 114–115
- RISC processors, 148
- RMW (read-modify-write) operation, 303–304
- `RoundedRectangle`, 272, 286–287, 289
- `RoundedShape`, 286–287, 289
- `runtime_error`, 137, 209–210, 300
- runtime type information (RTTI), 215, 245, 255, 276

特征, 实现, 41-42 类型, 隐式转换, 171-173
 3 PointerToObj, 118 PointerTraits, 41-42 PointerType, 160, 190-191 策略. *See also* 策略类的基本描述, 3, 7-11 BasicDispatcher 和, 293-294 BasicFastDispatcher 和, 293-294 兼容, 17-18 个分解类, 19-20 个丰富的、12 个多方法和、293-294 个不兼容、17-18 个正交、20 个单例和、149-150、152-153 个库存、152-153 个策略 (按名称列出). *See also* 策略数组策略、19-20 CastingPolicy 策略、288, 299 CatchAll 策略、260, 262 检查策略、15-16, 188, 193-194 转换策略、36-37, 188, 192-193 创建策略、151, 153 创建者策略、8-9, 11-14, 149, 155, 156 销毁策略, 20 DispatcherBackend 策略、294 FactoryError 策略、208-209、217-218、234 生命周期策略、149-153 所有权策略、183-184、186、188、190-192 存储策略、17、185、188、189-190 结构策略、16-17 ThreadingModel 策略、16、149、151-153、186、303-309 策略类. *See also* 类的基本描述、3-22 组合、14-16 自定义结构、16-17 析构函数、12-13 实现、10-11 Poly、271、279 Polygon、203、212 多态性、78, 163-164 抽象工厂实现 and, 228-229 多方法 and, 264 对象工厂 and, 197-200, 211-212 prev_, 187 打印机, 161-162 printf, 105 printingPort_, 130 priority_queue, 142-145

ProductCreator, 210-211, 216-217 原型设计模式, 228-233 PrototypeFactoryUnit, 231-232, 234 原型, 228-233 pTrackerArray, 143

问

预选赛、剥离、44

右

RasterBitmap、237、241、247、249、256 RasterBitmapVisitor、247、251 realloc、143 接收器、100 矩形、267、279、289 RectanglePoly、279 重做、125-126 RefCounted、6、192 RefCountedMT、192 引用计数、165-167、186 链接、167-168、186-187 ReferencedType、41-42、47 RefLinked、192 RegisterShape、206 RejectNull、194 RejectNullStatic、193-194 RejectNullStrict、194 发布、161-162、192、305、306 Replace、76 ReplaceAll、61、76 Reset、162 资源泄漏、133 Result、55 ResultType、111、269、284 返回类型转换、114-115 协变、228 广义函子和、114-115 RISC处理器、148 RMW (读取-修改-写入) 操作、303-304 RoundedRectangle、272、286-287、289 RoundedShape、286-287、289 运行时错误、137、209-210、300 运行时类型信息 (RTTI)、215、245、255

S

- safe_reinterpret_cast, 26
- SafeWidgetPtr, 17
- sameType, 36
- Save, 201–203
- scanf, 105
- ScheduleDestruction, 149–150
- Schmidt, Douglas, 146–147
- Scroll, 122
- ScrollBar, 50, 61–62
- Secretary, 5
- Section, 254
- Select, 33–34, 62
- semantics
 - failure of the do-it-all interface and, 4–5
 - functors and, 99
 - locking, 306–308
- semaphores, 309
- SetLongevity, 140–145, 149
- SetPrototype, 12, 14, 232
- Shape, 201–202, 265, 268, 279, 286–289
- ShapeCast, 289
- ShapeFactory, 204–205, 215
- Shapes, 290
- SillyMonster, 219–222, 226
- SillySoldier, 219–222, 230
- SillySuperMonster, 219–222
- SingleThreaded, 153, 308
- singleton(s). *See also* SingletonHolder
 - basic C++ idioms supporting, 131–132
 - dead reference problem and, 135–142
 - destroying, 133–135
 - Double-Checked Locking pattern and, 146–147
 - failure of the do-it-all interface and, 4–5
 - implementing, 129–154
 - longevity and, 139–145
 - Meyers singleton, 133–134
 - multithreading and, 145–148
 - Phoenix, 137–142, 149, 153
 - static functions and, 130
 - uniqueness of, enforcing, 132–133
- SingletonHolder, 3, 91, 129–130, 148–153, 215.
 - See also* singletons
 - assembling, 150–152
 - decomposing, 149–150
 - quick facts, 155–156
 - working with, 153–156
- SingletonWithLongevity, 153, 154
- sizeof, 25, 34–35, 82, 90–91
- size_t, 36, 79
- skinnable programs, 103
- SmallAlloc.h, 93–95
- small-object allocation
 - basic description of, 77–96
 - default free store allocators and, 78
 - fixed-size allocators and, 84–87
 - hat trick and, 89–91
 - memory chunks and, 81–84
 - quick facts, 94–95
- SmallObjAllocator, 80–81, 87–89, 92–95
- SmallObject, 80–81, 89, 91–95, 123–124
- smart pointer(s)
 - address-of operator and, 170–171
 - arrays and, 183–184
 - basic description of, 157–195
 - checking issues and, 181–182
 - copy on write (COW) and, 165
 - deep copy and, 163–164
 - destructive copy and, 168–170
 - equality and, 173–178
 - error reporting and, 181–182
 - failure of the do-it-all interface and, 5
 - implicit conversion and, 171–173
 - inequality and, 173–178
 - multithreading and, 184–187
 - ordering comparison operators and, 178–181
 - ownership-handling strategies, 163–170
 - quick facts, 194–195
 - raw, 171–173
 - reference counting and, 165–167, 186
 - reference linking and, 167–168
 - reference tracking and, 186–187
 - types, implicit conversion, 171–173
- SmartPtr, 3, 6–7, 14–19, 44, 87, 157–195
- Soldier, 219–222, 224, 229–230
- SomeLhs, 278, 284
- SomeRhs, 278, 284
- SomeThreadingModel, 309
- SomeVisitor, 250, 254
- spImpl_, 112
- statements
 - if, 238, 267, 305
 - if-else, 29, 267, 268, 270
- static, 280
- static_cast, 114, 285–290, 299
- STATIC_CHECK, 25
- StaticDispatcher, 268–276, 297–298
- static manipulation, 6
- Statistics, 249
- stats, 246–247
- STL, 115, 171
- StorageImpl, 189, 190
- Storage policy, 17, 185, 188, 189–190
- Strategy design pattern, 8

S

safe_reinterpret_cast、26 SafeWidgetPtr、17 same Type、36 保存、201–203 scanf、105 ScheduleDescription、149–150 Schmidt、Douglas、146–147 滚动、122 ScrollBar、50、61–62 秘书、5 部分、254 选择、33–34、62 语义失败的全能接口、4–5 个函子和、99个锁定、306–308个信号量、309 SetLongevity、140–145、149 SetPrototype、12、14、23 2 Shape、201–202、265、268、279、286–289 ShapeCast、289 ShapeFactory、204–205、215 Shapes、290 SillyMonster、219–222、226 SillySoldier、219–222、230 SillySuperMonster、219–222 SingleThreaded、153、308 singleton(s)。See also SingletonHolder 基本 C++ 惯用法支持、131–132 死引用问题、135–142 破坏、133–135 双重检查锁定模式、146–147 万能接口失败、4–5 实现、129–154 寿命、139–145 Meyers 单例、133–134 多线程、145–148 Phoenix、137–142、149、153 静态函数和、130 唯一性、执行、132–133 SingletonHolder、3、91、129–130、148–153、215。See also 单例组装、150–152 分解、149–150 快速事实、155–156 使用、153–156 SingletonWithLongevity、153、154 sizeof、25、34–35、82、90–91 size_t、36、79 可换肤程序、103

SmallAlloc.h、93–95 小对象分配基本描述、77–96 默认自由存储分配器和、78 固定大小分配器和、84–87 帽子戏法和、89–91 内存块和、81–84 快速事实、94–95 SmallObjAllocator、80–81、87–89、92–95 SmallObject、80–81、89、91–95、123–124 智能指针地址运算符 and、170–171 数组和、183–184 基本描述、157–195 检查问题和、181–182 写入时复制(COW)和、165 深度复制和、163–164 破坏性复制和、168–170 等式和、173–178 错误报告和、181–182 万能接口失败和、5 隐式转换和、171–173 不等式和、173–178 多线程和、184–187 排序比较运算符和、178–181 所有权处理策略、163–170 速览、194–195 原始、171–173 引用计数和、165–167、186 引用链接和、167–168 引用跟踪和、186–187 类型、隐式转换、171–173 SmartPtr、3、6–7、14–19、44、87、157–195 士兵、219–222、224、229–230 SomeLhs、278、284 SomeRhs、278、284 SomeThreadingModel、309 SomeVisitor、250、254 spImpl_、112 条语句 if、238、267、305 if-else、29、267、268、270 static、280 static_cast、114、285–290、299 STATIC_CHECK、25 StaticDispatcher、268–276、297–298 静态操作、6 统计、249 统计、246–247 STL、115、171 StorageImpl、189、190 存储策略、17、185、188、189–190 策略设计模式、8

String, 302–303, 307
 Stroustrup, Bjarne, 202
 struct, 45
 structure(s)
 customizing, with policy classes, 16–17
 POD (plain old data), 45–46
 specialization of, 7
 Structure policy, 16–17
 SuperMonster, 219–222, 229
 SUPERSUBCLASS, 37, 62
 Surprises.cpp, 224
 Sutter, Herb, 77
 switch, 203
 SwitchPrototype, 13–14
 symmetry, 273–74
 synchronization objects, 303

T

template(s)
 advantages of, 6–7
 implementing policy classes with, 11
 skeleton, Functor class, 104–108
 specialization, partial, 53–54
 template parameters, 10–11, 64, 105
 templated operators, 176–177
 TemporarySecretary, 5
 Tester, 178
 Tester*, 178
 TestFunction, 113–115
 TextDocument, 198
 ThreadingModel policy, 16, 149, 151–153, 186, 303–309
 time
 separation, 101
 slicing, 201
 TList, 53–65, 75, 120, 127, 223, 231–234, 261
 Tools menu, 102
 trampoline functions (thunks), 280–283
 Trampoline, 281
 translation units, 132
 Triangle, 289
 tuples, generating, 70
 type(s)
 atomic operations on, 303–304
 conversions, generalized functors and, 114–115
 detection of fundamental, 42–43
 identifiers, 201, 206–207
 integral, 303–305
 modifiers, 308–309
 multiple inheritance and, 6

 multithreading and, 303–305
 parameters, optimized, 43–44
 replacing an element in, 60–61
 safety, loss of static, 4
 selection, 33–34
 -to-type mapping, 31–33
 traits, 38–46
 Type2Type, 32–33, 223
 TypeAt, 55, 76
 TypeAtNonStrict, 55, 76, 109
 typedef, 14, 19, 51, 54, 215, 295
 typeid, 38, 267
 type_info, 37–39, 54, 206–207, 213–215, 276–277, 295
 TypeInfo, 38–39, 48
 TYPELIST, 295
 Typelist.h, 51, 55, 75
 typelists, 223, 268, 272
 appending to, 57–58
 basic description of, 49–76
 calculating length and, 53–54
 class generation with, 64–75
 compile-time algorithms operating on, 76
 creating, 52–53
 defining, 51–52
 detecting fundamental types and, 42–43
 need for, 49–41
 partially ordering, 61–64
 quick facts, 75
 searching, 56–57
 TypesLhs, 269–270, 274
 TypesRhs, 269, 270, 274
 TypeTraits, 41–48, 123, 183

U

UCHAR_MAX, 83
 Undo, 125–126
 unique bouncing virtual functions, 238
 UnLock, 184
 UpdateStats, 237–238
 upper_bound, 144
 use_Size, 91

V

ValueType, 33
 vector, 183
 VectorGraphic, 244

字符串、302–303、307 Stroustrup、Bjarne、202 结构、45 个结构定制、带策略类、16–17 POD（普通旧数据）、45–46 专门化、7 结构策略、16–17 SuperMonster、219–222、229 SUPERSUBCLASS、37、62 Surprises.cpp、224 Sutter、Herb、77 switch、203 SwitchPrototype、13–14 对称性、273–74 同步对象、303

时间

模板的优点、6–7 实现策略类、11 骨架、Functor 类、104–108 专业化、部分、53–54 个模板参数、10–11、64、105 个模板化运算符、17 6–177 TemporarySecretary、5 Tester、178 Tester*、178 TestFunction、113–115 TextDocument、198 ThreadingModel 策略、16、149、151–153、186、303–309 时间分隔、101 切片、201 TList、53–65、75、120、127、223、231–234、261 工具菜单、102 个蹦床功能（重击）、280–283 蹦床、281 个翻译单元、132 个三角形、289 个元组、生成、70 个类型原子操作、303–304 个转换、广义函子和、114–115 个基本检测、42–43 个标识符、201、206–207 个积分、303–305 修饰符、308–309 多重继承和、6

多线程、303–305 个参数、优化、43–44 替换元素、60–61 安全性、静态丢失、4 选择、33–34 到类型映射、31–33 特征、38–46 Type2Type、32–33、223 TypeAt、55、76 TypeAtNonStrict、55、76、109 typedef、14、19、51、54、215、295 typeid、38、267 type_info、37–39、54、206–207、213–215、276–277、295 TypeInfo、38–39、48 TYPELIST、295 Typelist.h、51、55、75 类型列表、223、268、272 附加、57–58 基本描述、49–76 计算长度、53–54 类生成、64–75 编译时算法操作、76 创建、52–53 定义、51–52 检测基本类型、42–43 需要、49–41 部分排序、61–64 快速事实、75 搜索、56–57 TypesLhs、269–270、274 TypesRhs、269、270、274 TypeTraits、41–48、123、183

U

UCHAR_MAX、83 撤消、125–126 个唯一弹跳虚拟函数、238 个解锁、184 个 UpdateStats、237–238 upper_bound、144 use_Size、91

V

值类型、33 矢量、183 矢量图形、244

VectorizedDrawing, 259
 Veldhuizen, Todd, 54
 virtual
 constructor dilemma, 229
 functions, 238
 Visit, 249–250, 254, 256
 Visitable, 249, 251, 252–254
 Visitor design pattern
 acyclic, 243–248
 basic description of, 235–262
 catch-all function and, 242–243, 258–259
 cyclic, 243–248, 254–257
 generic implementation of, 248–255
 hooking variations, 258–260
 nonstrict visitation and, 261
 quick facts, 261–262
 VisitParagraph, 240, 242
 VisitRasterBitmap, 242
 VisitVectorGraphic, 244
 Vlissides, John, 133
 void*, 172

volatile, 151, 308–309, 308–309
 VolatileType, 151, 309

W

Widget, 26–32, 38, 44, 61–62, 159, 164, 184,
 309
 WidgetEventHandler, 71–74
 WidgetFactory, 49–51
 WidgetInfo, 65–67
 WidgetManager, 9–14, 19–20
 Window, 50
 Withdrawal, 306

X

X Windows, 103

VectorizedDrawing, 259 Veldhuizen, Todd, 5
4 虚拟构造函数困境, 229 函数, 238 访问, 2
49–250, 254, 256 可访问, 249, 251, 252–25
4 非循环访问者设计模式, 243–248 基本描
述, 235–262 包罗万象的函数, 242–243, 25
8–259 循环, 243–248, 254–257 通用实现, 2
48–255 挂钩变体, 258–260 非严格访问, 26
1 快速事实, 261–262 VisitParagraph, 240, 2
42 VisitRasterBitmap, 242 VisitVectorGraph
ic, 244 约翰·弗利塞德斯, 133 无效*, 172

挥发性, 151, 308–309, 308–309
挥发性类型, 151, 309

瓦

Widget, 26–32、38、44、61–62、159、16
4、184、309 WidgetEventHandler, 71–74
WidgetFactory, 49–51 WidgetInfo, 65–67
WidgetManager, 9–14、19–20 窗口、50
提款、306

X

X 窗口, 103